Smart Network Interface Card Packet Classification Accelerator

By:

Uchenna Obikwelu, 101241887
Ben Narmer, 101192825

Final Report

SYSC 4320A

Case Studies in Computer Systems

Department of Systems and Computer Engineering
Faculty of Engineering
Carleton University

April 16, 2026

Table Of Contents

1. Introduction

1.1 Problem Definition

High-speed networks demand efficient packet classification to support real-time data processing. Software-based packet inspection introduces latency and cannot sustain the throughput required by modern data centers, cloud services, and high-performance computing systems. As network speeds continue to increase, traditional CPU-based approaches become a performance bottleneck.

Field-Programmable Gate Arrays (FPGAs) provide a compelling solution by enabling parallel processing, deterministic performance, and hardware-level acceleration. Unlike general-purpose processors, FPGAs can be tailored to execute packet classification at line rate while maintaining low latency and high energy efficiency.

However, designing high-performance FPGA-based systems presents several challenges. Engineers must balance throughput, latency, and hardware utilization while ensuring timing correctness. Meeting setup and hold constraints and achieving timing closure through Static Timing Analysis (STA) are critical to ensuring reliable system operation.

This project addresses these challenges by developing an FPGA-based Smart Network Interface Card (SmartNIC) packet classification accelerator capable of processing packets in real time.

1.2 Proposed Solution

To overcome the limitations of software-based packet classification, this project proposes a hardware-accelerated SmartNIC implemented on an FPGA. SmartNICs offload network processing tasks from the CPU onto dedicated hardware accelerators, thereby improving system performance, reducing latency, and freeing computational resources for other applications.

The proposed SmartNIC inspects key fields from simplified IPv4 headers, specifically the destination IP address and protocol, and matches them against predefined rules to determine whether packets should be allowed or dropped.

To evaluate performance trade-offs, two optimized architectures were developed:

- **High-Throughput Architecture:** Utilizes pipelining and parallel comparisons to achieve single-cycle packet classification.

- **Area-Optimized Architecture:** Employs resource sharing and sequential rule evaluation to minimize hardware utilization.

Both architectures were implemented in Verilog HDL, synthesized and analyzed using Xilinx Vivado, and validated through simulation. The high-throughput design was also validated through FPGA deployment on a Zynq-7000 Zybo-10 platform. Performance metrics including timing, resource utilization, and power consumption were evaluated to compare the two designs.

In addition, the system incorporates a custom-designed asynchronous FIFO to safely transfer packet data across different clock domains. In practical FPGA systems, the write and read operations often operate at different clock frequencies, requiring reliable clock domain crossing (CDC) to prevent data corruption and metastability. By implementing the FIFO from scratch, the design provides full control over pointer logic, synchronization mechanisms, and buffering behavior. This is especially important for the area-optimized architecture, where unnecessary logic must be avoided to minimize resource usage, while still ensuring correct and efficient data transfer between domains.

1.3 Objectives

The objectives of this project are:

- Design a SmartNIC packet classifier on FPGA
- Implement both throughput-optimized and area-optimized architectures
- Achieve real-time packet processing with minimal latency
- Ensure timing closure using Static Timing Analysis (STA) and proper constraints
- Evaluate trade-offs between performance, area, resource utilization, and power consumption
- Demonstrate functionality through simulation and FPGA deployment.
- Compare architectural optimizations using quantitative performance metrics.

1.4 Applications

The SmartNIC Packet Classification Accelerator has applications in modern high-performance networking systems, including:

- **Network switching & routing** – classify packets to determine forwarding paths and routing decisions
- **Load balancing** – distribute traffic across multiple servers or cores based on packet attributes
- **Intrusion detection / security** – detect malicious traffic patterns and enforce filtering rules

- **Quality of Service (QoS)** – prioritize traffic types (e.g., video, voice, data)
- **Data center acceleration** – offload packet processing from CPUs to improve system performance
- **Edge/IoT networking** – enable low-latency, real-time packet inspection in constrained environments
- **Content Addressable Memory (CAM)-Style Packet Classification** – support high-speed lookup operations similar to those used in routers and firewalls
- **SmartNIC-Based CPU Offloading** – reduce processor workload by executing networking tasks directly on programmable hardware

2. Background

This section presents the theoretical foundations and architectural techniques that underpin the design of the SmartNIC Packet Classification Accelerator. It introduces key concepts in high-performance digital system design, including pipelining, parallelism, resource sharing, and clock domain crossing. Timing analysis principles such as setup and hold constraints, slack, and critical path evaluation are discussed to provide the necessary context for the optimization and implementation of the proposed architectures.

2.1 Related Architectural Techniques

- **Parallelism (Low Latency Design)**
  Execute multiple comparisons simultaneously to minimize processing time, at the cost of higher resource usage.
- **Pipelining (High Throughput Design)**
  Divide processing into stages so multiple packets are processed concurrently, increasing throughput with added registers.
- **Resource Sharing (Area Optimization)**
  Reuse hardware across multiple cycles to reduce LUT and register usage, increasing latency.
- **Loop Unrolling vs Rolling**
  Unrolling increases parallelism and speed; rolling reuses logic to reduce area.
- **Efficient Data Path Design**
  Minimize logic depth and routing complexity to improve timing performance.
- **Clock Domain Management (CDC)**
  Use synchronization (e.g., FIFOs) when crossing clock domains to ensure data integrity

- **Constraint-Driven Design**
  Apply timing constraints (e.g., create_clock, I/O delays) to guide synthesis and ensure timing closure.

2.1 Relevant Theory

- **Setup and Hold Time**
  Data must be stable before (setup) and after (hold) the clock edge for correct register operation. Violations cause incorrect behavior.
- **Static Timing Analysis (STA)**
  Verifies timing correctness by analyzing all paths without simulation. Ensures data arrives within required timing constraints.
- **Slack (Timing Margin)**
  Slack = Required Time − Arrival Time.
  Positive slack → valid design, negative slack → timing violation.
- **Critical Path**
  The longest delay path in the design determines the maximum operating frequency.
- **Clock Skew and Uncertainty**
  Variations in clock arrival times affect timing and must be accounted for in design constraints.
- **Throughput vs Latency**
  Throughput = rate of processing (improved with pipelining).
  Latency = time per packet (minimized with parallelism).
- **FPGA Resource Model**
  Logic elements include LUTs and flip-flops; efficient designs leverage both to balance performance and area.

2.2 Packet Trace and Dataset Preparation

To evaluate the SmartNIC Packet Classification Accelerator using realistic traffic, a packet trace was captured from a local network environment and converted into a compact memory format suitable for simulation and FPGA testing. The final dataset contained 124 packet entries, and the same packet trace was used for both the high-throughput and area-optimized implementations to ensure a fair comparison.

2.2.1 Packet Capture Methodology

Network traffic was first captured in Wireshark and saved as a packet capture (.pcap) file. The captured traffic was then processed using TShark, together with sed and awk, to extract and reformat selected IPv4 header fields.

The extraction script processed the packet capture by reading the following fields from each IPv4 packet:

- IP version
- IP protocol
- Source IP Address
- Destination IP Address

Although additional IPv4 fields were initially extracted during preprocessing, the final SmartNIC classifier used only the fields required by the hardware design: the **IP version, protocol, source address**, and **destination address.**

2.2.2 Data Preprocessing

After the .pcap file was captured, the packet trace was converted into a hexadecimal format suitable for hardware-oriented testing. This preprocessing step was performed using TShark for field extraction, sed for cleanup, and awk for formatting and IP address conversion.

The preprocessing flow consisted of the following steps:

1. **Packet extraction:** TShark was used to read packet header fields from the input .pcap file.
2. **Field cleanup:** sed was used to remove the 0x prefix from hexadecimal fields.
3. **Address conversion:** awk was used to convert dotted-decimal IP addresses into 32-bit hexadecimal-compatible values.
4. **Word formatting:** The selected fields were packed into a 76-bit hexadecimal word.
5. **Memory initialization:** The formatted packet words were saved into pck_stream.mem for use in simulation and FPGA testing.

The exact preprocessing command used is shown below:

```
tshark -r input.pcap -Y "ip" -T fields \
-e ip.version \
-e ip.proto \
-e ip.src \
-e ip.dst \
-E separator=, \
| awk -F',' '{
    # Convert source IP address from dotted decimal to 32-bit integer
    split($3, src, ".");
    src_ip = (src[1] * 16777216) + (src[2] * 65536) + (src[3] * 256) + src[4];

    # Convert destination IP address from dotted decimal to 32-bit integer
    split($4, dst, ".");
    dst_ip = (dst[1] * 16777216) + (dst[2] * 65536) + (dst[3] * 256) + dst[4];

    # Format packet fields into hexadecimal representation
    # Format: [Version][Protocol][Source IP][Destination IP]
    printf "%01X%02X%08X%08X\n",
        $1,        # IP version (4 bits)
        $2,        # Protocol (8 bits)
        src_ip,    # Source IP (32 bits)
        dst_ip;    # Destination IP (32 bits)
}' > pck_stream.mem
```

Figure 1. Packet Trace Preprocessing Script for Generating pck_stream.mem

Note: This script in Figure 1. extracts IPv4 header fields from the captured .pcap file using TShark and converts them into a 76-bit hexadecimal format using AWK. The generated pck_stream.mem file is used consistently across both simulation and FPGA implementations to ensure reproducibility and fair architectural comparison.

A sample of the processed packet entries is shown below.

```
4110A000024E00000FB
4110A000024E00000FB
4110A000024E00000FB
4060A0000DE0A0000F0
4110A000024E00000FB
4060A0000F00A0000DE
4110A000024E00000FB
4060A0000DED8EF2215
406D8EF22150A0000DE
406D8EF22150A0000DE
```

Figure 2. First 10 entries from pck_stream.mem

Note: The complete memory file was included with the project submission and used consistently across both architectural versions.

2.2.3 Packet Format

Each packet in pck_stream.mem is stored as a 76-bit word, corresponding to 19 hexadecimal digits. The SmartNIC design uses the following packet format:

- Bits [75:72] → IP version (4bits)
- Bits [71:64] → Protocol (8 bits)
- Bits [63:32] → Source IP address (32 bits)
- Bits [31:0] → Destination IP address (32 bits)

In hexadecimal form, the packet is represented as:

[1 hex digit version][2 hex digits protocol][8 hex digits source IP][8 hex digits destination IP]

For example, consider the first packet in Figure 1 above:

4110A000024E00000FB

This value is decoded as:

- Version = 4 → IPv4
- Protocol = 11 → UDP (decimal 17)
- Source Address = 0A000021 → 10.0.0.36
- Destination Address = E00000FB → 224.0.0.251

This encoding matches the packet parser implementation used in the SmartNIC design, where the hardware extracts the version, protocol, source address, and destination address directly from fixed bit positions.

Using this compact representation made it possible to stream packet data efficiently into the hardware, reuse the same packet file across simulation and FPGA validation, and verify classifier behavior using realistic traffic patterns.

2.3 Rule Table Design

2.3.1 Purpose of the Rule Table

The rule table serves as the decision-making component of the SmartNIC packet classifier. It defines whether incoming packets should be allowed or dropped based on their destination IP address and protocol. The rule table served three primary purposes in this project:

1. **Verification of Functional Correctness:** It provided a deterministic mechanism to validate classifier behavior during simulation and hardware testing.

2. **Performance Evaluation:** It enabled meaningful comparison between the high-throughput and area-optimized architectures by ensuring both designs operated

under identical classification policies.

3. **Realistic Network Emulation:** It modeled packet-filtering behavior similar to firewall and access-control-list (ACL) systems used in modern networking environments.

Because both architectures used the same rule set, differences in timing, area, and power consumption can be attributed to architectural optimizations rather than variations in classification policy.

2.3.2 Rule Table Structure

The SmartNIC packet classifier operates using a predefined rule table that determines whether packets should be allowed or dropped based on their destination IP address and protocol type.

Each rule consists of a 32-bit destination IP address, an 8-bit protocol field, and a 1-bit action flag where 1 indicates that the packet is allowed, and 0 that the packet should be dropped.

To ensure a fair and consistent comparison, the same rule table was used for both the high-throughput and area-optimized architectures.

The rule table was derived from patterns observed in the packet trace captured from a local network. By analyzing common multicast, private, broadcast, and public traffic, representative rules were constructed to evaluate the SmartNIC under realistic operating conditions.

Table 1. SmartNIC Packet Classification Rule Table

Rule ID	Destination IP Address	Hexadecimal Value	Protocol	Action
0	224.0.0.251	E00000FB	UDP (17)	Allow
1	224.0.0.22	E0000016	IGMP (2)	Allow
2	10.0.0.222	0A0000DE	UDP (17)	Allow
3	10.0.0.222	0A0000DE	TCP (6)	Allow

4	239.255.255.250	EFFFFFFA	UDP (17)	Drop
5	255.255.255.255	FFFFFFFF	UDP (17)	Drop
6	10.0.0.255	0A0000FF	UDP (17)	Drop
7	162.159.134.234	A29F86EA	TCP (6)	Allow
8	142.126.244.119	8E7EF477	TCP (6)	Allow
9	216.239.34.21	D8EF2215	TCP (6)	Allow
10	34.107.243.93	226BF35D	TCP (6)	Allow
11	64.71.255.204	4047FFCC	UDP (17)	Drop
12	10.0.0.240	0A0000F0	TCP (6)	Drop
13	224.0.0.251	E00000FB	TCP (6)	Drop
14	224.0.0.22	E0000016	UDP (17)	Drop
15	255.255.255.255	FFFFFFFF	TCP (6)	Drop

2.3.3 Design Rationale

The rule table was designed to include a mix of traffic patterns that are representative of realistic network behavior. These rules include:

- Multicast traffic, such as 224.0.0.251 and 224.0.0.22
- Private-network traffic, such as 10.0.0.222, 10.0.0.240, and 10.0.0.255
- Broadcast traffic, such as 255.255.255.255
- Selected public Internet addresses

- Both allowed and dropped cases for the same or similar destination classes

This mixture ensured that the classifier was tested under both positive and negative matching cases, rather than only on packets that should pass.

3. Architecture Design

This chapter presents the architectural design of the SmartNIC Packet Classification Accelerator. It introduces the high-level structure, datapath organization, and control mechanisms of both the high-throughput and area-optimized implementations.

3.1 Block Diagrams

3.1.1 High-Throughput Design

The high-throughput SmartNIC architecture is designed to classify packets at line rate using pipelining, parallelism, and safe clock domain crossing. This design achieves high performance by evaluating classification rules concurrently and processing packets in a continuous streaming pipeline. The objective is to maximize throughput while ensuring reliable operation across multiple clock domains.

Top-Level Architecture

Figure 3. below illustrates the elaborated top-level block diagram of the high-throughput SmartNIC design, generated using Xilinx Vivado. Although derived from the RTL elaborated view, the figure represents the architectural hierarchy and data flow of the system. It highlights the integration of an asynchronous FIFO for clock domain crossing and a pipelined packet classification engine.

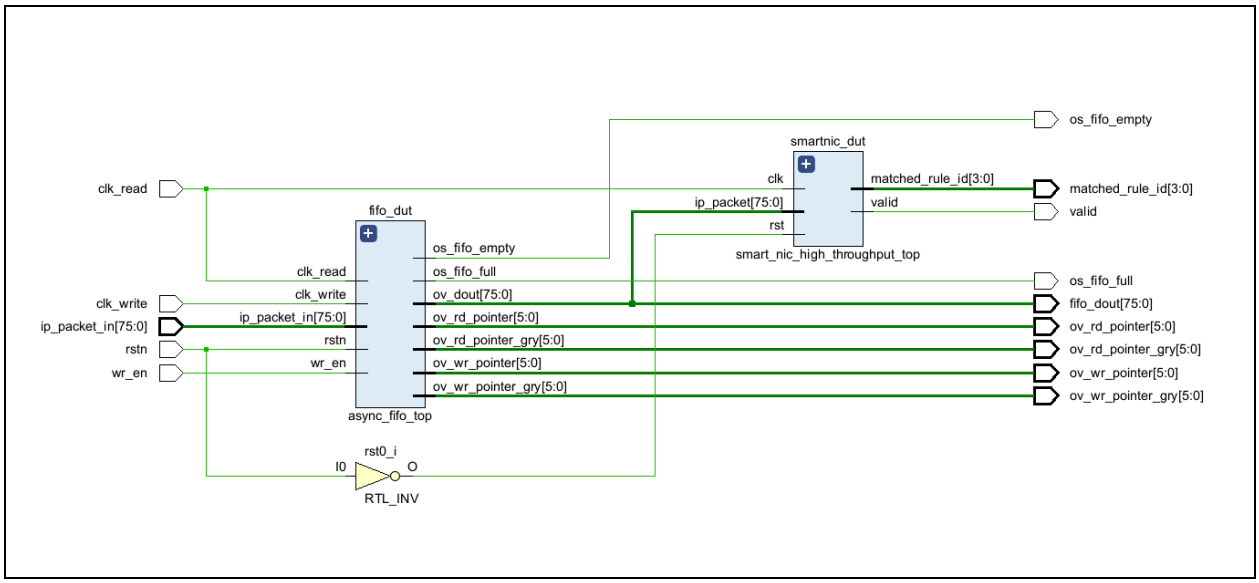


Figure 3. High-Throughput SmartNIC Top-Level Block Diagram (Elaborated RTL View)

This design is implemented in the synthesis wrapper **smartnic_cdc_high_throughput_top**, defined in the file:**smartnic_cdc_high_throughput_top_for_synthesis.v.** The module integrates the clock domain crossing (CDC) FIFO with the SmartNIC packet parsing and classification pipeline.

Top-Level Signal Description

Table 2 summarizes the primary input and output signals of the high-throughput SmartNIC architecture.

Table 2. Top-Level Description

Signal	Direction	Width (bits)	Description
clk_write	Input	1	Write-domain clock for packet injection
clk_read	Input	1	Read-domain clock for packet classification
rstn	Input	1	Active-low system reset
wr_en	Input	1	Write enable signal for the FIFO
ip_packet_in	Input	76	Incoming packet data
valid	Output	1	Indicates a valid classification result

match_rule_id	Output	4	Identifier of the matched rule

Architecture Data Flow

The operational flow of the high-throughput SmartNIC architecture is summarized as follows:

1. **Packet Injection:** Incoming packets are supplied to the system via the ip_packet_in signal in the write clock domain (clk_write). The wr_en signal controls data entry into the FIFO.

2. **Clock Domain Crossing (CDC):** An asynchronous FIFO (async_fifo_top) transfers packet data safely from the write clock domain to the read clock domain (clk_read). This ensures reliable data synchronization between subsystems operating at different frequencies.

3. **Packet Parsing:** In the read clock domain, the packet_parser module extracts key header fields—including the IP version, protocol, source address, and destination address—from each 76-bit packet.

4. **High-Throughput Classification:** The parsed packet fields are forwarded to the rule_classifier_high_throughput module. This classifier employs a pipelined parallel comparator architecture to evaluate all predefined rules simultaneously. By performing concurrent comparisons, the design enables single-cycle packet classification after pipeline fill, thereby achieving high throughput. This approach is conceptually similar to Content Addressable Memory (CAM) techniques used in high-performance networking hardware, although it is implemented here using standard FPGA logic.
5. **Output Generation:** Once a rule match is detected, the system asserts the valid signal and outputs the corresponding rule identifier through matched_rule_id.

Design Rationale

The high-throughput architecture was developed to achieve maximum packet processing performance. Key design decisions include:

- **Pipelining:** Enables concurrent processing of multiple packets, increasing throughput.
- **Parallel Rule Comparison:** Implements parallel rule matching to evaluate all classification rules simultaneously.
- **Clock Domain Crossing:** Utilizes an asynchronous FIFO to ensure reliable communication between independent clock domains.

- **Modular Design:** Separates parsing, classification, and synchronization for scalability and maintainability.
- **Constraint-Driven Implementation:** Ensures timing closure and stable operation at high clock frequencies.

These choices enable deterministic, high-speed packet classification suitable for SmartNIC and data center acceleration applications.

3.1.2 Area Optimized Design Block Diagram

The area-optimized SmartNIC architecture was designed to reduce hardware utilization while still performing correct packet classification on realistic network traffic. Unlike the high-throughput design, which evaluates all rules in parallel, the area-optimized architecture uses resource sharing and sequential rule evaluation to minimize logic cost. This design choice reduces LUT and register usage at the expense of increased classification latency.

Top-Level Architecture

Figure 4. below shows the elaborated top-level block diagram of the area-optimized SmartNIC design generated in Vivado. Although the figure is taken from the RTL elaborated view, it accurately reflects the major architectural modules and the overall flow of data through the system.

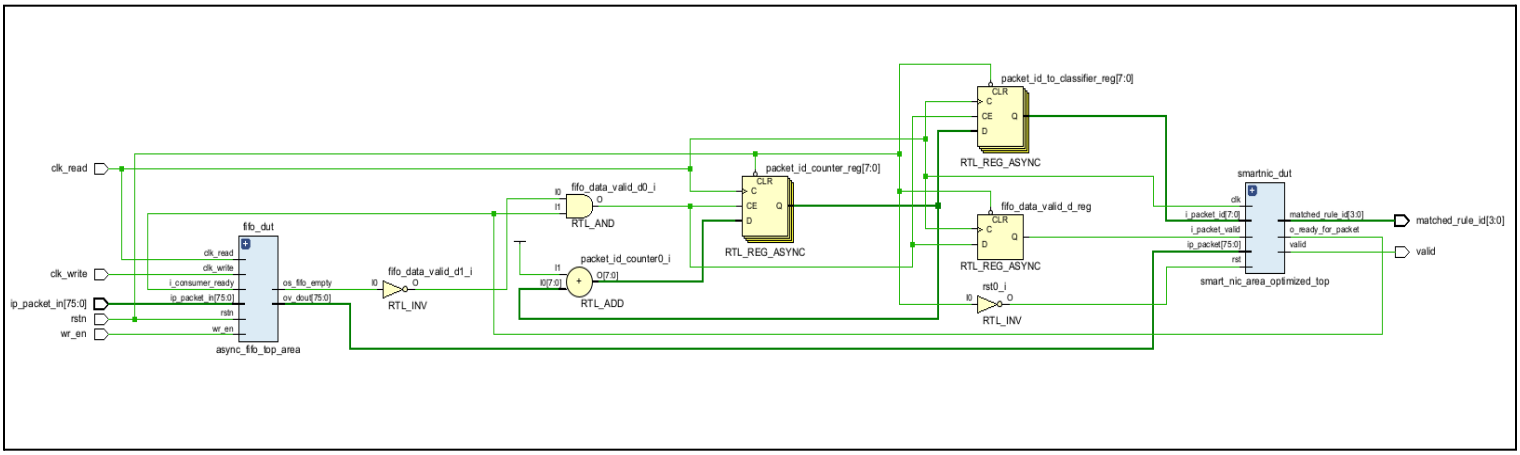


Figure 4. Area Optimized SmartNIC Top-Level Block Diagram (Elaborated RTL View)

This design is implemented in the synthesis wrapper **smartnic_cdc_area_optimized_top**, defined in the file: **smartnic_cdc_area_optimized_top_for_synthesis.v**

The module integrates three major functions:

1. Asynchronous FIFO-based clock domain crossing
2. Packet-valid and packet-ID generation in the read domain
3. Area-optimized packet parsing and classification

Top-Level Signal Description

Table 3 summarizes the main input and output signals of the area-optimized SmartNIC architecture.

Table 3. Top-Level Signal Description

Signal	Direction	Width (bits)	Description
clk_write	Input	1	Write-domain clock for packet injection
clk_read	Input	1	Read-domain clock for packet classification
rstn	Input	1	Active-low system reset
wr_en	Input	1	Write enable signal for the FIFO
ip_packet_in	Input	76	Incoming packet data
valid	Output	1	Indicates a valid classification result
match_rule_id	Output	4	Identifier of the matched rule

Although the top-level interface is similar to that of the high-throughput design, the internal control behavior differs significantly due to the use of sequential rule evaluation and handshake-driven packet acceptance.

Architecture Data Flow

The operational flow of the area-optimized SmartNIC architecture is as follows:

- **Packet Injection:** Incoming packets are supplied through ip_packet_in and written into the FIFO in the write clock domain when wr_en is asserted.
- **Clock Domain Crossing**: The module async_fifo_top_area transfers packets from the write domain to the read domain. Unlike the throughput FIFO, this version includes consumer-ready gating so that packets are released only when the classifier is ready to

accept a new one.

- **Packet-Valid and Packet-ID Generation:** In the read domain, additional control logic generates: a delayed packet-valid signal (fifo_data_valid_d) and a packet ID (packet_id_to_classifier) for tracking classification results

  This logic is necessary because the area-optimized classifier processes packets sequentially and therefore requires handshake coordination.

- **Packet Parsing:** The packet_parser module extracts the protocol and destination address, along with the remaining IPv4 header fields, from the 76-bit packet.

- **Sequential Rule Classification:** The parsed packet fields are passed to the smart_nic_area_optimized_top module, which instantiates rule_classifier_area_optimized. Instead of checking all rules simultaneously, this classifier scans one rule at a time using shared comparison logic.

- **Output Generation:** After a matching rule is found, or after all rules have been checked, the classifier asserts valid, produces matched_rule_id, and raises a completion signal internally.

Design Rationale

The area-optimized architecture was developed to minimize hardware cost while preserving functional correctness. Several key design choices support this objective:

- **Resource Sharing:** A single comparator is reused across multiple cycles instead of implementing 16 parallel comparators.

- **Sequential Rule Evaluation:** Rules are scanned one at a time, which reduces combinational logic and register duplication.

- **Handshake-Based Control:** The classifier exposes a ready signal so that new packets are accepted only when the previous packet has been fully processed.

- **Consumer-Gated FIFO Read:** The FIFO in this design only releases data when the classifier is ready, preventing packet loss or misalignment between data and control.

- **Packet ID Tracking:** Packet IDs are generated in the read domain to support result correlation during simulation and verification.

These choices make the design significantly smaller than the throughput-optimized version, while clearly demonstrating the trade-off between **area efficiency** and **classification latency**.

3.2 Datapath Explanation

3.2.1 High-Throughput Datapath

Figure Reference: Refer to the high-throughput top-level block diagram **Figure 3** in Section 3.1.1.

The datapath of the high-throughput SmartNIC architecture is designed to classify packets at line rate through pipelining and parallelism. As illustrated in Figure 3, packet data flows sequentially through the asynchronous FIFO, packet parser, and parallel rule classifier.

1. **Packet Input:** Incoming packets are provided as a 76-bit word (ip_packet_in) and written into the system using the write clock (clk_write).

2. **Clock Domain Crossing:** The module async_fifo_top buffers packets and transfers them safely from the write clock domain to the read clock domain (clk_read) using Gray-coded pointers and synchronizers.

3. **Packet Parsing:** The packet_parser extracts key IPv4 header fields from the incoming packet: Version, Protocol, Source IP Address, Destination IP Address

4. **Parallel Rule Classification:** The extracted fields are forwarded to the rule_classifier_high_throughput module, which compares them against all predefined rules simultaneously using parallel comparators to enable high-speed packet classification.

5. **Output Generation:** Once a match is found, the classifier produces:

    - valid: indicates whether the packet is allowed.

    - matched_rule_id: identifies the matching rule.

This pipelined architecture enables continuous packet processing, achieving high throughput with minimal latency after pipeline fill.

3.2.2 Area-Optimized Datapath

Figure Reference: Refer to the area-optimized block diagram, Figure 4 in Section 3.1.2

The area-optimized datapath minimizes hardware utilization by employing resource sharing and sequential rule evaluation.

1. **Packet Input and FIFO Buffering:** Incoming packets are written into the asynchronous FIFO (async_fifo_top_area) in the write clock domain.

2. **Clock Domain Crossing:**The FIFO safely transfers data to the read clock domain while preventing overflow and underflow.

3. **Packet Parsing:**The packet_parser extracts the protocol and destination IP address required for classification.

4. **Sequential Rule Classification:** The rule_classifier_area_optimized evaluates one rule per clock cycle using a shared comparator. This significantly reduces logic utilization compared to the parallel architecture.

5. **Handshake and Packet Tracking:** Additional signals ensure controlled data flow:
    - i_packet_valid indicates the availability of a new packet.
    - o_ready_for_packet signals readiness to accept new data.
    - o_result_done indicates completion of classification.

6. **Output Generation:** The classifier produces the final decision via: valid and matched_rule_id

This approach trades throughput for reduced area and power consumption.

3.3 Control logic Explanation

3.3.1 High-Throughput Control Logic

The high-throughput SmartNIC architecture is designed to support continuous packet classification with minimal latency. Its control logic enables pipelined execution and uninterrupted data flow between modules.

Packet transfer between clock domains is managed by the asynchronous FIFO (async_fifo_top), which ensures safe communication between the write clock (clk_write) and read clock (clk_read). Gray-coded pointers and multi-stage synchronizers prevent metastability and guarantee reliable clock domain crossing. The FIFO status signals os_fifo_full and os_fifo_empty regulate write and read operations, preventing overflow and underflow.

Within the classification stage, control is governed by a pipelined structure. Input data is latched on each clock cycle, and parallel comparisons are performed simultaneously.

Pipeline registers synchronize operations across stages, allowing new packets to be processed every cycle once the pipeline is filled. This mechanism eliminates stalls and maximizes throughput.

System initialization is handled through a synchronous reset signal (rst), which loads the rule table and clears internal registers to establish a known starting state. Upon completion of classification, the module asserts the valid signal along with the corresponding matched_rule_id, indicating the classification outcome.

The control logic in the high-throughput architecture prioritizes performance by enabling continuous streaming, deterministic timing, and efficient synchronization across clock domains.

3.3.2 Area-Optimized Datapath Control Logic

The area-optimized SmartNIC architecture minimizes hardware utilization by employing sequential rule evaluation and handshake-driven control. Unlike the high-throughput design, this architecture processes one rule per clock cycle using shared resources.

The asynchronous FIFO (async_fifo_top_area) manages clock domain crossing while incorporating consumer-ready gating. Packets are released only when the classifier asserts readiness, ensuring controlled and synchronized data transfer.

Classification is coordinated using a ready–valid handshake protocol. The signal **i_packet_valid** indicates the arrival of a new packet, while **o_ready_for_packet** signals when the classifier is prepared to accept it. This prevents data loss and ensures correct sequencing. A busy state within the classifier prevents additional packets from being processed until the current classification is complete.

A rule index counter **(rule_idx)** sequentially scans the rule table, evaluating one rule per clock cycle. Once a match is found, or all rules have been examined, the classifier asserts o_result_done, generates the output signals valid and matched_rule_id, and releases the packet identifier **(o_packet_id)**. These signals enable precise tracking and verification during simulation.

As with the throughput-optimized design, a reset signal initializes all internal registers, counters, and flags to ensure deterministic system startup.

This control strategy reduces logic utilization through **resource sharing**. The trade-off is increased latency, which highlights the balance between area efficiency and performance.

4. Version 1 – High-Throughput Design

This section presents the implementation and evaluation of the high-throughput SmartNIC Packet Classification Accelerator. The design emphasizes pipelining and parallelism to achieve single-cycle packet classification. All modules were developed in Verilog HDL and synthesized using Xilinx Vivado for architectural evaluation.

4.1 Architecture

The high-throughput SmartNIC architecture implements a pipelined packet classification engine capable of evaluating multiple rules concurrently. Clock domain crossing between packet injection and classification stages is handled by an asynchronous FIFO.

File Structure

Table 4. High-Throughput Design Files

File Name	Description
smartnic_cdc_high_throughput_top_for_synthesis.v	Top-level synthesis wrapper integrating CDC FIFO and classification pipeline
cdc_fifo_throughput.v	Asynchronous FIFO for clock domain crossing
packet_parser.v	Extracts protocol and IP fields from incoming packets
rule_classifier_high_throughput.v	Parallel, pipelined rule classifier
packet_parser_rule_classifier_top_sim.v	Integrates parser and classifier
snic_throughput_cdc_testbench.v	Functional simulation testbench
smartnic_cdc_constraints.xdc	Timing constraints for synthesis and implementation
pck_stream.mem	Packet dataset used for simulation and validation\

4.2 Implementation Details

The RTL implementation of the high-throughput SmartNIC design was organized into modular Verilog files to separate packet parsing, rule classification, and clock domain crossing functionality. This modular structure improved readability, simplified debugging, and ensured that each architectural function could be verified independently before full-system synthesis. The implementation was also divided into simulation-oriented and synthesis-oriented files. In particular, a dedicated top-level synthesis wrapper was created to integrate the asynchronous FIFO and classification pipeline into a single hierarchy suitable for RTL elaboration, synthesis, and implementation analysis in Vivado.

4.2.1 Packet Parser

File: **packet_parser.v**

The file packet_parser.v implements the packet parsing stage of the SmartNIC pipeline. The module extracts four fields from the 76-bit packet format used throughout the project: IP version, protocol, source IP address, and destination IP address. The parser is implemented using purely combinational logic, allowing field extraction to occur without additional sequential latency.

The field positions were fixed in the packet format so that extraction could be performed using direct bit slicing. This simplified the design and reduced logic complexity. Although the parser extracts all four fields, the classification logic primarily relies on the protocol and destination address. Including source address and version in the parser maintained consistency with the packet dataset format and preserved flexibility for future extensions of the classifier.

The parser forms the first stage of the SmartNIC classification pipeline and ensures that relevant packet fields are made available in a structured form for the downstream classifier.

4.2.2 High-Throughput Rule Classifier

File: **rule_classifier_high_throughput.v**

The file rule_classifier_high_throughput.v implements the core high-throughput classification engine. This module was designed to maximize classification speed by combining pipelining with parallel rule comparison. The classifier uses a rule table containing 16 entries, where each rule stores a destination IP address, protocol value, and associated action bit.

The implementation is organized into three logical stages. In the first stage, the destination address and protocol are latched into pipeline registers. In the second stage, the packet fields are compared against all rule entries simultaneously using parallel comparator logic. This allows all rules to be evaluated within the same cycle. The intermediate comparison and action results are then stored in a second set of registers. In the third stage, a priority-selection mechanism identifies the first matching rule and produces the output signals valid and matched_rule_id.

This structure was chosen to reduce the effective critical path and support high-frequency operation. Instead of checking rules sequentially, parallel comparison allows the classifier to sustain significantly higher throughput. The trade-off is increased hardware usage, since separate comparison logic is required for all rules.

4.2.3 Asynchronous FIFO

File: **cdc_fifo_throughput.v**

The file cdc_fifo_throughput.v implements the asynchronous FIFO used for clock domain crossing in the high-throughput design. The FIFO separates packet injection from packet classification by allowing writes in one clock domain and reads in another. This was necessary because the design uses distinct write and read clocks during simulation and synthesis analysis.

The FIFO is composed of several submodules: a write controller, a read controller, pointer synchronizers, and a dual-clock memory block. The read and write controllers maintain binary pointers for addressing the FIFO memory and convert those pointers into Gray code before transferring them across clock domains. Three-stage synchronizers are used to safely capture the Gray-coded pointers in the opposite clock domain, thereby reducing the risk of metastability.

Full and empty detection logic prevents invalid writes and reads. The FIFO memory stores incoming packet data until it is safely consumed by the classifier in the read domain. This architecture was selected because asynchronous FIFOs are a standard and reliable technique for transferring data between independently clocked subsystems.

4.2.4 Top-Level Synthesis Wrapper

File: **smartnic_cdc_high_throughput_top_for_synthesis.v**

The file smartnic_cdc_high_throughput_top_for_synthesis.v defines the module smartnic_cdc_high_throughput_top, which serves as the synthesis-level top wrapper for the high-throughput design. This wrapper was created specifically to support RTL elaboration, synthesis, and implementation in Vivado. Its role is to integrate the asynchronous FIFO with the high-throughput SmartNIC classification pipeline in a single top-level module.

The wrapper accepts packet data in the write clock domain through ip_packet_in, together with clk_write, clk_read, wr_en, and rstn. Internally, it instantiates the module async_fifo_top, which performs clock domain crossing, and the module smart_nic_high_throughput_top, which performs packet parsing and classification. The FIFO output fifo_dout is forwarded directly into the classifier pipeline. The final outputs of the wrapper are valid and matched_rule_id, which represent the classification result.

This file was necessary because the simulation hierarchy and the synthesis hierarchy were not identical. A dedicated synthesis wrapper made it possible to elaborate and analyze the complete throughput-oriented architecture in Vivado using the intended module integration.

4.2.5 RTL Elaboration Diagram

The RTL elaborated diagrams illustrate the hierarchical structure of the SmartNIC architecture as interpreted by Vivado prior to synthesis. It confirms the correct integration of the packet parser, rule classifier, and supporting modules, ensuring alignment between the intended architecture and its HDL implementation.

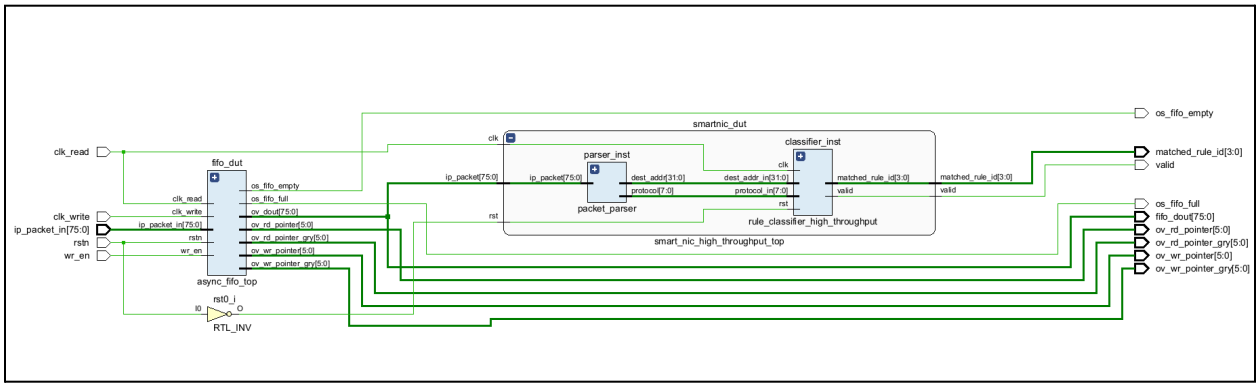


Figure 5. RTL elaborated view of the High-Throughput SmartNIC architecture.

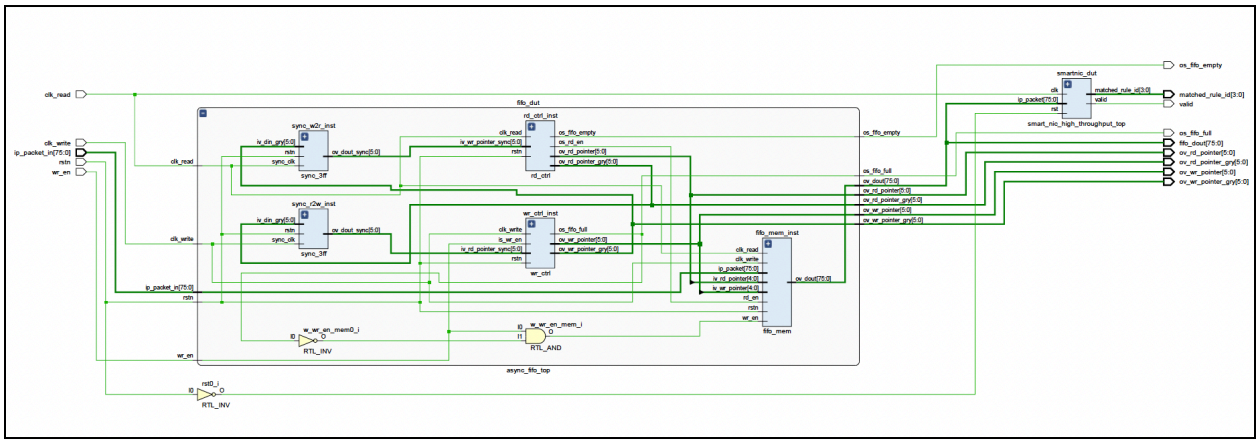


Figure 6. Expanded RTL elaboration showing the internal structure of the asynchronous FIFO.

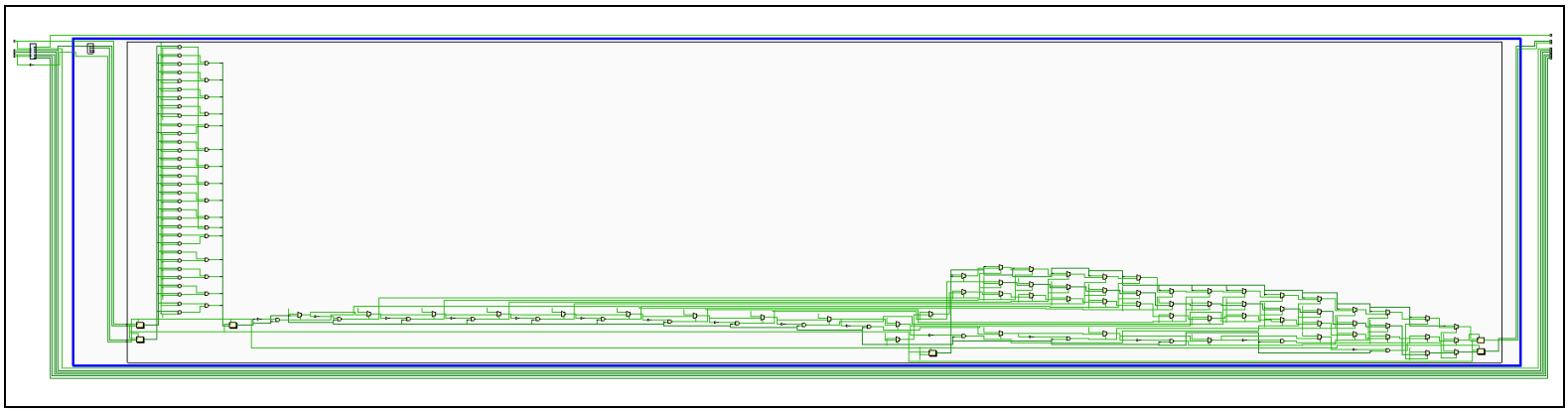


Figure 7. Fully Expanded RTL Schematic of the High-Throughput Classifier

Figure 7. presents the detailed RTL structure of the packet parser and parallel rule classifier. The widespread logic results from multiple comparators operating concurrently to evaluate classification rules in parallel. While this architecture significantly improves throughput, it increases hardware resource utilization, illustrating the trade-off between performance and area.

4.3 Results

4.3.1 Post-Synthesis Schematic

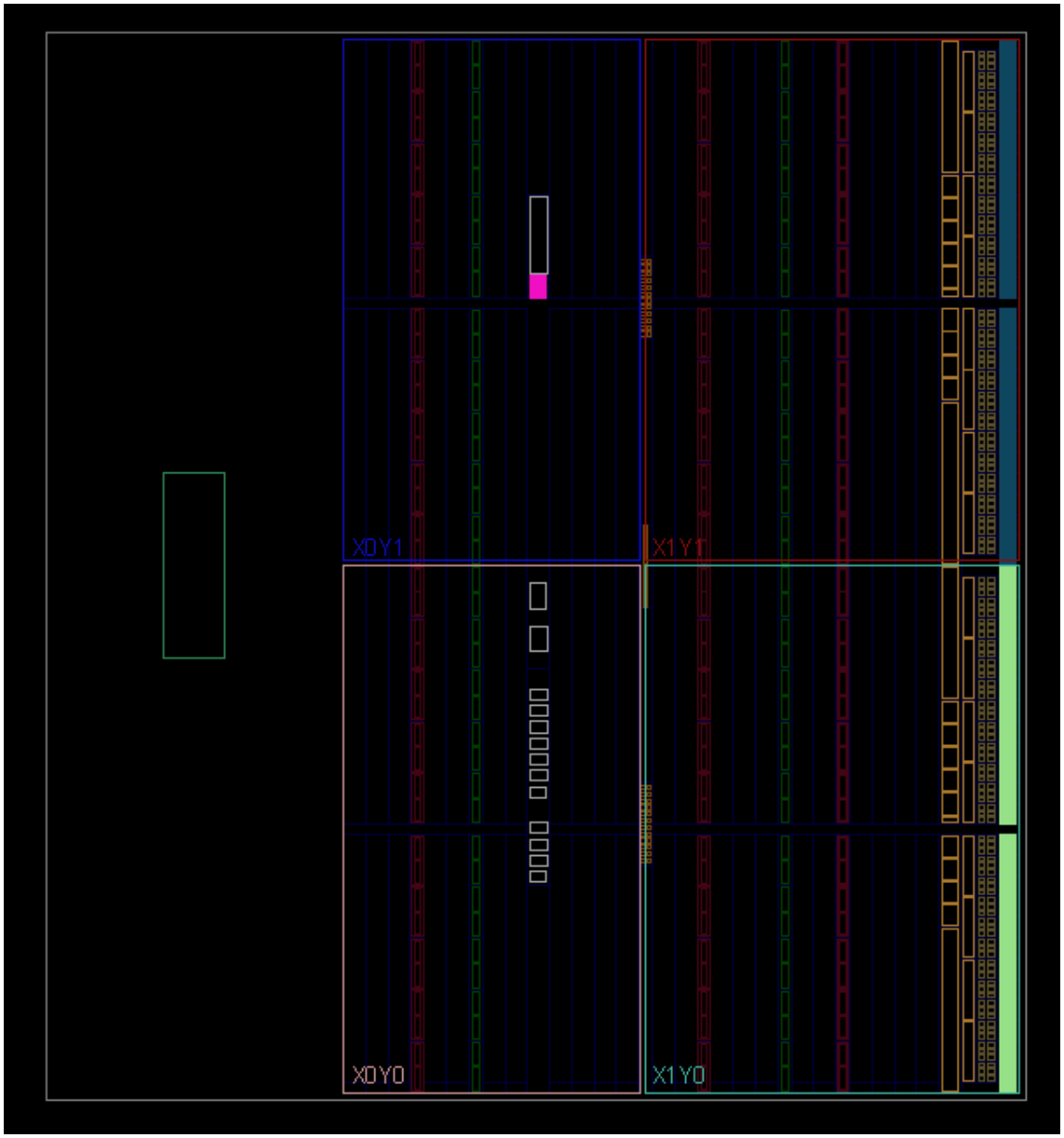


Figure 8. Post-Synthesis Device Utilization for the ZYBO Z7-10 FPGA

Figure 8. presents the post-synthesis device utilization. At this stage, the design has been translated into a gate-level netlist but has not yet undergone placement and routing. Physical placement and routing are performed during the implementation stage and are therefore not reflected in this view.

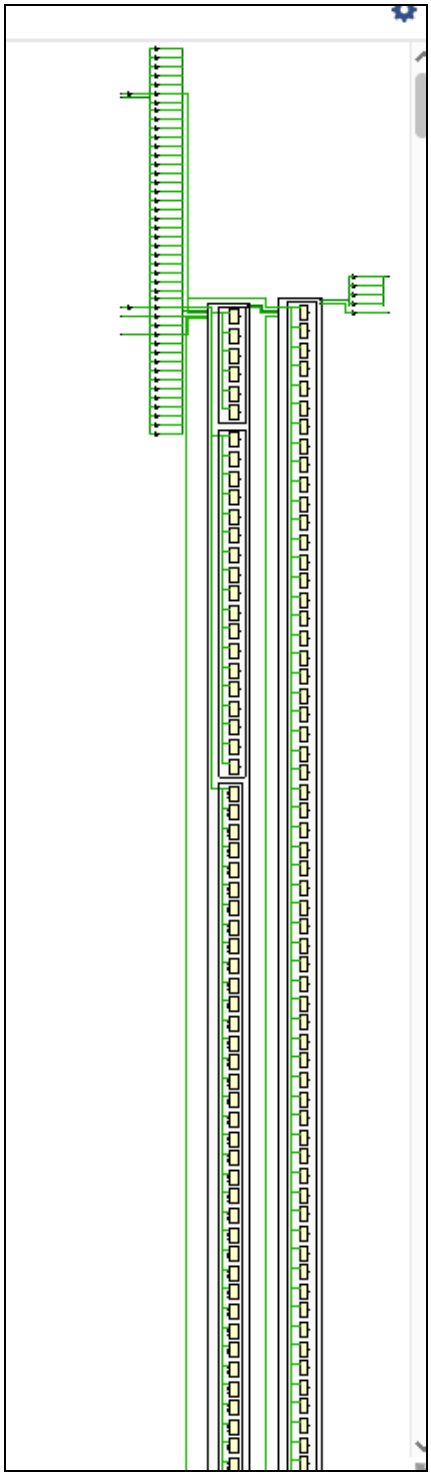


Figure 9.Post-Synthesis Gate-Level Schematic of the High-Throughput SmartNIC Design

Figure 9. presents the gate-level netlist generated after RTL synthesis of the high-throughput SmartNIC. The elongated structures correspond to memory and register arrays associated with the asynchronous FIFO, which buffers packet data and enables safe clock domain

crossing. Their vertical expansion reflects the FIFO's storage depth and data width as synthesized into distributed registers or memory resources.

The combinational logic blocks toward the center and right implement packet parsing and rule classification. These functions are realized using lookup tables (LUTs), multiplexers, and registers. The stretched appearance of the schematic results from Vivado's flattened netlist representation, where high-level modules are decomposed into primitive FPGA components.

4.3.2 Post-Implementation Schematic

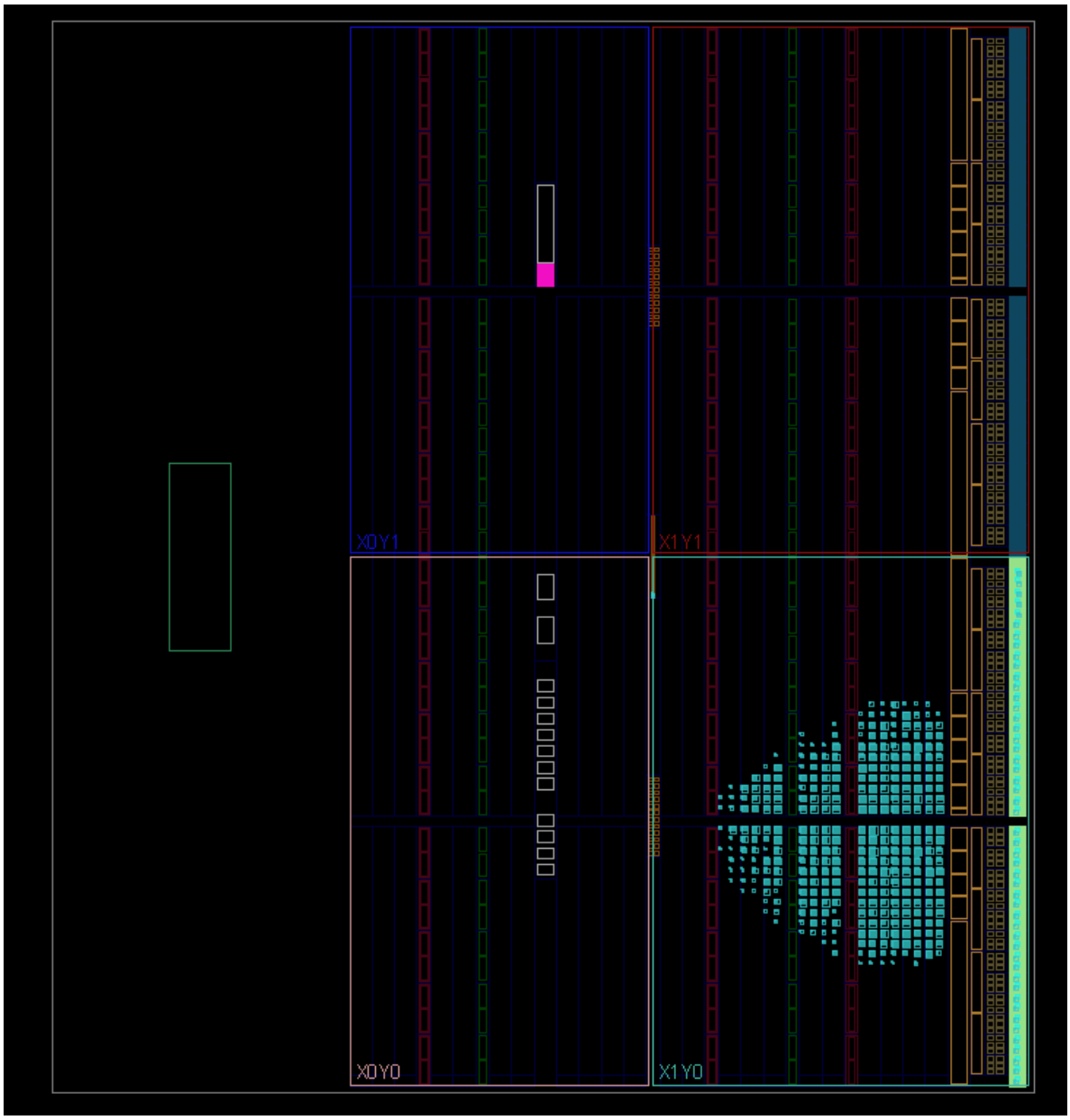


Figure 10. Post-Synthesis Device Utilization for the ZYBO Z7-10 FPGA

Figure 10. shows the post-implementation device utilization of the high-throughput SmartNIC on the ZYBO Z7-10 FPGA. The highlighted cyan region represents the portion of the FPGA fabric occupied by the implemented design, confirming successful placement and routing in Vivado.

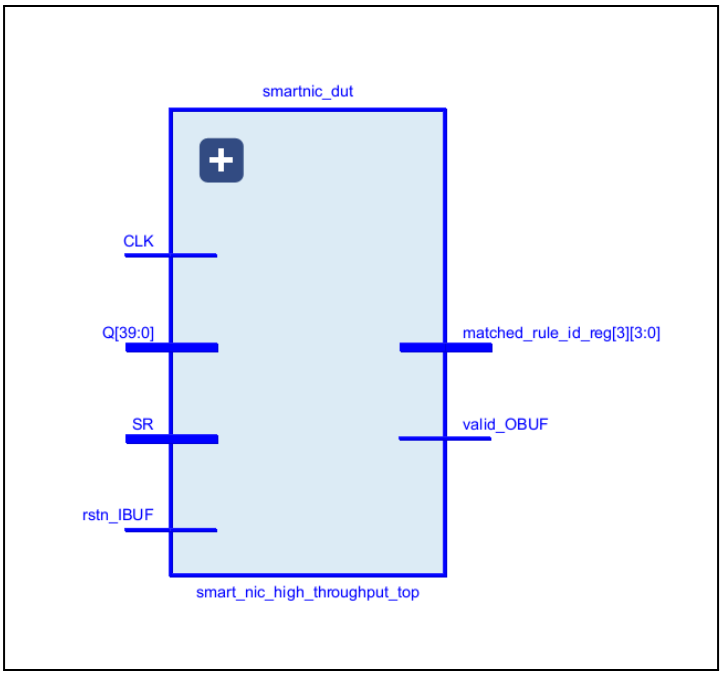


Figure 11 .Implemented Top-Level Module of the High-Throughput SmartNIC Design for the ZYBO Z7-10 FPGA.

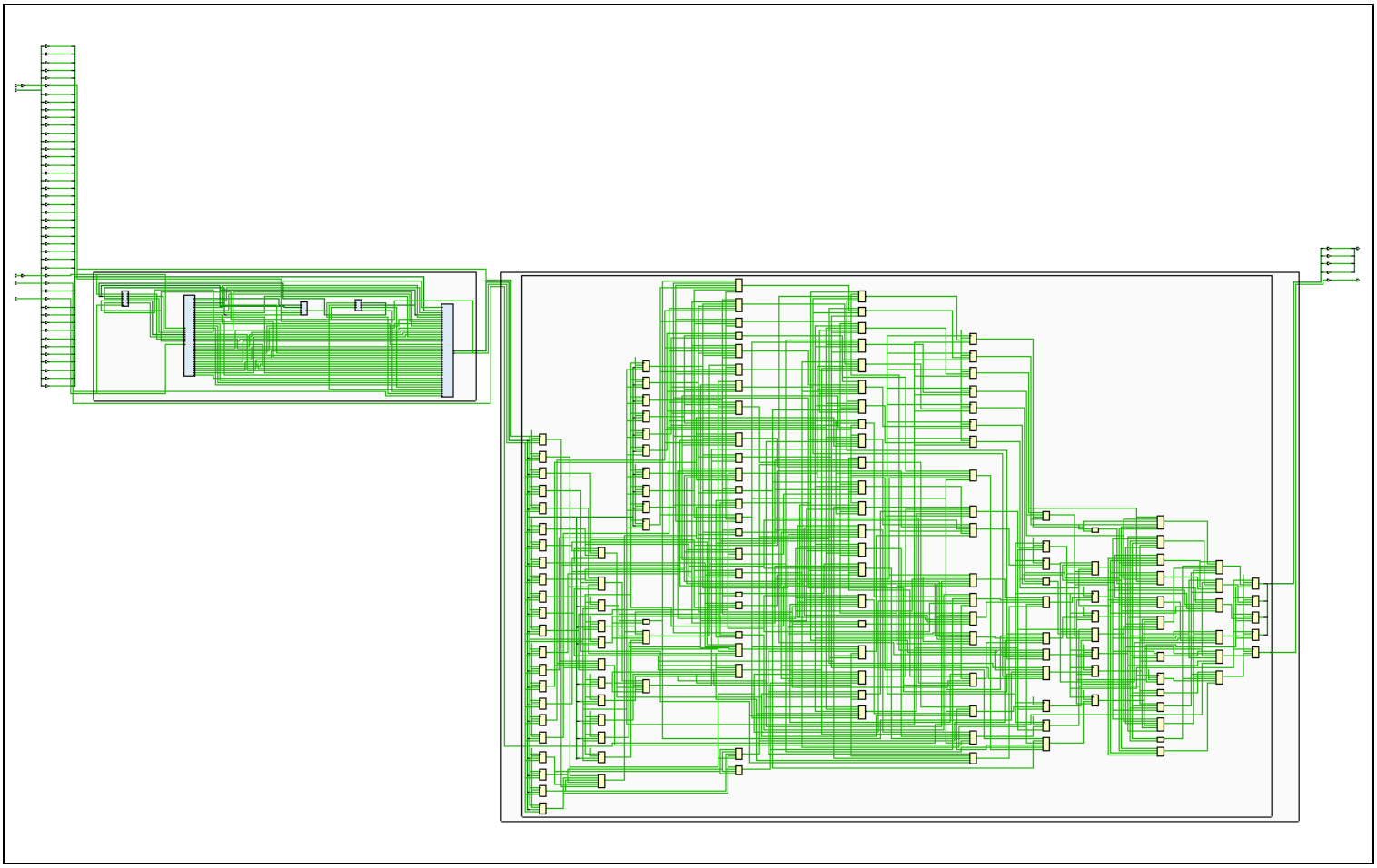


Figure 12.Post-Implementation Full Schematic of the High-Throughput SmartNIC Design

This figure presents the complete post-implementation schematic for the high-throughput SmartNIC packet classification accelerator. It illustrates the fully integrated hardware architecture after placement and routing on the target FPGA.

The structure on the left corresponds to the asynchronous FIFO, which facilitates safe clock domain crossing between the packet injection and classification stages. This module buffers incoming packet data and ensures reliable transfer between the write and read clock domains.

The expanded logic on the right represents the high-throughput classification pipeline. This section integrates the packet parser and the rule_classifier_high_throughput module, which employs a pipelined and parallel comparator architecture. The repeated vertical structures indicate multiple comparators operating concurrently, enabling simultaneous evaluation of classification rules. These parallel stages reduce processing latency and allow high-speed packet classification.

The layered arrangement of logic blocks reflects the pipeline stages of the design, where packet fields are parsed, compared against rule entries, and resolved through priority selection. The widespread distribution of components highlights the hardware cost of achieving high throughput, as increased parallelism requires greater FPGA resource utilization.

This schematic validates the successful implementation of the SmartNIC architecture and demonstrates the trade-off between performance and area. The extensive parallel structures confirm that the design prioritizes throughput by enabling multiple comparisons within each clock cycle.

4.3.3 Timing Analysis

This section presents the timing analysis of the high-throughput SmartNIC design. Timing constraints were iteratively refined to achieve optimal performance while ensuring timing closure on the ZYBO Z7-10 FPGA. Optimization techniques such as constraint tuning, pipelining, and register balancing were applied to improve the clock period (T_clk) and eliminate timing violations. Both synthesis and implementation results are provided to demonstrate design correctness and performance improvement.

The primary timing constraints used were:

- **clk_write:** 5 ns (200 MHz)
- **clk_read:** 6 ns (166.67 MHz)

Asynchronous clock groups were defined to prevent false timing violations between clock domains, and a false path was set for the reset signal to ensure accurate timing analysis.

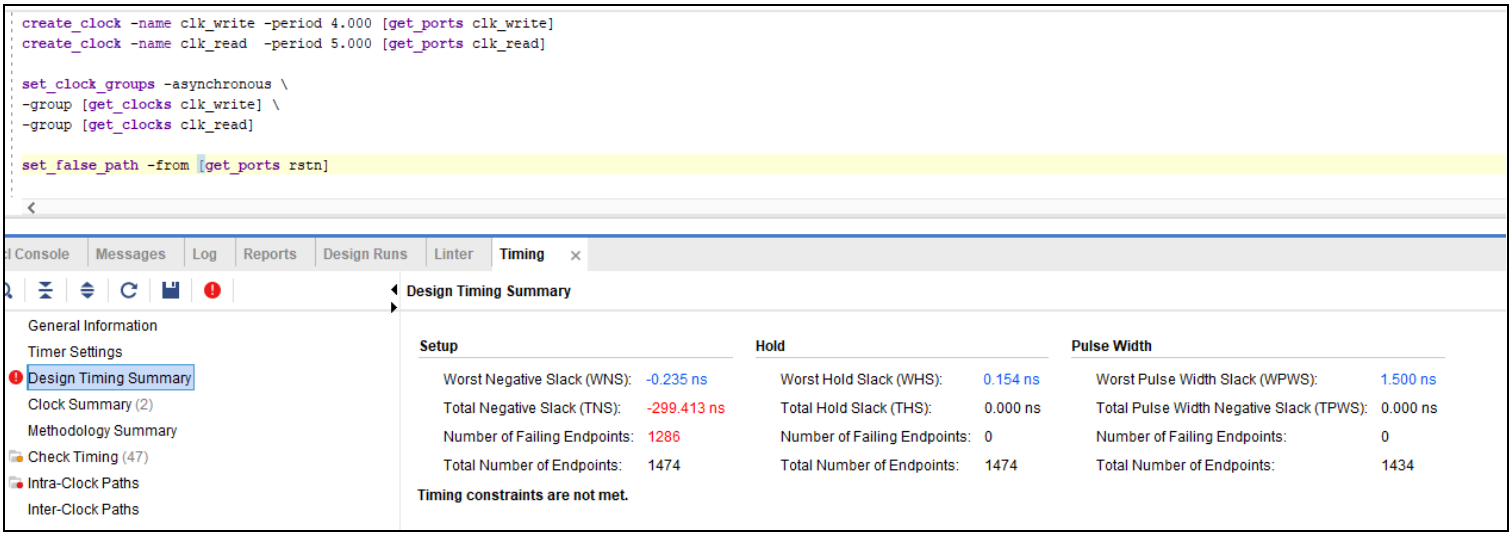


Figure 13. Initial Timing Analysis of the High-Throughput Design at Synthesis

The initial timing analysis was performed using clock constraints of 4 ns for the write clock and 5 ns for the read clock. The synthesis results indicated timing violations:

- Worst Negative Slack (WNS): –0.235 ns
- Total Negative Slack (TNS): –299.413 ns
- Failing Endpoints: 1,286

These violations indicate that the design could not meet the required setup timing at the specified frequencies. The negative slack resulted from long combinational paths within the parallel rule classifier, where multiple comparators operate simultaneously. This configuration increased propagation delay and established the critical path of the design.

However, the hold timing remained satisfied:

- Worst Hold Slack (WHS): 0.154 ns

This confirmed that the violations were limited to setup timing and required constraint refinement rather than architectural modification.

```
create_clock -name clk_write -period 11.000 [get_ports clk_write]
create_clock -name clk_read  -period 11.000 [get_ports clk_read]

set_clock_groups -asynchronous \
-group [get_clocks clk_write] \
-group [get_clocks clk_read]

set_false_path -from [get_ports rstn]
```

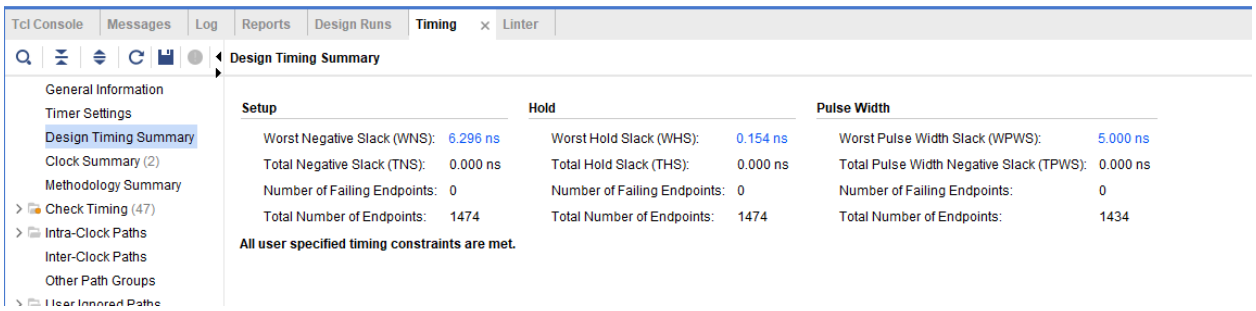


Figure 14. Intermediate Timing Analysis after Constraint Refinement Post Synthesis

To address the setup violations, the clock constraints were relaxed to 11 ns for both clock domains. This adjustment allowed the design to meet timing requirements successfully:

- Worst Negative Slack (WNS): 6.296 ns
- Total Negative Slack (TNS): 0.000 ns
- Failing Endpoints: 0

These results confirmed that the design was functionally correct and capable of meeting timing under relaxed constraints. This intermediate step validated the integrity of the pipelined architecture before further optimization toward higher operating frequencies.

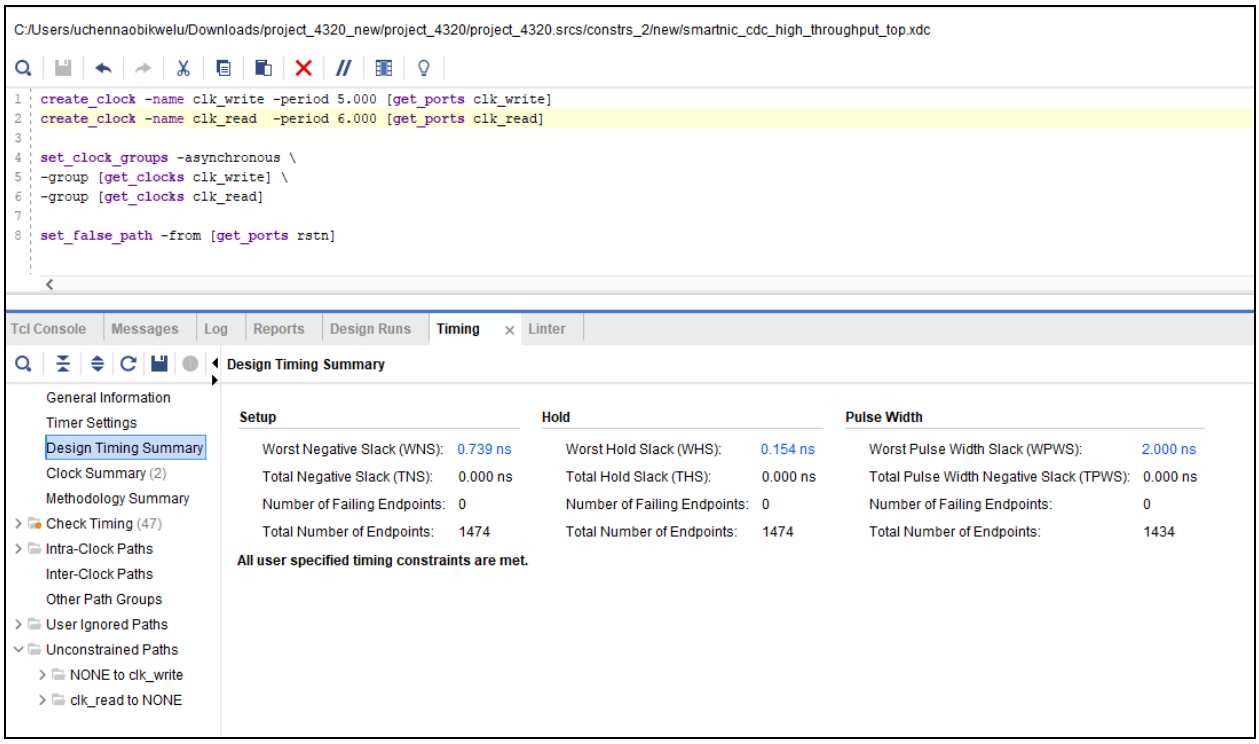


Figure 15. Final Timing Results Achieving Timing Closure at Synthesis

Following iterative optimization, the clock periods were refined to 5 ns (clk_write) and 6 ns (clk_read). The final synthesis results demonstrated successful timing closure:

- Worst Negative Slack (WNS): 0.739 ns
- Total Negative Slack (TNS): 0.000 ns
- Worst Hold Slack (WHS): 0.154 ns
- Failing Endpoints: 0

These results confirm that all timing constraints were satisfied. The positive slack indicates that the design meets performance requirements with margin, validating the effectiveness of pipelining and register balancing in reducing the critical path delay.

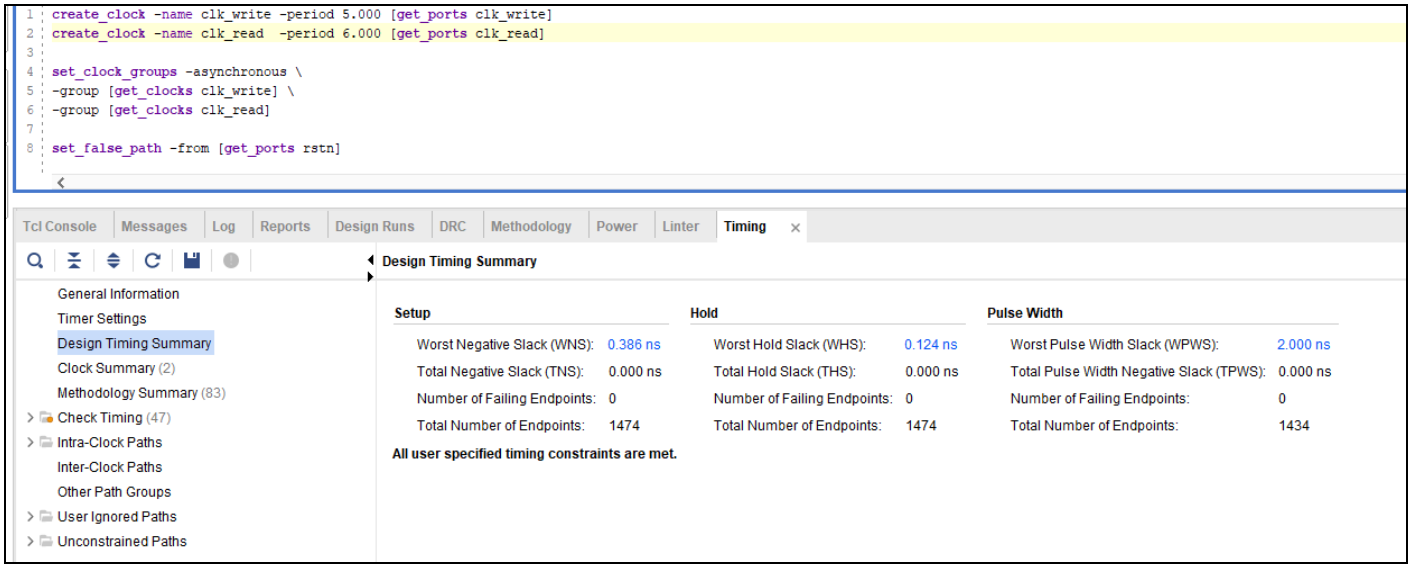


Figure 16. Post-Implementation Timing Summary

The optimized constraints from synthesis were applied during implementation. The final post-implementation timing results confirmed reliable hardware operation:

- Worst Negative Slack (WNS): 0.386 ns
- Total Negative Slack (TNS): 0.000 ns
- Worst Hold Slack (WHS): 0.124 ns
- Failing Endpoints: 0

These results demonstrate that the design operates successfully at:

- 200 MHz (clk_write)
- 166.67 MHz (clk_read)

The slight reduction in slack after implementation is expected due to routing delays introduced during placement and routing. Nevertheless, the positive slack confirms that the design meets all timing requirements on the ZYBO Z7-10 FPGA.

4.3.4 Resource Utilization Analysis

This section evaluates the FPGA resources consumed by the high-throughput SmartNIC design after synthesis and implementation for the ZYBO Z7-10 (Zynq-7000) FPGA. The results demonstrate how architectural choices such as pipelining, parallelism, and asynchronous clock domain crossing influence hardware utilization.

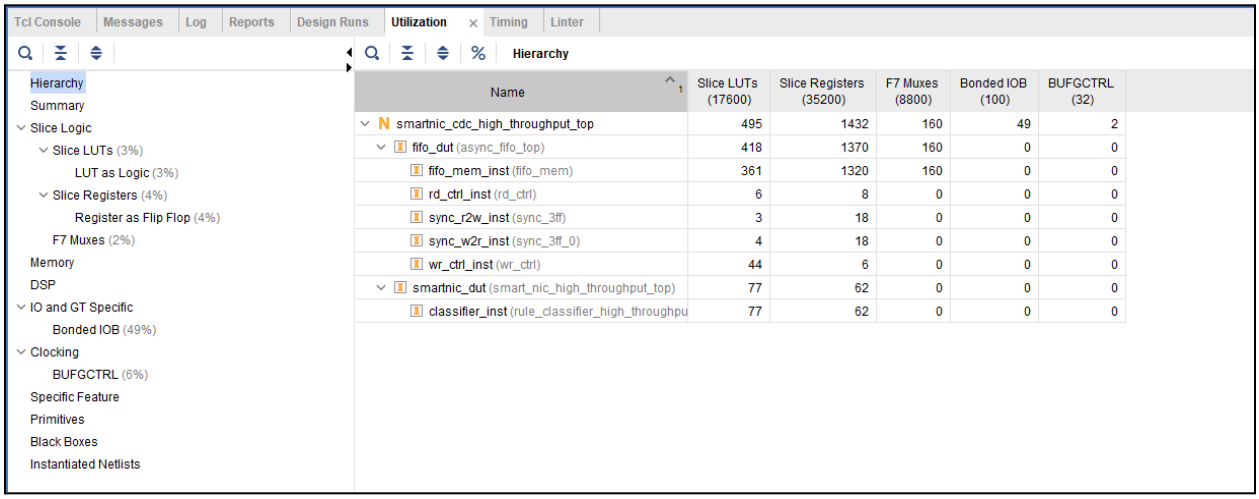


Figure 17. Post-Synthesis Resource Utilization

Figure 17. provides an estimated breakdown of resource consumption of high throughput post-synthesis

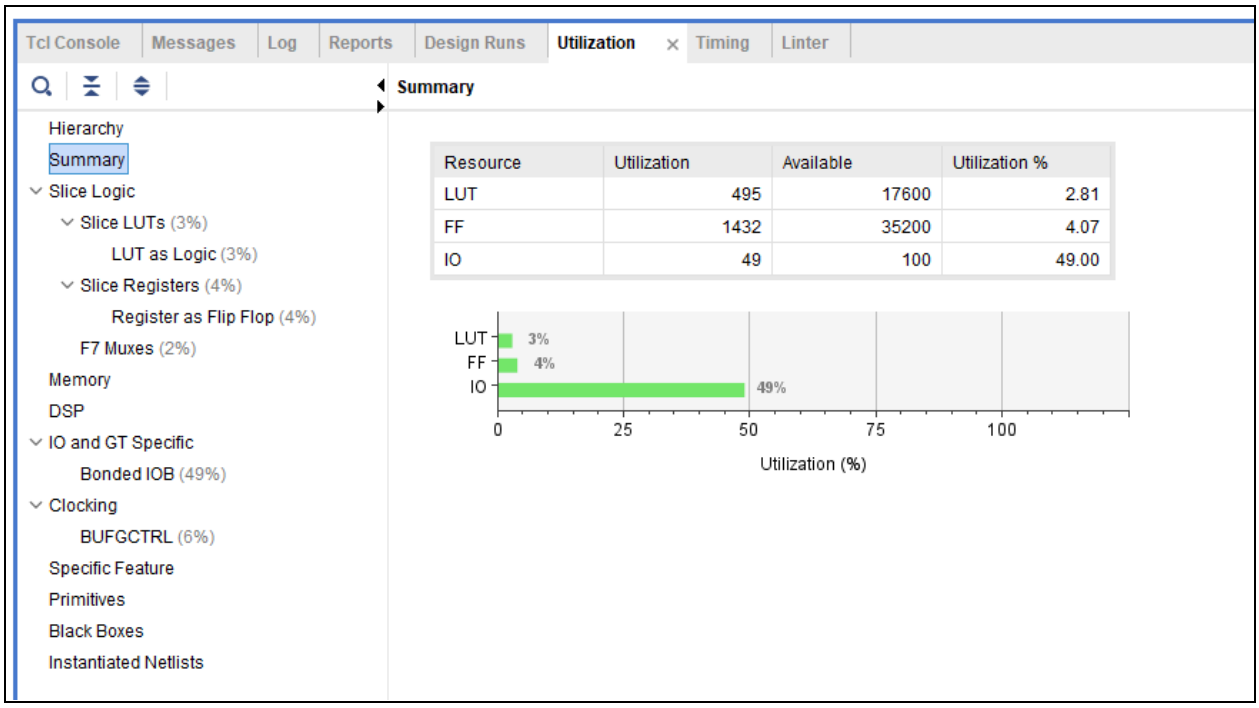


Figure 18. Post-Synthesis Resource Utilization Summary

Table 5. High-Throughput Post-Synthesis Resource Utilization Summary

Resource	Utilized	Available	Utilized
LUT	496	17600	2.81%
Flip-Flops(FFs)	1432	35200	4.07%
I/O Pins	49	100	49.00%

The post-synthesis resource utilization results indicate that the high-throughput SmartNIC design consumes a small portion of the available FPGA resources. The design utilizes approximately 2.81% of LUTs and 4.07% of flip-flops, demonstrating an efficient hardware implementation despite the use of parallel processing techniques.

The LUT usage is primarily attributed to the rule_classifier_high_throughput module, where multiple comparators operate concurrently to enable single-cycle packet classification. Similarly, the flip-flop utilization arises from pipeline registers and synchronization logic required for clock domain crossing through the asynchronous FIFO.

Notably, no DSP blocks or Block RAM (BRAM) were utilized, as the design relies solely on logical comparisons and distributed memory structures. This confirms that packet classification is implemented using combinational and sequential logic rather than arithmetic computation or dedicated memory resources.

The reported 49% I/O utilization at this stage is an estimate generated by Vivado. Since physical pin assignments have not yet been finalized, the tool provides a conservative approximation based on the declared top-level ports. The actual I/O utilization is **expected to be significantly lower after deployment on the physical FPGA, as discussed later in Section 8.**

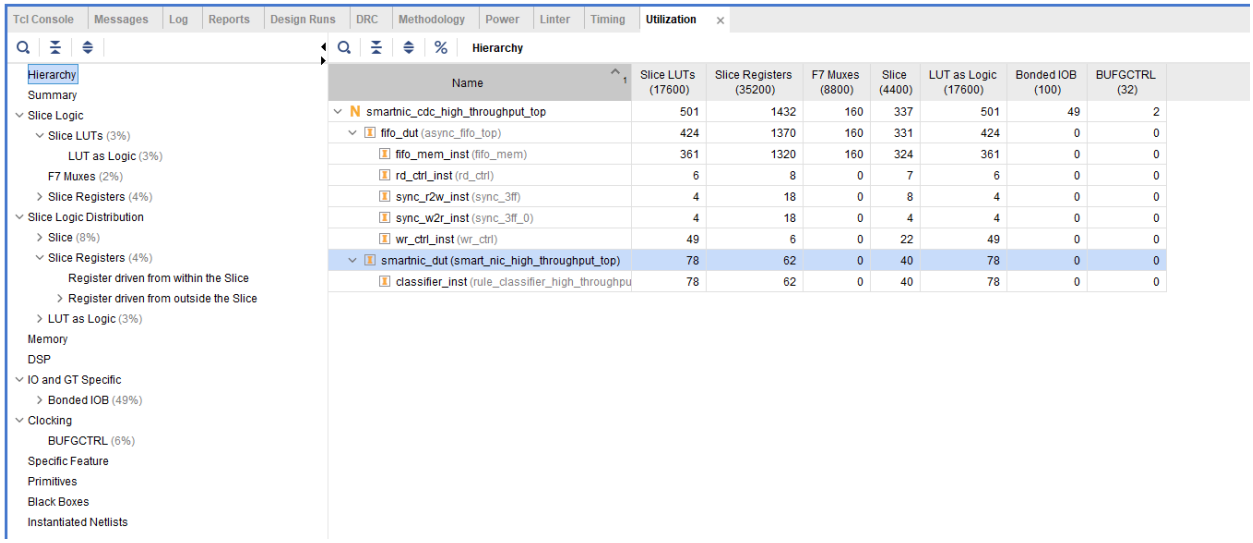


Figure 19. Post-Implementation Resource Utilization on the ZYBO Z7-10 FPGA

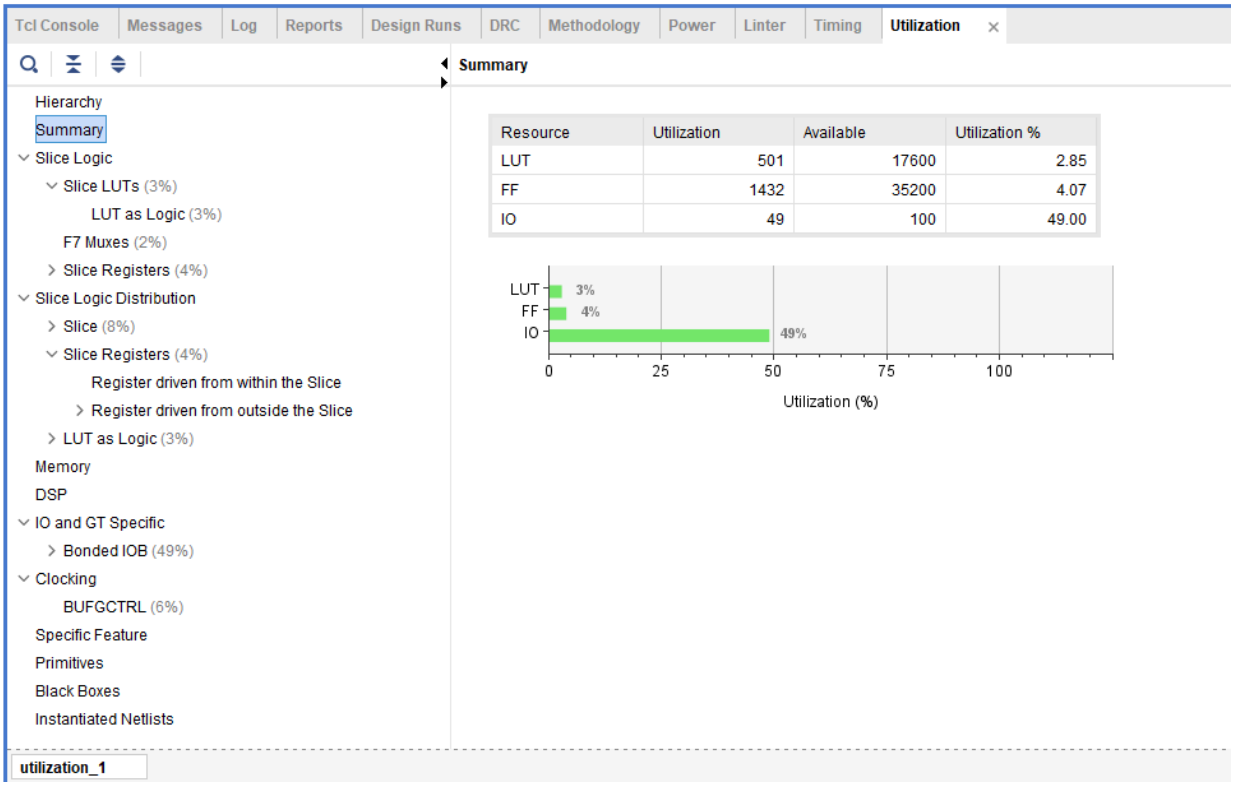


Figure 20. Post-Implementation Resource Utilization Summary

The implementation report presents the actual resource consumption after placement and routing. The results confirm that the design fits within the available FPGA resources.

Table 6. High-Throughput Post-Implementation Resource Utilization Summary

Resource	Utilized	Available	Utilized
LUT	496	17600	2.85%
Flip-Flops(FFs)	1432	35200	4.07%
I/O Pins	49	100	49.00%

Similarly to the post-synthesis results shown in Figure 18, the post-implementation resource utilization in Figure 20 indicates comparable hardware consumption. The design utilizes approximately **2.85% of LUTs** and **4.07% of flip-flops**, confirming that placement and routing did not introduce significant overhead and that the synthesis estimates were accurate. These resources are primarily consumed by the parallel comparator structures within the high-throughput rule classifier, along with pipeline registers and clock domain crossing logic.

As in the synthesis stage, the reported **49% I/O utilization** remains an overestimation generated by Vivado prior to physical pin assignment. This value reflects conservative tool estimation rather than actual hardware usage. During real deployment on the **ZYBO Z7-10 FPGA**, the true I/O utilization is significantly lower at approximately **5% utilization**, as discussed in **Section 8**. This confirms that the implementation results align closely with the synthesis analysis and validates the efficiency of the high-throughput SmartNIC architecture.

4.3.5 Power Analysis

This section presents the power consumption analysis of the high-throughput SmartNIC design. These results provide insight into the impact of pipelining, parallelism, and optimized resource utilization on overall power consumption.

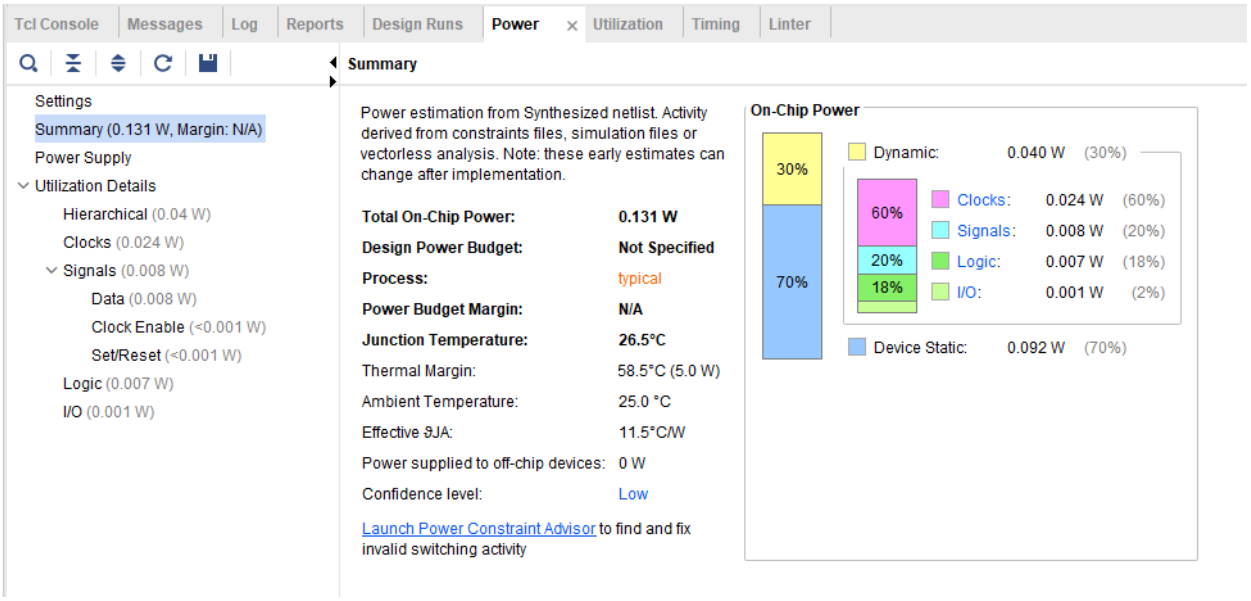


Figure 21. Power Consumption Post-Synthesis

The post-synthesis power report provides an early estimate of power consumption based on switching activity and inferred resource utilization. The total on-chip power is approximately **0.131 W,** indicating that the design is energy-efficient despite its high-throughput architecture.

The majority of the dynamic power is attributed to:

- Clock Networks: Required to drive synchronous pipeline stages.

- Logic Elements (LUTs and Registers): Used extensively in the parallel rule classifier and asynchronous FIFO.

- Signal Switching Activity: Resulting from concurrent packet processing.

This estimate serves as a preliminary evaluation to guide design optimization prior to hardware implementation. The absence of DSP blocks and Block RAM contributes to the relatively low power consumption at this stage.

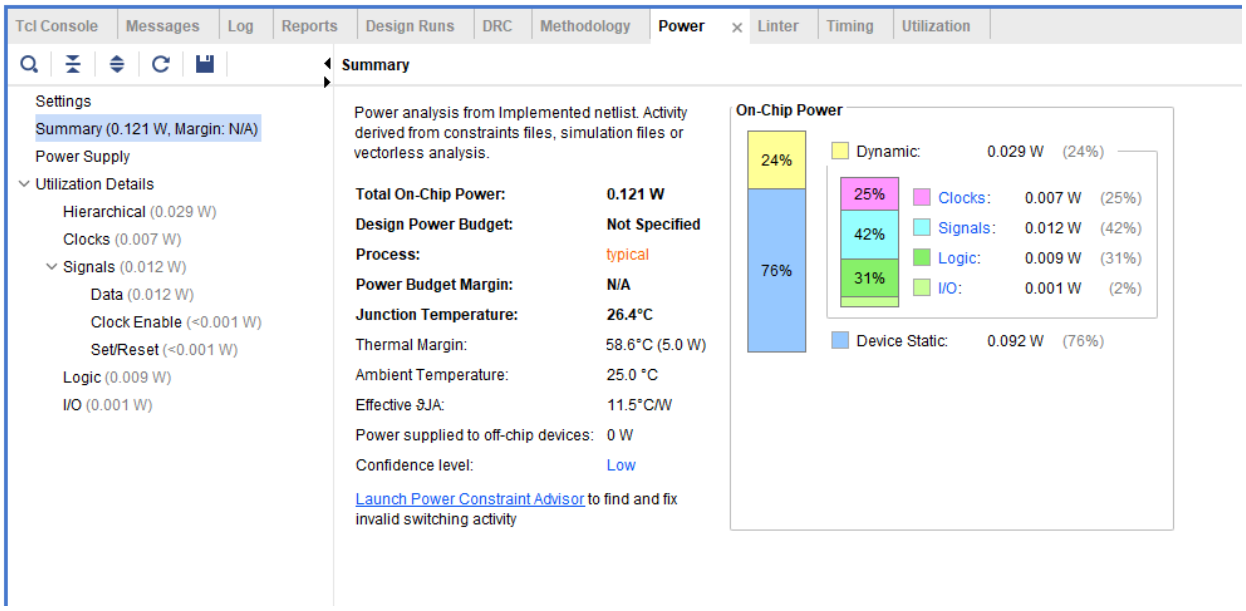


Figure 22. Power Consumption Post-Synthesis

Similarly to the post-synthesis results, the post-implementation power report provides a more accurate estimate after placement and routing. The total on-chip power is approximately **0.121 W**, which is slightly lower than the synthesis estimate due to refined routing and more accurate switching activity calculations.

The power distribution confirms that:

- **Dynamic Power** is primarily consumed by clock networks, logic, and signal transitions within the parallel classification engine.
- **Static Power** remains minimal, reflecting efficient utilization of FPGA resources.
- The reduction from synthesis to implementation demonstrates the effectiveness of Vivado's optimization during placement and routing.

These results indicate that the high-throughput SmartNIC design operates efficiently within acceptable thermal and power limits on the ZYBO Z7-10 FPGA.

5. Version 2 – Area-Optimized Design

This section presents the implementation and evaluation of the area-optimized SmartNIC Packet Classification Accelerator. Unlike the high-throughput design, which prioritizes speed through parallelism, this version focuses on minimizing hardware utilization using resource sharing and sequential processing. All modules were developed in Verilog HDL and synthesized using Xilinx Vivado for the ZYBO Z7-10 FPGA platform.

5.1 Architecture

The area-optimized classifier uses **resource sharing** by evaluating rules sequentially instead of in parallel. Rather than instantiating multiple comparators, a single comparison unit is reused across multiple clock cycles, significantly lowering logic utilization and routing complexity.

The architecture consists of three main components:

- **Input Registers** – store packet fields (e.g., destination address, protocol)
- **Control Unit (FSM/Counter)** – iterates through rules
- **Shared Comparator Logic** – evaluates one rule per cycle

Operation Flow:

1. Packet is loaded into registers
2. Control unit initializes rule index
3. Each cycle, one rule is compared
4. Match is detected or all rules are exhausted
5. Result is output

This sequential approach significantly reduces FPGA resource consumption at the cost of increased latency, making it suitable for area-constrained applications.

File Structure

Table 7. Area-Optimized Design Files

File Name	Description
smartnic_cdc_area_optimized_top_for_synthesis.v	Top-level synthesis wrapper integrating CDC FIFO and classification pipeline

cdc_fifo_area.v	Asynchronous FIFO for clock domain crossing
packet_parser.v	Extracts protocol and IP header fields from packets
rule_classifier_area_optimized.v	Sequential rule classifier using resource sharing
packet_parser_rule_classifier_top_sim.v	Integrates parser and area-optimized classifier
snic_area_cdc_testbench.v	Functional simulation testbench
smartnic_cdc_constraints.xdc	Timing constraints for synthesis and implementation
pck_stream.mem	Packet dataset used for verification

5.2 Implementation Detail

The RTL implementation realizes the architecture using a multi-cycle, state-driven design. The design prioritizes reduced hardware utilization while maintaining similar functional correctness.

- **Key Registers:**
  - key_reg: stores concatenated packet fields
  - rule_idx: tracks current rule
  - busy: indicates active processing
  - matched_rule_id: stores result
- **Control Logic:**
  A counter or FSM increments rule_idx each cycle, enabling sequential rule evaluation. Processing continues until a match is found or all rules are checked.
- **Rule Storage:**
  Rules are implemented using a case statement or small memory structure. Each cycle selects one rule based on rule_idx.
- **Comparator Logic:**
  A single comparator evaluates:
  - if (key_reg == rule_key)
  - This shared logic is reused across all rules, minimizing area.
- **Optimization Strategies:**
  - **Resource sharing:** single comparator reused across cycles
  - **Minimal datapath width:** only required fields extracted
  - **Sequential control:** avoids parallel hardware duplication

- **RTL Behavior:**
    1. On i_packet_valid, input is latched
    2. rule_idx initialized
    3. Each clock cycle evaluates one rule
    4. On match, output is asserted and processing stops
    5. If no match, classifier outputs default result

This implementation reduces LUT usage and routing complexity, but increases latency due to multi-cycle processing, consistent with area-efficient FPGA design principles.

5.2.1 Packet Parser

File: packet_parser.v

The same packet parser used in the high-throughput architecture is reused in the area-optimized design. This ensures consistency in packet preprocessing across both implementations and enables fair performance comparisons.

The module extracts essential header fields from the 76-bit packet format, including:

- IP version
- Protocol
- Source IP address
- Destination IP address

The parser is implemented using combinational logic to avoid additional latency. It serves as the first stage of the SmartNIC pipeline, providing structured packet fields to the downstream classifier. Reusing this module maintains design uniformity while reducing development complexity.

5.2.2 Area-Optimized Rule Classifier

This module implements the core area-efficient packet classification engine. Unlike the high-throughput design, which evaluates all rules in parallel, the area-optimized classifier evaluates one rule per clock cycle using a shared comparator.

Key Features

- **Sequential Rule Evaluation:** A counter (rule_idx) iterates through stored rules.
- **Resource Sharing:** A single comparator is reused across cycles, reducing LUT utilization.
- **Packet ID Tracking:** Ensures accurate mapping between input packets and classification results.

- **Handshake Mechanism:** The o_ready_for_packet signal enables controlled packet acceptance.
- **Result Signaling:** The o_result_done signal indicates classification completion.

This design significantly reduces hardware consumption while introducing multi-cycle latency, demonstrating the trade-off between area efficiency and throughput.

5.2.3 Asynchronous FIFO

This module implements the asynchronous FIFO responsible for clock domain crossing between the packet injection and classification stages. Unlike the FIFO used in the high-throughput architecture, this version incorporates additional control logic tailored to the sequential nature of the area-optimized classifier.

Key Enhancements

- **Consumer-Ready Gating:** FIFO reads occur only when the classifier is ready to accept new data.
- **Packet Alignment Mechanism:** Prevents packet loss and misalignment during multi-cycle processing.
- **Gray-Code Pointer Synchronization:** Ensures safe data transfer between asynchronous clock domains.
- **Three-Stage Synchronizers:** Reduce metastability risks.
- **Packet ID Tracking:** Maintains correct packet-to-result association.

FIFO Submodules

- wr_ctrl – Write pointer and full detection logic
- rd_ctrl – Read pointer and empty detection logic
- sync_3ff – Three-stage synchronizers for CDC reliability
- fifo_mem – Dual-clock memory storage
  async_fifo_top_area – Top-level FIFO integration module

These modifications ensure reliable data transfer and correct synchronization between clock domains while supporting the sequential processing requirements of the area-optimized classifier.

5.2.4 Top-Level Synthesis Wrapper

File: smartnic_cdc_area_optimized_top_for_synthesis.v

This file serves as the synthesis-level wrapper for the complete area-optimized SmartNIC design. It integrates the asynchronous FIFO with the classification pipeline, enabling full-system RTL elaboration and implementation in Vivado.

Functional Responsibilities

- Connects the FIFO write and read domains
- Generates packet-valid and packet-ID signals
- Ensures proper clock domain crossing
- Provides system-level integration for synthesis and implementation

This wrapper was necessary because the simulation and synthesis hierarchies differ. It enables accurate analysis of the complete CDC-enabled architecture within Vivado.

5.2.5 RTL Elaboration Diagram

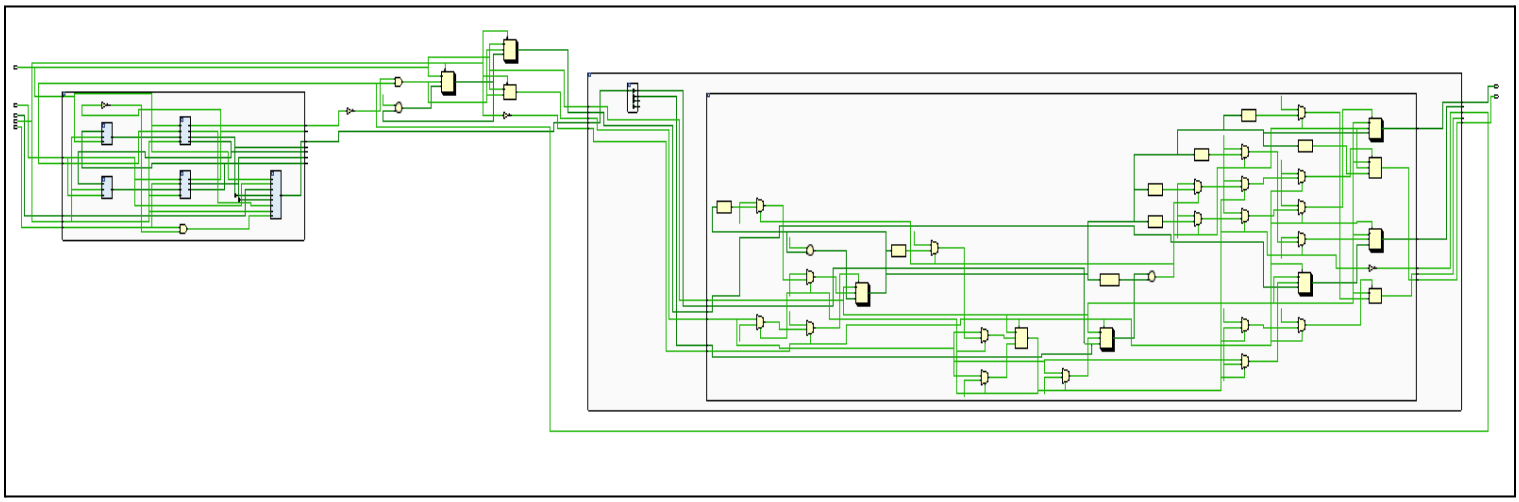


Figure 23. Fully Expanded RTL Schematic of the Area-Optimized Classifier.

This figure illustrates the hierarchical RTL elaboration of the area-optimized SmartNIC architecture. The asynchronous FIFO is positioned on the left, while the packet parser and sequential rule classifier appear on the right. Compared to the high-throughput architecture, this design occupies significantly less area due to the use of resource sharing and multi-cycle processing. Instead of deploying multiple parallel comparators, the classifier evaluates one rule per clock cycle using a single comparator. This reduction in parallel hardware results in lower LUT utilization and routing complexity.

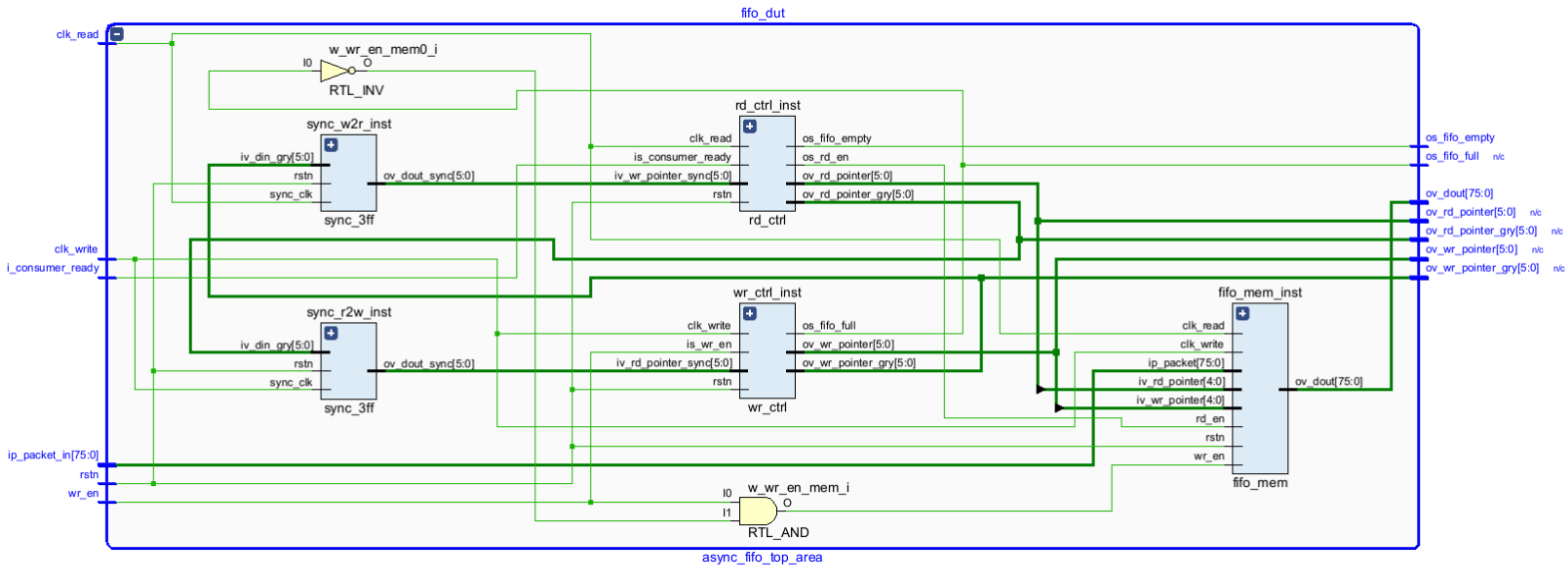


Figure 24. Expanded RTL Elaboration Showing the Asynchronous FIFO Structure.

As shown in Figure 24, the asynchronous FIFO facilitates safe clock domain crossing between the write and read domains. The design includes Gray-coded pointers, three-stage synchronizers, read and write controllers, and dual-clock memory to ensure reliable data transfer.

A key enhancement in the area-optimized design is the inclusion of the i_consumer_ready signal. This handshake mechanism ensures that data is read from the FIFO only when the classifier is ready to process a new packet. Without this control, packets could be skipped or overwritten if the FIFO continued advancing while the classifier remained busy performing sequential rule evaluations. By synchronizing FIFO reads with classifier readiness, this mechanism prevents packet loss, maintains packet-to-result alignment, and guarantees correct operation in a multi-cycle architecture.

Compared to the high-throughput implementation, the FIFO integration here supports controlled data flow rather than continuous streaming, making it better suited for the latency-aware, area-efficient design.

5.3 Results

5.3.1 Post-Synthesis Schematic

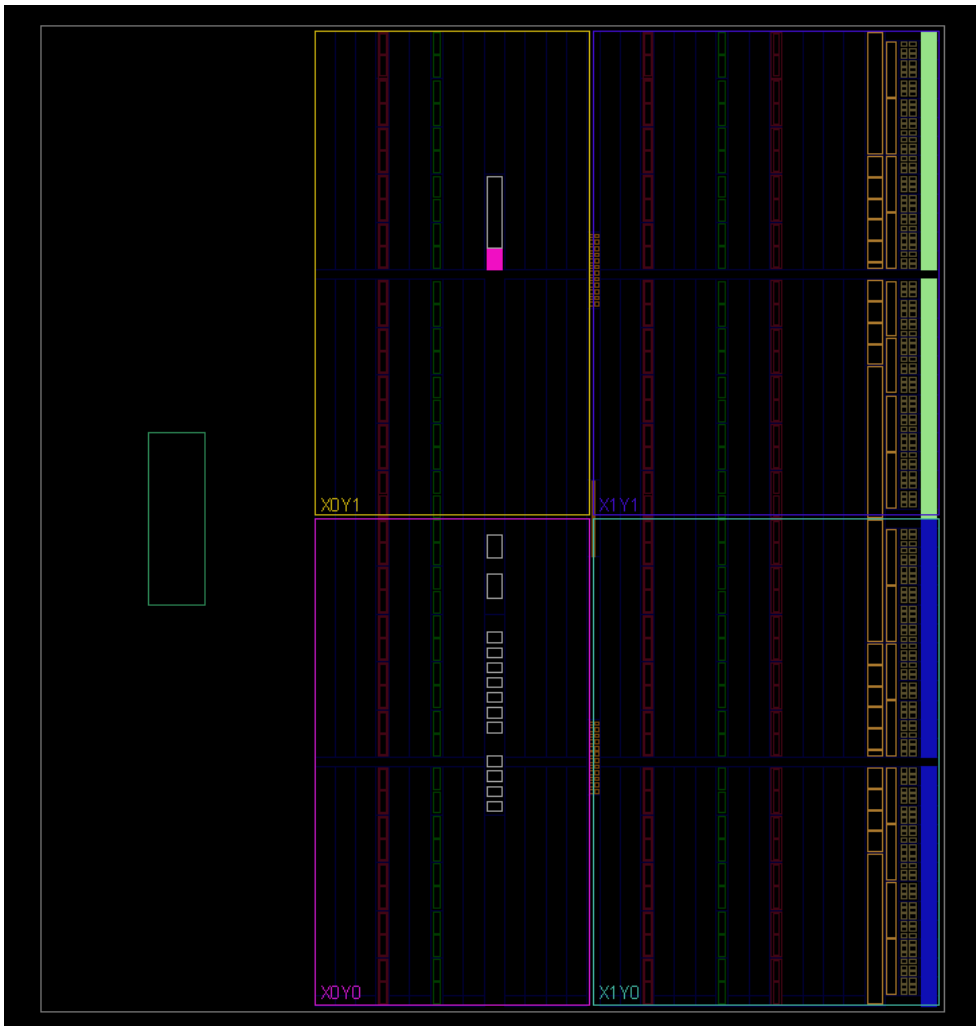


Figure 25. Post-Synthesis Device Utilization for the ZYBO Z7-10 FPGA

Figure 25. presents the post-synthesis device utilization. At this stage, the design has been translated into a gate-level netlist but has not yet undergone placement and routing. Physical placement and routing are performed during the implementation stage and are therefore not reflected in this view.

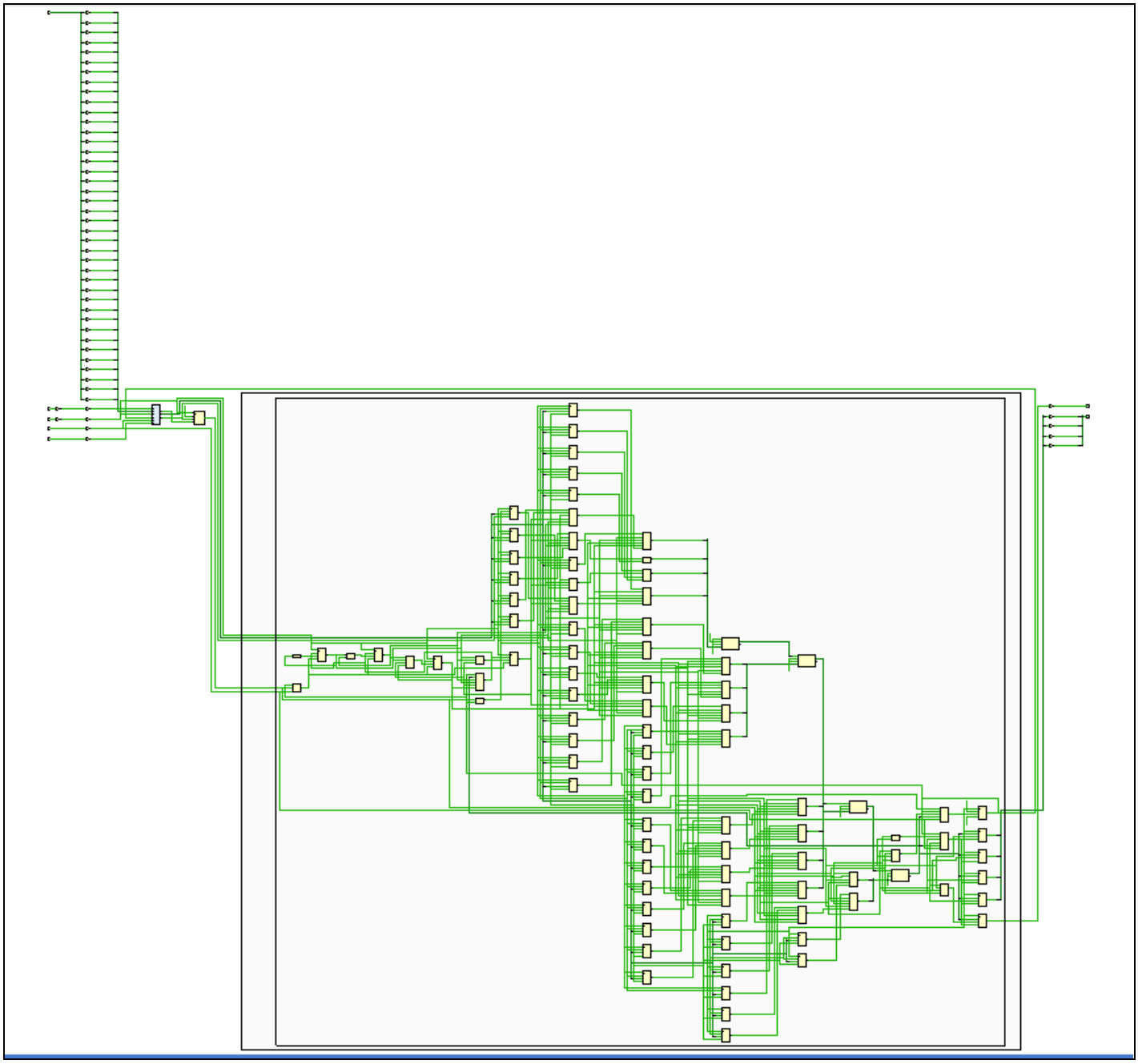


Figure 26: Post-Synthesis Gate-Level Schematic of the Area-Optimized SmartNIC Design

Figure 26. presents the gate-level netlist generated after RTL synthesis of the area-optimized SmartNIC. The elongated structures on the left correspond to the asynchronous FIFO, which buffers packet data and enables safe clock domain crossing between the write and read domains. Their vertical expansion reflects the FIFO's storage depth and data width as synthesized into distributed registers and memory resources.

The logic blocks toward the center and right implement packet parsing and sequential rule classification. Unlike the high-throughput design, the classifier here consists of only a few stages and is not pipelined. Instead, it relies on register reuse and a shared comparator, as defined in the rule_classifier_area_optimized.v module. The key_reg, rule_idx, and busy registers control the multi-cycle evaluation process, where one rule is checked per clock cycle using a case-statement-based rule table.

This compact structure demonstrates resource sharing in hardware. By eliminating parallel comparator arrays and deep pipeline registers, the design significantly reduces LUT and flip-flop utilization. The flattened appearance of the schematic results from Vivado's synthesis process, which decomposes high-level RTL modules into primitive FPGA components such as lookup tables, multiplexers, and registers.

The schematic confirms that the RTL implementation faithfully realizes the intended area-optimized architecture, achieving reduced hardware consumption at the expense of increased classification latency.

5.3.2 Post-Implementation Schematic

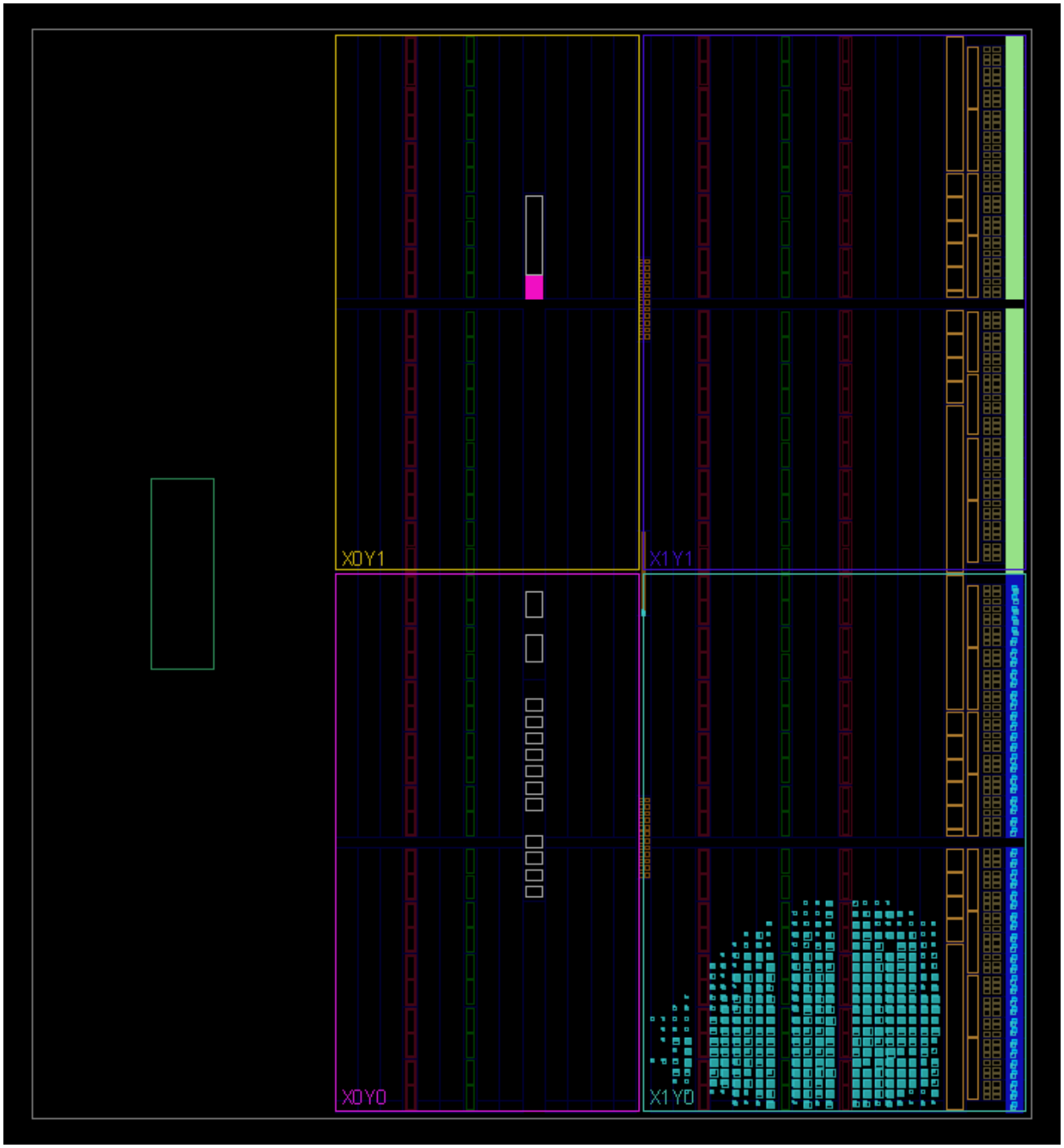


Figure 27. Post-Implementation Device Utilization of the Area-Optimized SmartNIC for the ZYBO Z7-10 FPGA

Figure 27. shows the post-implementation device utilization of the area-optimized SmartNIC on the ZYBO Z7-10 FPGA. The cyan-highlighted region represents the portion of the FPGA fabric occupied after placement and routing. Compared to the high-throughput design, this

implementation occupies slightly more physical space due to additional control logic, including handshake mechanisms, packet ID tracking, and FIFO gating required for reliable multi-cycle processing. The layout confirms the successful mapping of the RTL design onto the FPGA while maintaining efficient resource utilization and functional correctness.

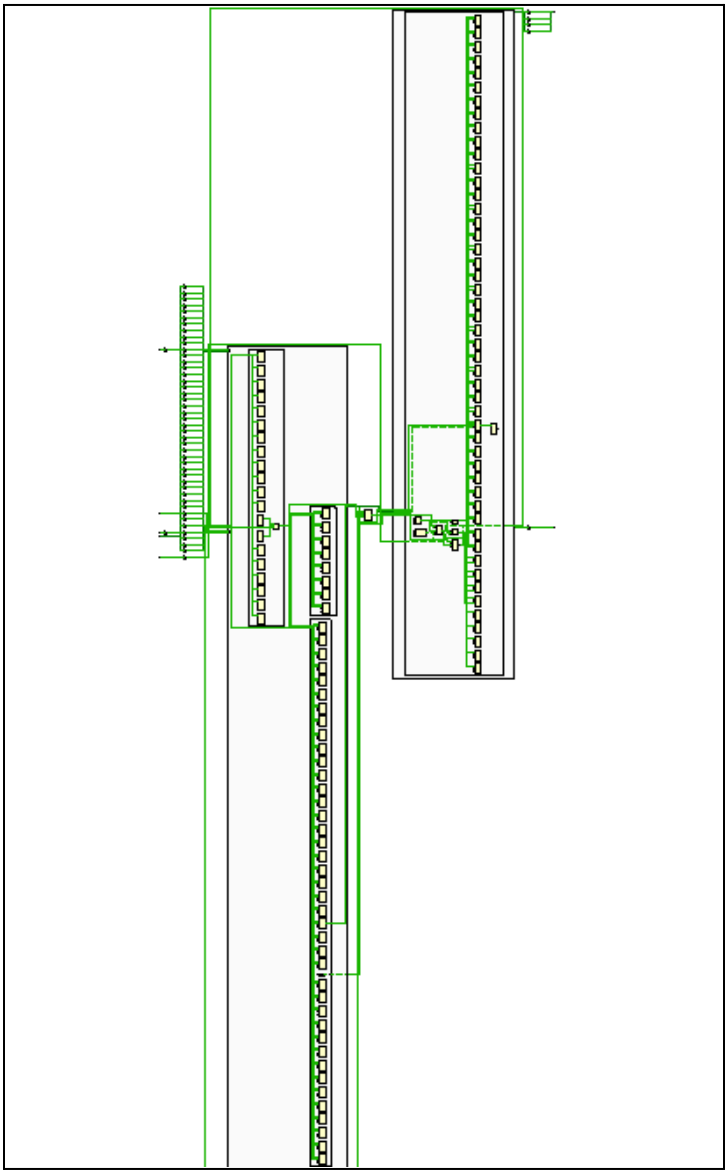


Figure 27. Post-Implementation Gate-Level Schematic of the Area-Optimized SmartNIC Design

This figure presents the complete post-implementation schematic of the area-optimized SmartNIC. The elongated structures correspond to the asynchronous FIFO and associated register arrays, which facilitate clock domain crossing and packet buffering. The compact

logic blocks reflect the sequential classifier and parser, illustrating the reduced hardware footprint achieved through resource sharing.

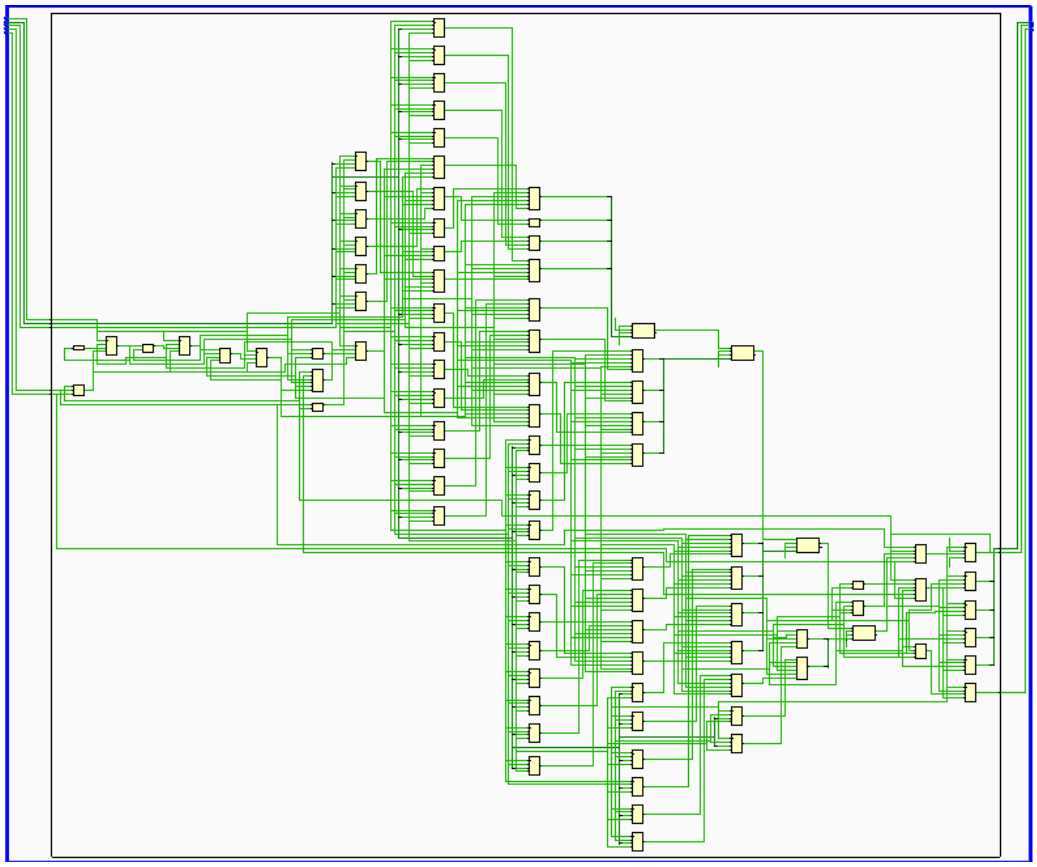


Figure 28. Post-Implementation Gate-Level Schematic of the Area-Optimized Rule Classifier

This figure shows the synthesized schematic of the area-optimized rule classifier. Unlike the high-throughput design, the classifier uses a single shared comparator and sequential control logic to evaluate rules over multiple clock cycles. The reduced number of logic elements and absence of deep pipelining demonstrate its efficiency in minimizing LUT and register utilization, confirming that the architecture is optimized for area.

5.3.3 Timing Analysis

This section presents the timing analysis of the area-optimized SmartNIC design. The same timing constraints that successfully achieved closure in the high-throughput architecture were applied here to ensure consistency and enable a fair comparison. Specifically, a clock period of **5 ns (200 MHz)** was used for clk_write, and **6 ns (166.67 MHz)** for clk_read. Asynchronous clock groups were defined to prevent false timing violations between clock domains, and a false path was specified for the reset signal. The results confirm that the area-optimized design meets timing requirements under identical operating conditions. This is primarily due to its sequential, resource-sharing architecture, which introduces shorter combinational paths compared to the deeply pipelined parallel classifier in the high-throughput design.

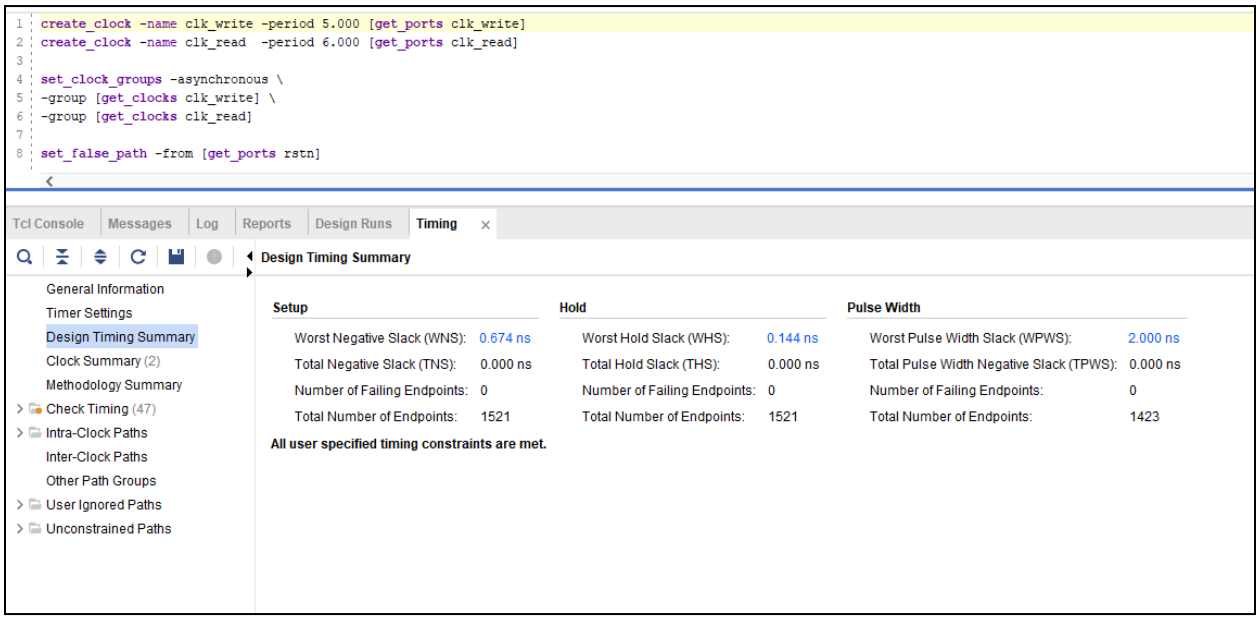


Figure 29. Timing Constraints and Analysis of the Area-Optimized SmartNIC Design

Figure 29 presents the post-synthesis timing results of the area-optimized SmartNIC using the same constraints as the high-throughput design. The synthesis report indicates a Worst Negative Slack (WNS) of 0.674 ns, a Worst Hold Slack (WHS) of 0.144 ns, and a Total Negative Slack (TNS) of 0.000 ns, with no failing endpoints. These results confirm that the design satisfies all setup and hold timing requirements. The positive slack is attributed to the sequential rule evaluation strategy, which reduces the critical path by reusing a single comparator instead of instantiating multiple parallel comparison units.

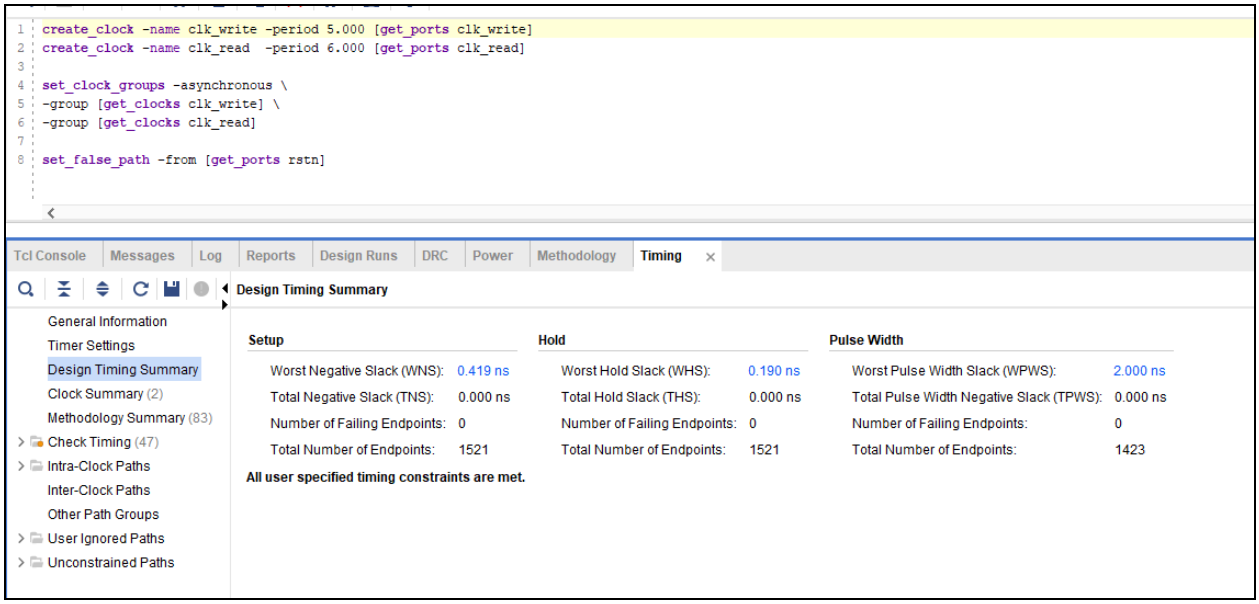


Figure 30. Post-Implementation Timing Summary of the Area-Optimized SmartNIC Design

Figure 30 illustrates the post-implementation timing performance after placement and routing for the ZYBO Z7-10 FPGA. The design continues to meet all specified constraints, achieving a Worst Negative Slack (WNS) of 0.419 ns, a Worst Hold Slack (WHS) of 0.190 ns, and a Total Negative Slack (TNS) of 0.000 ns, with zero failing endpoints. The slight reduction in setup slack compared to synthesis is expected due to routing delays introduced during implementation. Nevertheless, the positive slack confirms stable operation at 200 MHz for clk_write and 166.67 MHz for clk_read. Compared to the high-throughput architecture, the area-optimized design demonstrates improved timing margins due to its reduced combinational depth and efficient resource-sharing approach.

5.3.4 Resource Utilization Analysis

This section evaluates the FPGA resource consumption of the area-optimized SmartNIC design after synthesis and implementation. The results demonstrate that the sequential, resource-sharing architecture significantly reduces hardware utilization.

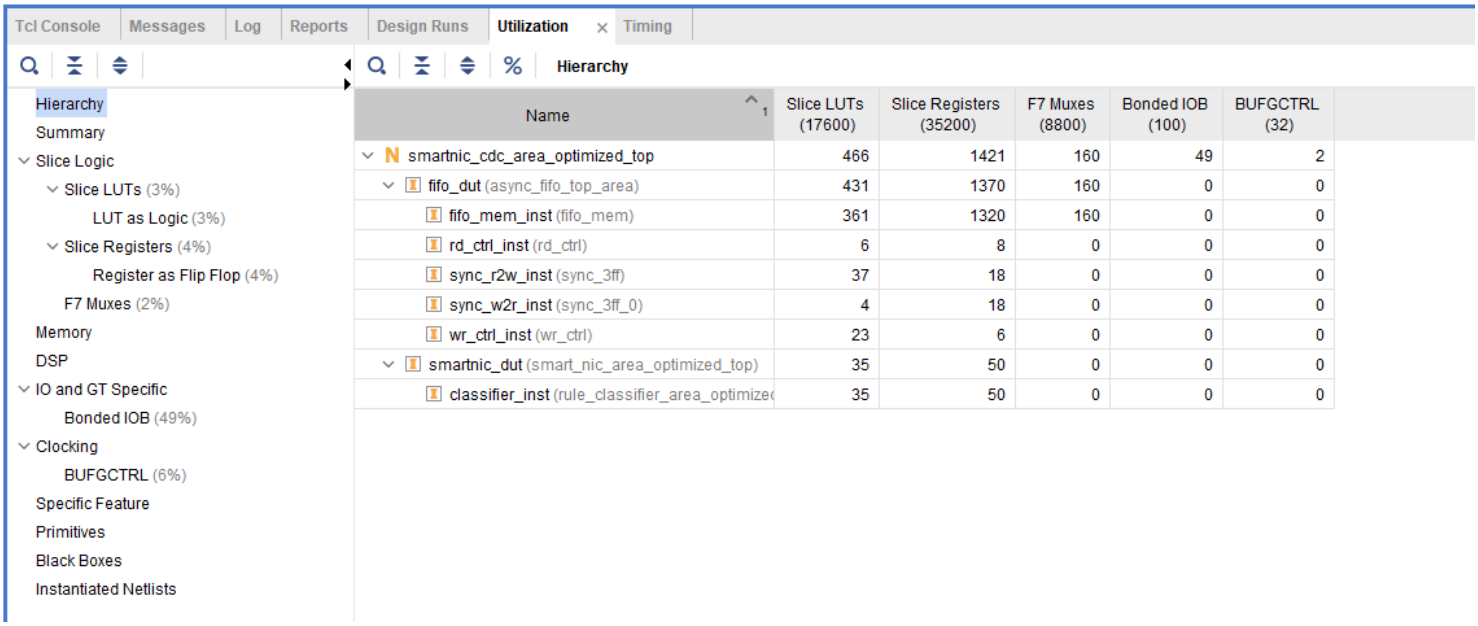


Figure 31. Post-Synthesis Resource Utilization of the Area-Optimized SmartNIC Design

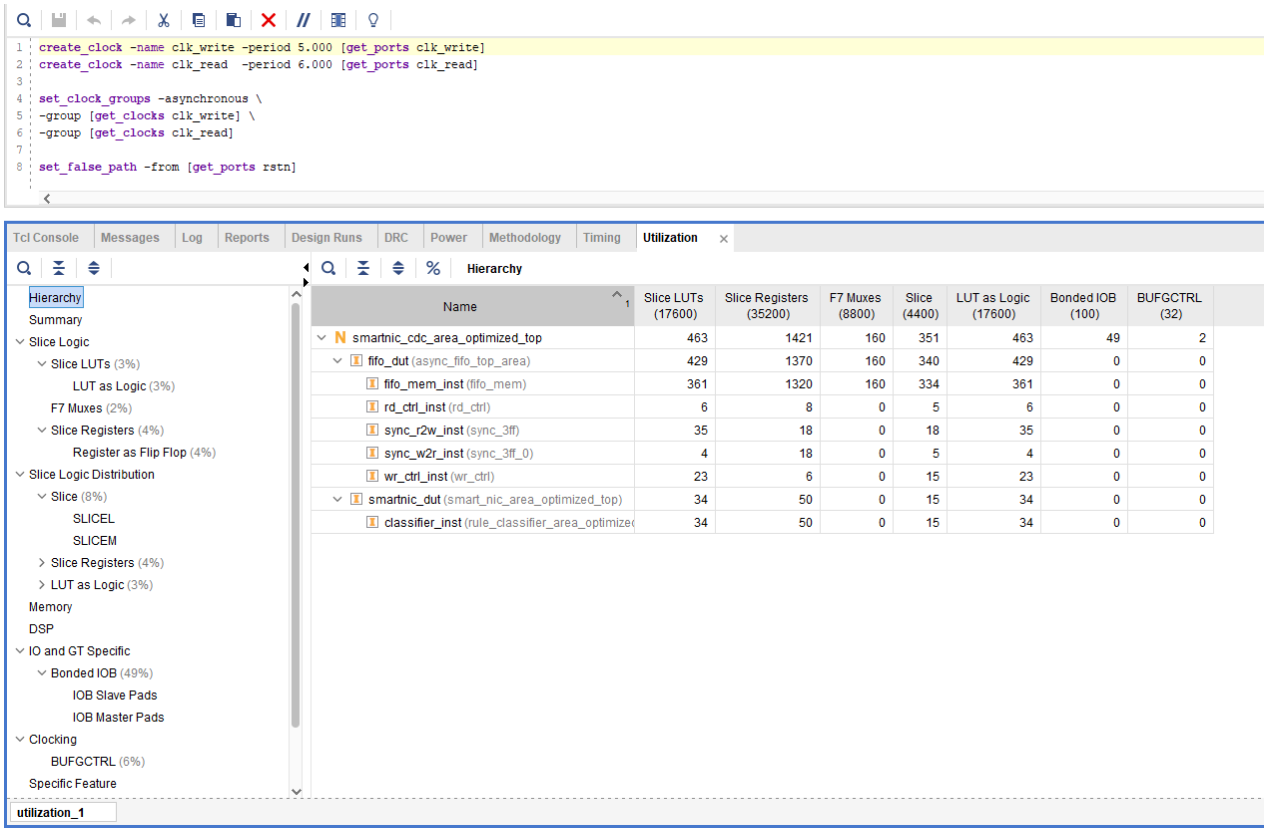


Figure 32. Post-Implementation Resource Utilization of the Area-Optimized SmartNIC on the ZYBO Z7-10 FPGA

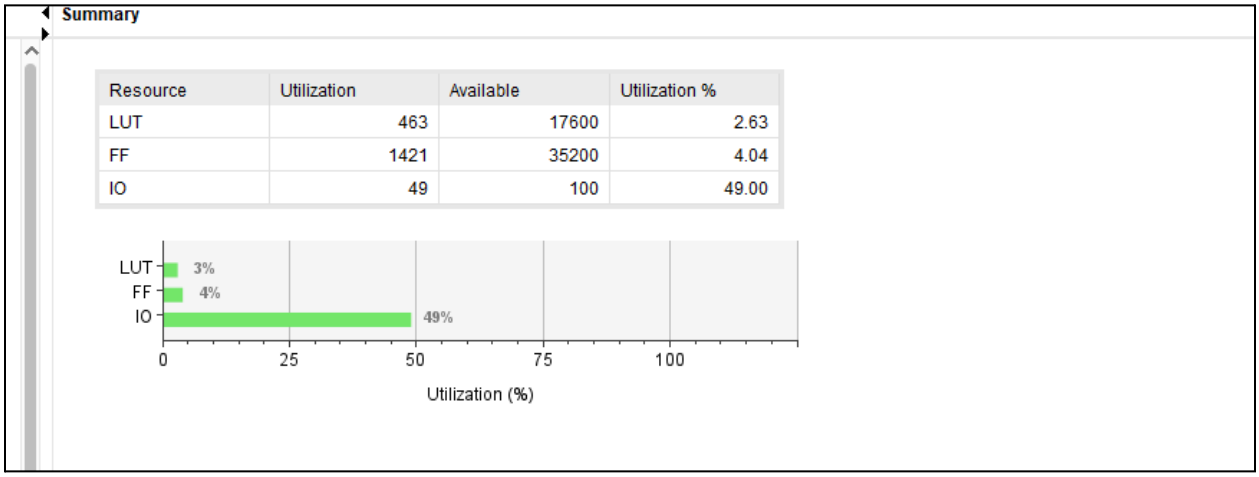


Figure 33. Post-Implementation Resource Utilization Summary of the Area-Optimized SmartNIC for the ZYBO Z7-10 FPGA

Figure 33. presents the post-synthesis resource utilization summary of the area-optimized SmartNIC. The design consumes approximately 2.63% of LUTs and 4.04% of flip-flops, confirming its efficiency. The reduced logic utilization is primarily attributed to the reuse of a single comparator and the sequential rule evaluation mechanism implemented in the rule_classifier_area_optimized module.

No DSP blocks or Block RAM (BRAM) were utilized, as packet classification relies on logical comparisons rather than arithmetic operations, and FIFO storage is implemented using distributed registers. The reported 49% I/O utilization is an overestimation generated by Vivado prior to physical pin assignment. As discussed in Section 8, the actual I/O usage on the ZYBO Z7-10 FPGA is significantly lower after real hardware deployment.

5.3.5 Power Report

This section presents the power consumption analysis of the area-optimized SmartNIC design. Both synthesis and implementation estimates are provided to evaluate energy efficiency and validate the suitability of the design for FPGA deployment.

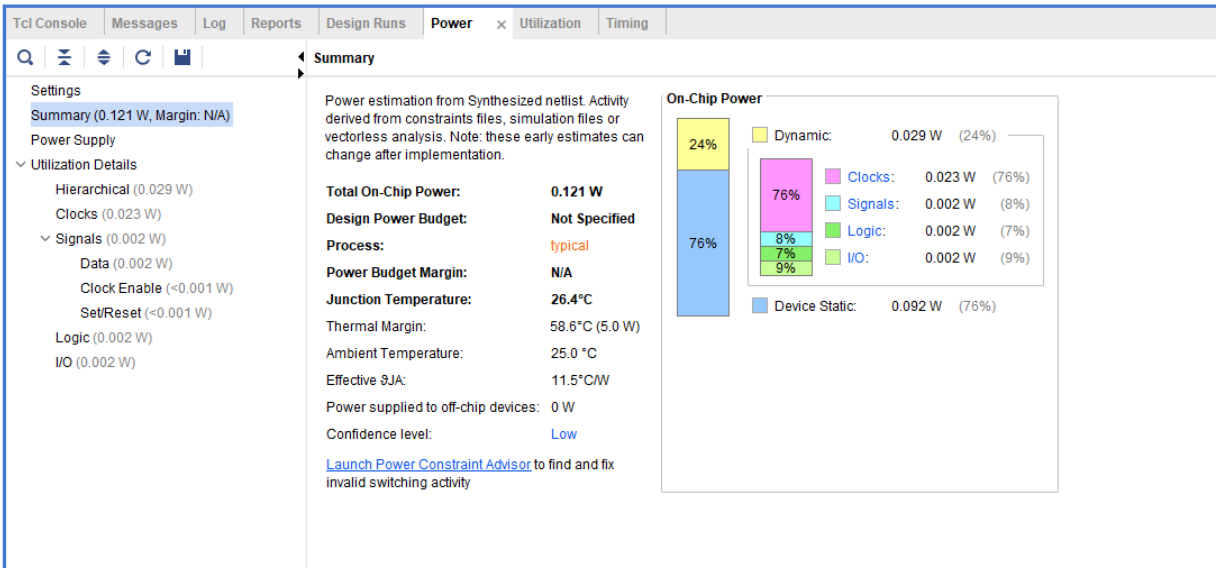


Figure 34. Power Consumption of the Area-Optimized SmartNIC at Synthesis

Figure 34 shows the estimated power consumption obtained after synthesis. The total on-chip power is approximately **0.121 W,** dominated by dynamic power contributions from clock networks, logic elements, and signal transitions. The relatively low power consumption reflects the reduced switching activity resulting from the sequential rule evaluation strategy and minimized parallel hardware.

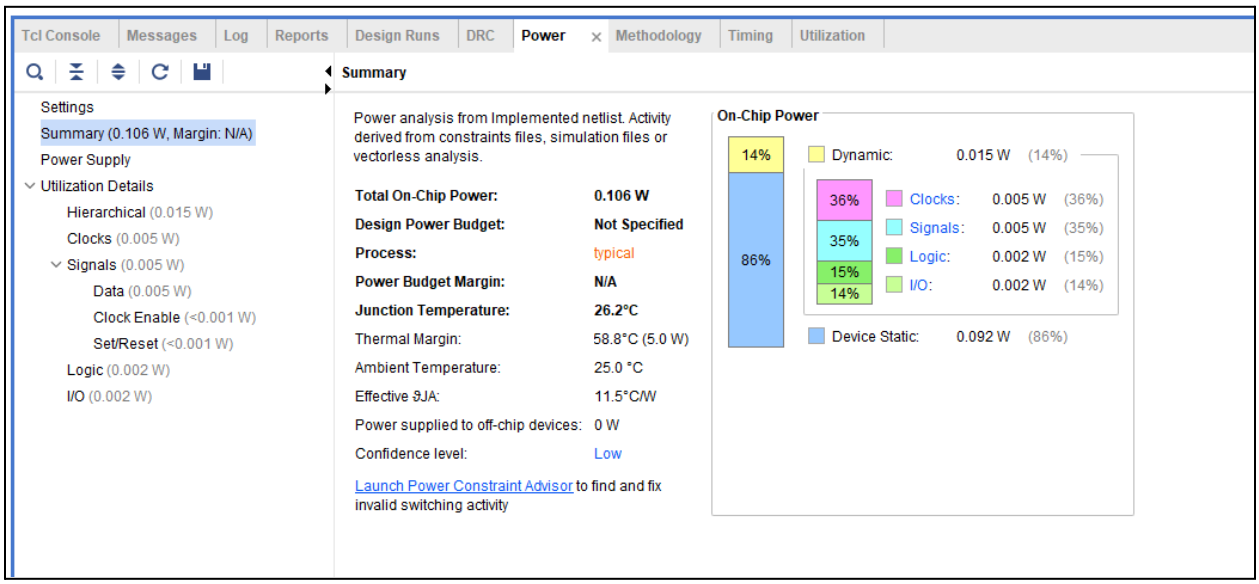


Figure 35. Post-Implementation Power Consumption of the Area-Optimized SmartNIC

Figure 35 presents the post-implementation power analysis of the area-optimized SmartNIC design after placement and routing on the ZYBO Z7-10 FPGA. The total on-chip power consumption is **0.106 W,** indicating a highly energy-efficient implementation.

Static power dominates the overall consumption at 0.092 W (86%), which represents the inherent leakage and biasing power of the FPGA device. In contrast, dynamic power accounts for only 0.015 W (14%), reflecting the low switching activity of the sequential, resource-sharing architecture. Within the dynamic component, clocks and signals each contribute approximately 36% and 35%, respectively, while logic and I/O contribute about 15% and 14%, confirming the lightweight computational footprint of the design.

The low dynamic power is attributed to the reuse of a single comparator and the absence of large parallel processing units. These results demonstrate that the area-optimized architecture achieves excellent power efficiency while maintaining correct functionality and timing closure.

6. Functional Simulations

Functional simulations were conducted to verify correct packet parsing, rule matching, and clock domain crossing (CDC) across both SmartNIC architectures. These simulations validated system functionality, timing behavior, and data integrity under realistic operating conditions. Waveform analyses and testbench outputs confirm correct classification results and demonstrate compliance with design specifications.

To ensure completeness, individual modules, including the packet parser, rule classifiers, and asynchronous FIFO, were tested independently before system-level verification. Corner cases such as FIFO empty/full conditions, asynchronous clock interactions, and packet sequencing were evaluated to confirm robust and reliable operation.

6.1 High-Throughput SmartNIC Design

6.1.1 High-Throughput SmartNIC Top Module

File: packet_parser_rule_classifier_top_sim.v
Module Name: smart_nic_high_throughput_top

This module integrates the packet parser and the high-throughput rule classifier for system-level functional simulation. The parser extracts the packet fields, and the classifier evaluates all rules in parallel to produce a classification result after pipeline latency.

6.1.2 Functional Simulation Testbench

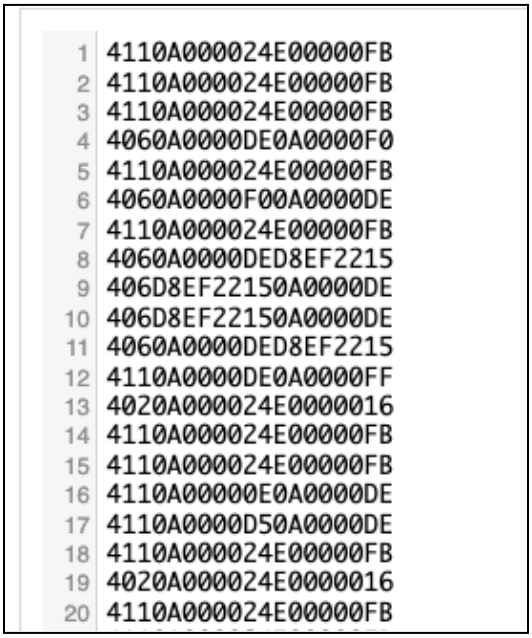
File: snic_throughput_cdc_tb.v

The testbench verifies the high-throughput SmartNIC by injecting packets into the FIFO write domain and validating classification results in the read domain.

Key Features:

- Dual-clock domain verification
- Packet injection using pck_stream.mem
- Parallel rule classification validation
- FIFO full and empty condition testing
- Packet ID tracking for result alignment
- Debug outputs for pointer and classification activity

6.1.3 Simulation Results and Waveform Analysis



```
1  4110A000024E00000FB
2  4110A000024E00000FB
3  4110A000024E00000FB
4  4060A0000DE0A0000F0
5  4110A000024E00000FB
6  4060A0000F00A0000DE
7  4110A000024E00000FB
8  4060A0000DED8EF2215
9  406D8EF22150A0000DE
10 406D8EF22150A0000DE
11 4060A0000DED8EF2215
12 4110A0000DE0A0000FF
13 4020A000024E0000016
14 4110A000024E00000FB
15 4110A000024E00000FB
16 4110A00000E0A0000DE
17 4110A0000D50A0000DE
18 4110A000024E00000FB
19 4020A000024E0000016
20 4110A000024E00000FB
```

Figure 36. Packet Memory Initialization Snippet (pck_stream.mem)

This figure shows a segment of the input packet stream used during functional simulation. Each 76-bit hexadecimal entry represents a test packet injected into the SmartNIC to validate parsing and rule classification.

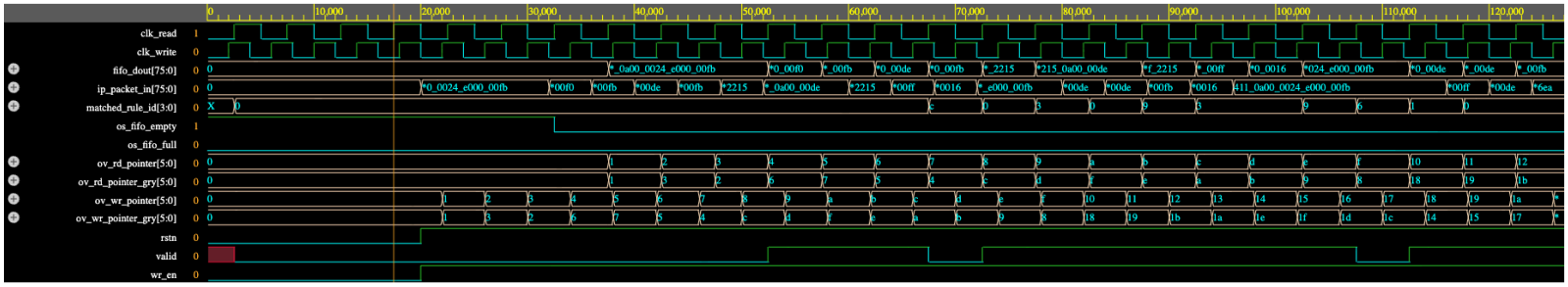


Figure 37. Functional Simulation Waveform of the SmartNIC Design

This waveform illustrates the timing behavior of the SmartNIC, including asynchronous clock operation, FIFO buffering, packet parsing, and rule classification.

```
dumpfile fifo_smartnic_wave.vcd opened for output.
) | cycle=1 | fifo_empty=1 | fifo_full=0 | fifo_dout=0000000000000000000 | valid=0 | rule=0 | wr_ptr=1 | rd_ptr=0
) | cycle=2 | fifo_empty=1 | fifo_full=0 | fifo_dout=0000000000000000000 | valid=0 | rule=0 | wr_ptr=2 | rd_ptr=0
) | cycle=3 | fifo_empty=1 | fifo_full=0 | fifo_dout=0000000000000000000 | valid=0 | rule=0 | wr_ptr=3 | rd_ptr=0
) | cycle=4 | fifo_empty=0 | fifo_full=0 | fifo_dout=0000000000000000000 | valid=0 | rule=0 | wr_ptr=4 | rd_ptr=0
) | cycle=5 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a000024e00000fb | valid=0 | rule=0 | wr_ptr=6 | rd_ptr=1
) | cycle=6 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a000024e00000fb | valid=0 | rule=0 | wr_ptr=7 | rd_ptr=2
) | cycle=7 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a000024e00000fb | valid=0 | rule=0 | wr_ptr=8 | rd_ptr=3
) | cycle=8 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=1 | rule=0 | wr_ptr=9 | rd_ptr=4
) | cycle=9 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a000024e00000fb | valid=1 | rule=0 | wr_ptr=11 | rd_ptr=5
) | cycle=10 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000f00a0000de | valid=1 | rule=0 | wr_ptr=12 | rd_ptr=6
) | cycle=11 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a000024e00000fb | valid=0 | rule=12 | wr_ptr=13 | rd_ptr=7
) | cycle=12 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000ded8ef2215 | valid=1 | rule=0 | wr_ptr=14 | rd_ptr=8
) | cycle=13 | fifo_empty=0 | fifo_full=0 | fifo_dout=406d8ef22150a0000de | valid=1 | rule=3 | wr_ptr=16 | rd_ptr=9
) | cycle=14 | fifo_empty=0 | fifo_full=0 | fifo_dout=406d8ef22150a0000de | valid=1 | rule=0 | wr_ptr=17 | rd_ptr=10
) | cycle=15 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000ded8ef2215 | valid=1 | rule=9 | wr_ptr=18 | rd_ptr=11
) | cycle=16 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a0000de0a0000ff | valid=1 | rule=3 | wr_ptr=19 | rd_ptr=12
00 | cycle=17 | fifo_empty=0 | fifo_full=0 | fifo_dout=4020a000024e0000016 | valid=1 | rule=3 | wr_ptr=21 | rd_ptr=13
00 | cycle=18 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a000024e00000fb | valid=1 | rule=9 | wr_ptr=22 | rd_ptr=14
00 | cycle=19 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a000024e00000fb | valid=0 | rule=6 | wr_ptr=23 | rd_ptr=15
00 | cycle=20 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a00000e0a0000de | valid=1 | rule=1 | wr_ptr=24 | rd_ptr=16
00 | cycle=21 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a0000d50a0000de | valid=1 | rule=0 | wr_ptr=26 | rd_ptr=17
```

Figure 38. Simulation Console Output Demonstrating Classification Results

Waveform and Output Analysis (High-Throughput Design)

The functional simulation results validate correct operation of the SmartNIC architecture. As shown in **Figure 36**, packets from pck_stream.mem are injected into the system and processed sequentially. In **Figure 37**, the clk_write and clk_read signals operate asynchronously, confirming reliable clock domain crossing through the FIFO. Packet data written as ip_packet_in is transferred to fifo_dout when the FIFO is not empty, while the valid signal asserts after an initial pipeline latency of approximately three clock cycles. The corresponding matched_rule_id aligns with the predefined rule set, confirming accurate classification. For example, the first few packets produce valid outputs after the expected

latency of 3, demonstrating correct parsing and rule matching. The FIFO pointers (ov_wr_pointer and ov_rd_pointer) increment appropriately, indicating proper buffer management without overflow or underflow.

Figure 38 provides the console-based verification of these results. Each entry logs FIFO status, pointer values, classification decisions, and packet identifiers. The incremental write and read pointers confirm orderly data transfer across clock domains. Occasional non-consecutive pointer increments, such as transitions from 9 to 11 occur because FIFO updates are gated by valid write and read enable conditions; when no transaction occurs during a clock cycle, the pointer holds its value. This behavior is expected and verifies correct handshake-controlled FIFO operation. Together, the waveform and textual output confirm that all functional requirements and corner cases, including CDC behavior, packet sequencing, and rule validation are satisfied.

6.2 Area-Optimized SmartNIC Design

6.2.1 Area-Optimized SmartNIC Top Module

File: packet_parser_rule_classifier_top_sim.v
Module Name: smart_nic_area_optimized_top

This module integrates the packet parser and the area-optimized rule classifier for system-level functional simulation. The parser extracts the required packet fields from the 76-bit input packet, and the classifier then evaluates the parsed fields sequentially against the predefined rule set. Unlike the high-throughput version, this architecture processes one rule per clock cycle using shared comparison logic, which reduces hardware complexity at the cost of higher latency.

Pipeline Stages

1. Packet Parsing
2. Sequential Rule Classification

This module demonstrates the low-area SmartNIC architecture and is used in simulation, synthesis, and implementation analyses.

6.2.2 Functional Simulation Testbench

File: snic_area_cdc_testbench.v

The testbench verifies the functionality of the area-optimized SmartNIC design. It injects packets into the FIFO write domain and evaluates classification results in the read domain.

Key Features

- Dual-clock domain verification
- Packet injection using pck_stream.mem
- Packet ID tracking for result validation
- Debug outputs for FIFO and classifier activity
- Validation of CDC behavior and sequential classification

6.2.3 Simulation Results and Waveform Analysis

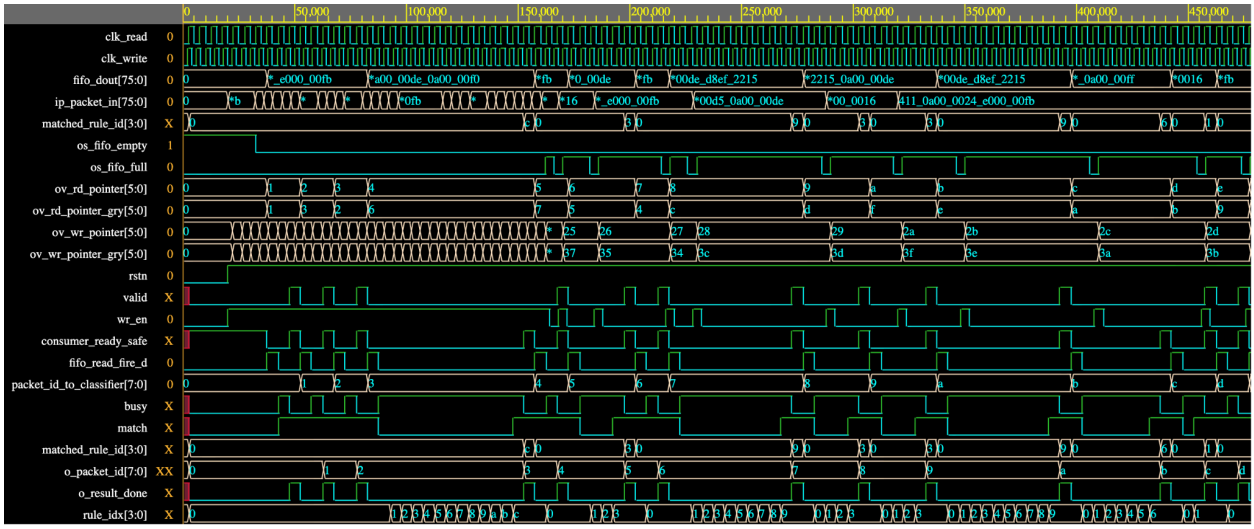


Figure 39. Functional Simulation Waveform of the Area-Optimized SmartNIC Design

This figure illustrates the timing behavior of the area-optimized SmartNIC, including FIFO-based clock domain crossing, sequential rule evaluation, and packet-to-result synchronization.

```
time=58000 | cycle=8 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a000024e00000fb | valid=0 | rule=0 | result_done=0 | result_packet_id=0 | wr_ptr=9 | rd_ptr=2
time=63000 | cycle=9 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a000024e00000fb | valid=0 | rule=0 | result_done=0 | result_packet_id=0 | wr_ptr=11 | rd_ptr=2
time=68000 | cycle=10 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a000024e00000fb | valid=1 | rule=0 | result_done=1 | result_packet_id=1 | wr_ptr=12 | rd_ptr=2
RESULT -> time=68000 | packet_id=1 | valid=1 | rule=0
time=73000 | cycle=11 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a000024e00000fb | valid=0 | rule=0 | result_done=0 | result_packet_id=1 | wr_ptr=13 | rd_ptr=3
time=78000 | cycle=12 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a000024e00000fb | valid=0 | rule=0 | result_done=0 | result_packet_id=1 | wr_ptr=14 | rd_ptr=3
RESULT -> time=83000 | packet_id=2 | valid=1 | rule=0
time=83000 | cycle=13 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a000024e00000fb | valid=1 | rule=0 | result_done=1 | result_packet_id=2 | wr_ptr=16 | rd_ptr=3
time=88000 | cycle=14 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=0 | rule=0 | result_done=0 | result_packet_id=2 | wr_ptr=17 | rd_ptr=4
time=93000 | cycle=15 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=0 | rule=0 | result_done=0 | result_packet_id=2 | wr_ptr=18 | rd_ptr=4
time=98000 | cycle=16 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=0 | rule=0 | result_done=0 | result_packet_id=2 | wr_ptr=19 | rd_ptr=4
time=103000 | cycle=17 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=0 | rule=0 | result_done=0 | result_packet_id=2 | wr_ptr=21 | rd_ptr=4
time=108000 | cycle=18 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=0 | rule=0 | result_done=0 | result_packet_id=2 | wr_ptr=22 | rd_ptr=4
time=113000 | cycle=19 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=0 | rule=0 | result_done=0 | result_packet_id=2 | wr_ptr=23 | rd_ptr=4
time=118000 | cycle=20 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=0 | rule=0 | result_done=0 | result_packet_id=2 | wr_ptr=24 | rd_ptr=4
time=123000 | cycle=21 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=0 | rule=0 | result_done=0 | result_packet_id=2 | wr_ptr=26 | rd_ptr=4
time=128000 | cycle=22 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=0 | rule=0 | result_done=0 | result_packet_id=2 | wr_ptr=27 | rd_ptr=4
time=133000 | cycle=23 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=0 | rule=0 | result_done=0 | result_packet_id=2 | wr_ptr=28 | rd_ptr=4
time=138000 | cycle=24 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=0 | rule=0 | result_done=0 | result_packet_id=2 | wr_ptr=29 | rd_ptr=4
time=143000 | cycle=25 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=0 | rule=0 | result_done=0 | result_packet_id=2 | wr_ptr=31 | rd_ptr=4
time=148000 | cycle=26 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=0 | rule=0 | result_done=0 | result_packet_id=2 | wr_ptr=32 | rd_ptr=4
time=153000 | cycle=27 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=0 | rule=0 | result_done=0 | result_packet_id=2 | wr_ptr=33 | rd_ptr=4
time=158000 | cycle=28 | fifo_empty=0 | fifo_full=0 | fifo_dout=4060a0000de0a0000f0 | valid=0 | rule=12 | result_done=1 | result_packet_id=3 | wr_ptr=34 | rd_ptr=4
RESULT -> time=158000 | packet_id=3 | valid=0 | rule=12
time=163000 | cycle=29 | fifo_empty=0 | fifo_full=1 | fifo_dout=4110a000024e00000fb | valid=0 | rule=0 | result_done=0 | result_packet_id=3 | wr_ptr=36 | rd_ptr=5
time=168000 | cycle=30 | fifo_empty=0 | fifo_full=0 | fifo_dout=4110a000024e00000fb | valid=0 | rule=0 | result_done=0 | result_packet_id=3 | wr_ptr=36 | rd_ptr=5
RESULT -> time=173000 | packet_id=4 | valid=1 | rule=0
time=173000 | cycle=31 | fifo_empty=0 | fifo_full=1 | fifo_dout=4110a000024e00000fb | valid=1 | rule=0 | result_done=1 | result_packet_id=4 | wr_ptr=37 | rd_ptr=5
```

Figure 40. Simulation Console Output Validating the Area-Optimized SmartNIC Design

This figure presents the textual simulation output used to verify packet classification accuracy, packet ID tracking, FIFO behavior, and sequential rule processing.

Waveform and Output Analysis (Area Optimized Design)

Figure 39 presents the functional simulation waveform of the area-optimized SmartNIC. The clk_write and clk_read signals operate asynchronously, confirming reliable clock domain crossing through the asynchronous FIFO. Packet data is written via ip_packet_in and becomes available at fifo_dout when os_fifo_empty is deasserted. The FIFO pointers (ov_wr_pointer, ov_rd_pointer) and their Gray-coded equivalents increment correctly, validating safe data transfer between clock domains.

The consumer_ready_safe signal ensures that packets are read only when the classifier is ready, preventing data loss. Once a packet is accepted, packet_id_to_classifier increments sequentially to uniquely tag each packet. The classifier asserts the busy signal while evaluating rules sequentially, with rule_idx incrementing each clock cycle to demonstrate the shared-comparator architecture. When a match is detected (match = 1), the outputs valid and matched_rule_id are asserted, and o_result_done generates a one-cycle pulse indicating completion. The corresponding o_packet_id confirms accurate packet-to-result association. This waveform verifies correct sequential rule evaluation, proper FIFO control, and reliable CDC operation in the area-optimized design.

Figure 40 shows the simulation console output used to validate the functionality of the area-optimized SmartNIC. Each log entry displays FIFO status (fifo_empty, fifo_full), packet data (fifo_dout), classification results (valid, rule), and pointer values (wr_ptr, rd_ptr).

Additional verification signals, including result_done and result_packet_id—confirm that each classification result is correctly aligned with its corresponding input packet.

The packet IDs increase sequentially, demonstrating accurate tracking and synchronization across the FIFO and classifier. The result_done signal indicates the completion of sequential rule evaluation, verifying the correct operation of the multi-cycle architecture. Occasional non-consecutive pointer increments occur because read and write operations are gated by handshake signals; when no transaction occurs during a clock cycle, the pointers hold their values. This behavior is expected and confirms proper FIFO control.

Together, the textual output provides clear and precise validation of packet parsing, rule matching, CDC integrity, and sequential processing. These results demonstrate that the area-optimized SmartNIC achieves correct functionality while maintaining data integrity and synchronization.

7. Comparative Analysis

This section compares the high-throughput and area-optimized SmartNIC architectures based on performance, timing, resource utilization, and power consumption. All results are derived from post-implementation analysis on the ZYBO Z7 FPGA.

7.1 Comparison Tables

Table 8. Architectural Comparison of Throughput and Area-Optimized Designs

Feature	**High-Throughput**	**Area-Optimized Design**
Classification Method	Parallel rule evaluation	Sequential rule evaluation
Processing Style	Fully pipelined	Multi-cycle, state-driven
Comparators	One per rule (replicated)	Single shared comparator
Throughput	One packet per clock cycle	One packet per multiple cycles
Latency	Fixed (~3 clock cycles)	Variable (up to number of rules)
Resource Sharing	Minimal	Extensive
Scalability	Optimized for high data rates	Optimized for reduced hardware usage

Design Objective	Maximum performance	Minimum area and power
FIFO integration	Supports continuous streaming	Ensures controlled packet processing

Table 9. Post-Implementation Timing Performance

Timing Metric	**High-Throughput**	**Area-Optimized Design**
Write Clock Frequency	200 MHz (5 ns)	200 MHz (5 ns)
Read Clock Frequency	166.67 MHz (6 ns)	166.67 MHz (6 ns)
Worst Negative Slack (WNS)	0.386 ns	0.419 ns
Worst Hold Slack (WHS)	0.124 ns	0.190 ns
Total Negative Slack (TNS)	0.000 ns	0.000 ns
Timing Status	Met	Met
Critical Path	Parallel comparator logic	Sequential comparator and control logic

Observation: Both architectures meet timing requirements. The area-optimized design achieves slightly better slack due to reduced combinational complexity.

Table 10. Post-Implementation Resource Utilization

Resource	**High-Throughput Design**	**Area-Optimized Design**	**Improvement**
LUTs	501	463	Reduce hardware usage
Flip-Flops (FFs)	1432	1421	Slight reduction
Bonded I/O	49	49	No change
LUT Utilization	2.85%	2.63%	Lower in area

(%)			design
FF Utilization (%)	4.07%	4.04%	Slightly lower
Primary Contributor	Parallel classifier logic	Shared comparator logic	

Observation: The area-optimized design achieves lower LUT utilization due to resource sharing and sequential rule evaluation, confirming its hardware efficiency.

Table 11. Post-Implementation Post-Implementation Power Consumption

Power Metric	**High Throughput Design**	**Area-Optimized Design**
Total On-Chip Power	**0.121 W**	**0.106 W**
Dynamic Power	**0.029 W (24%)**	**0.015 W (14%)**
Static Power	**0.092 W (76%)**	**0.092 W (76%)**
Clock Power	**0.007 W**	**0.005 W**
Signal Power	**0.012W**	**0.002 W**
Logic Power	**0.009 W**	**0.002 W**
I/O Power	**0.001 W**	**0.002 W**
Power Efficiency	**Lower**	**Higher**

Observation: The area-optimized design consumes less dynamic power due to reduced switching activity and hardware reuse, making it more energy-efficient.

It is important to note that a portion of the FPGA resource utilization is attributed to packet storage and buffering used during simulation and synthesis. In this implementation, packet data were preloaded into a ROM using a memory initialization file to enable deterministic functional verification. This ROM-based packet generation contributes to LUT or memory usage but does not represent a requirement in real-world SmartNIC deployments. In practical

networking systems, packets are streamed directly from external interfaces such as Ethernet MACs or DMA engines. Consequently, the measured resource utilization slightly overestimates the hardware footprint of the classifier itself, with the primary architectural differences arising from the parallel and sequential rule evaluation strategies.

7.2 Comparative Analysis and Discussion

Comparative Discussion

The high-throughput and area-optimized SmartNIC architectures demonstrate distinct trade-offs in performance, resource utilization, and energy efficiency. The high-throughput design employs parallel rule evaluation and pipelining, enabling continuous packet processing at a rate of one packet per clock cycle after pipeline filling. In contrast, the area-optimized design utilizes sequential rule scanning and resource sharing, significantly reducing hardware utilization at the expense of increased latency.

Timing and Critical Path Analysis

Both architectures were evaluated using identical timing constraints: a **5 ns period (200 MHz) for clk_write** and a **6 ns period (166.67 MHz) for clk_read**. Asynchronous clock groups and false-path constraints were applied to ensure accurate clock domain crossing (CDC) analysis. Post-implementation results confirm that both designs meet timing requirements with positive slack.

The critical path of the **high-throughput design** lies within the parallel comparator logic and associated routing, where multiple rules are evaluated simultaneously. This increases combinational delay but enables maximum throughput. Conversely, the **area-optimized design** has a shorter combinational path per cycle due to the reuse of a single comparator and simplified control logic. Although it requires multiple cycles per packet, the reduced logic depth results in slightly improved slack values. These findings align with FPGA design principles discussed in Lecture 6, where parallelism improves speed while resource sharing reduces hardware complexity.

Resource Utilization Analysis

Post-implementation results indicate that the high-throughput design consumes **501 LUTs and 1,432 flip-flops**, while the area-optimized design uses **463 LUTs and 1,421 flip-flops**. The reduction in LUT usage confirms the effectiveness of sequential processing and resource sharing in minimizing hardware overhead.

Power Consumption Analysis

Power analysis further highlights the trade-offs between the two architectures. The high-throughput design consumes **0.121 W**, whereas the area-optimized design consumes **0.106 W**. The reduction in dynamic power is attributed to decreased switching activity

resulting from sequential rule evaluation. Static power remains dominant in both designs due to FPGA leakage characteristics.

Overall Trade-Off

- **High-Throughput Design:** Maximum speed, fixed low latency, higher area, and increased power consumption.
- **Area-Optimized Design:** Reduced hardware utilization and power consumption, with increased and variable latency.

These results demonstrate that the throughput design is best suited for high-speed networking applications, while the area-optimized design is ideal for resource-constrained and energy-efficient FPGA deployments.

8. FPGA Implementation

8.1 LED-Based Hardware Implementation

To validate the real-time operation of the SmartNIC Packet Classification Accelerator, the high-throughput design was implemented on the ZYBO Z7 FPGA. The classification results were displayed using onboard LEDs, providing a simple and effective method for visual verification of correct hardware functionality.

8.1.1 Architecture

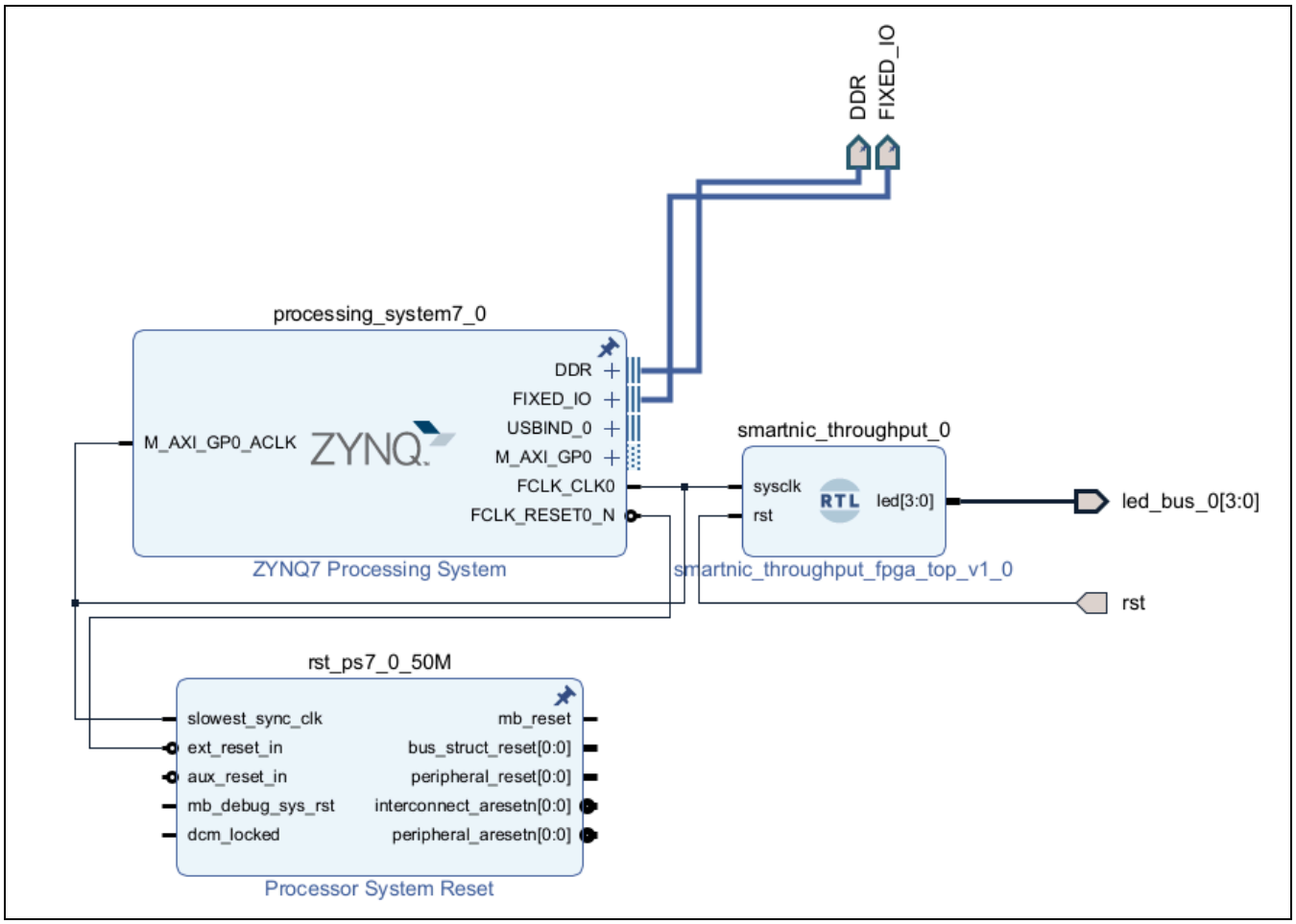


Figure 41: Block Diagram of the LED-Based FPGA Implementation

Figure 41 illustrates the block diagram of the LED-based FPGA implementation. The Zynq Processing System (PS) generates the system clock and reset signals, which drive the programmable logic (PL). The SmartNIC core performs packet classification, and the resulting matched rule identifiers are displayed on the onboard LEDs.

The architecture consists of the following components:

- **Zynq Processing System (PS):** Supplies the system clock (sysclk) and reset signals.
- **SmartNIC Throughput Core:** Performs high-speed packet parsing and rule classification.
- **Packet ROM:** Provides deterministic packet inputs initialized from pck_stream.mem.
- **Asynchronous FIFO:** Ensures safe clock domain crossing between write and read domains.
- **LED Display Interface:** Outputs classification results for real-time observation.

8.1.2 Implementation Details

The LED FPGA implementation utilizes a top-level wrapper to interface the SmartNIC core with the Zynq system and onboard LEDs. A clock divider is incorporated to slow down the output, ensuring that classification results are visible to the human eye.

Top-Level Module

File: throughput_fpga_top_level.v
Module Name: smartnic_throughput_fpga_top

This module instantiates the SmartNIC processing pipeline and periodically samples the output for display on the LEDs. The SmartNIC operates at full speed, while the LED outputs are updated at a slower rate using a counter-based sampling mechanism.

Packet ROM

File: packet_rom.v

The packet ROM supplies predefined packet data to the SmartNIC using a memory initialization file (pck_stream.mem). This approach ensures deterministic and repeatable hardware verification.

FPGA Constraints

File: throughput_fpga_constraints.xdc

The constraint file maps LED outputs and reset signals to the ZYBO Z7 FPGA pins, enabling proper hardware interfacing.

Note: All source files related to the FPGA implementation are located in the directory: **LED_FPGA_Implementation (High Throughput Design)**.

8.1.2.1 RTL Elaboration Diagram

The RTL elaboration diagrams confirm the correct structural integration of the SmartNIC core, packet ROM, and LED interface. These schematics illustrate the interconnection between the processing system, programmable logic, and output peripherals, validating the synthesized design hierarchy.

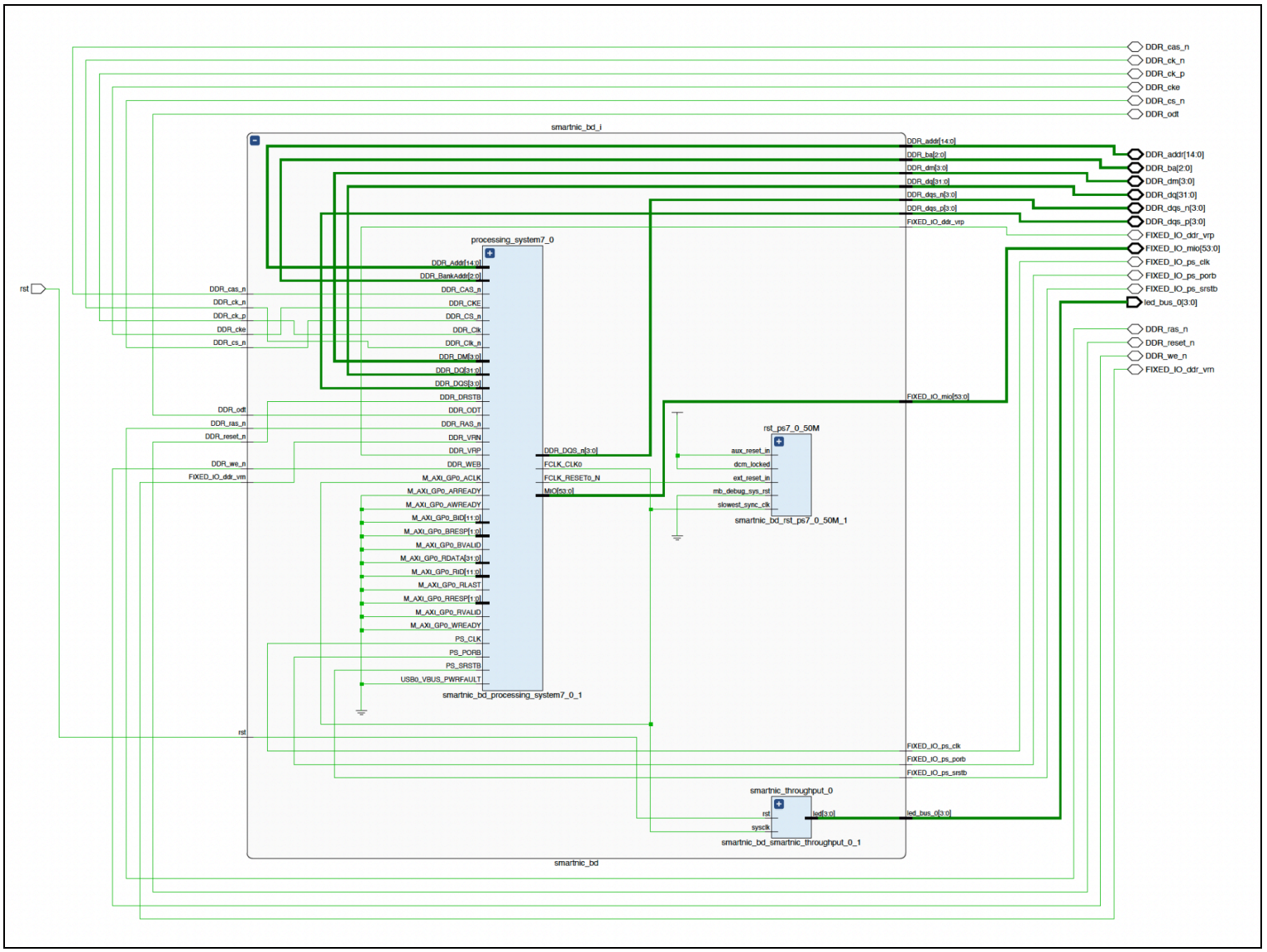


Figure 42: RTL Elaboration of the SmartNIC FPGA Design

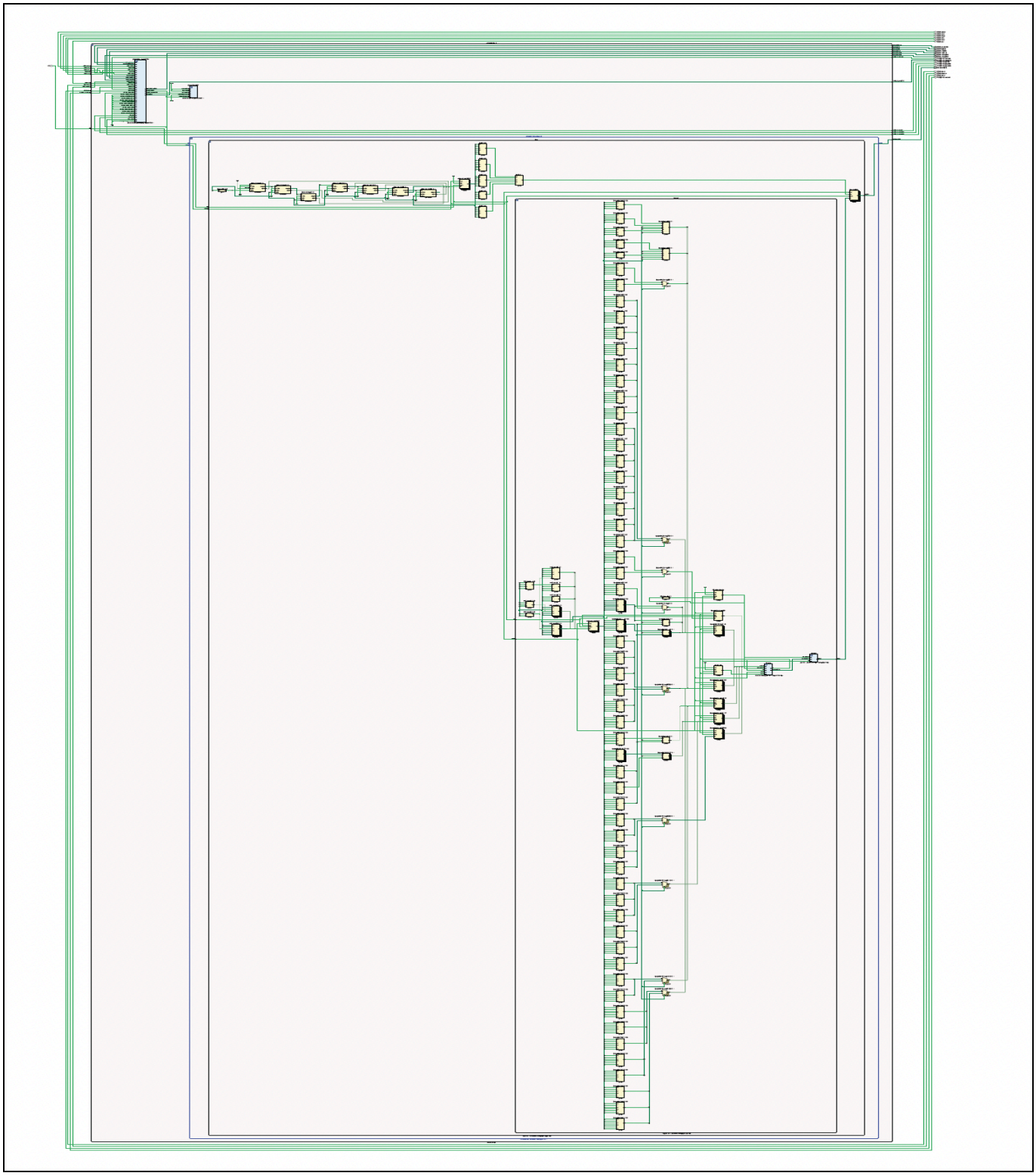


Figure 43: Expanded RTL Schematic of the FPGA Implementation

8.1.3 Results

8.1.3.1 Post-Implementation Schematic

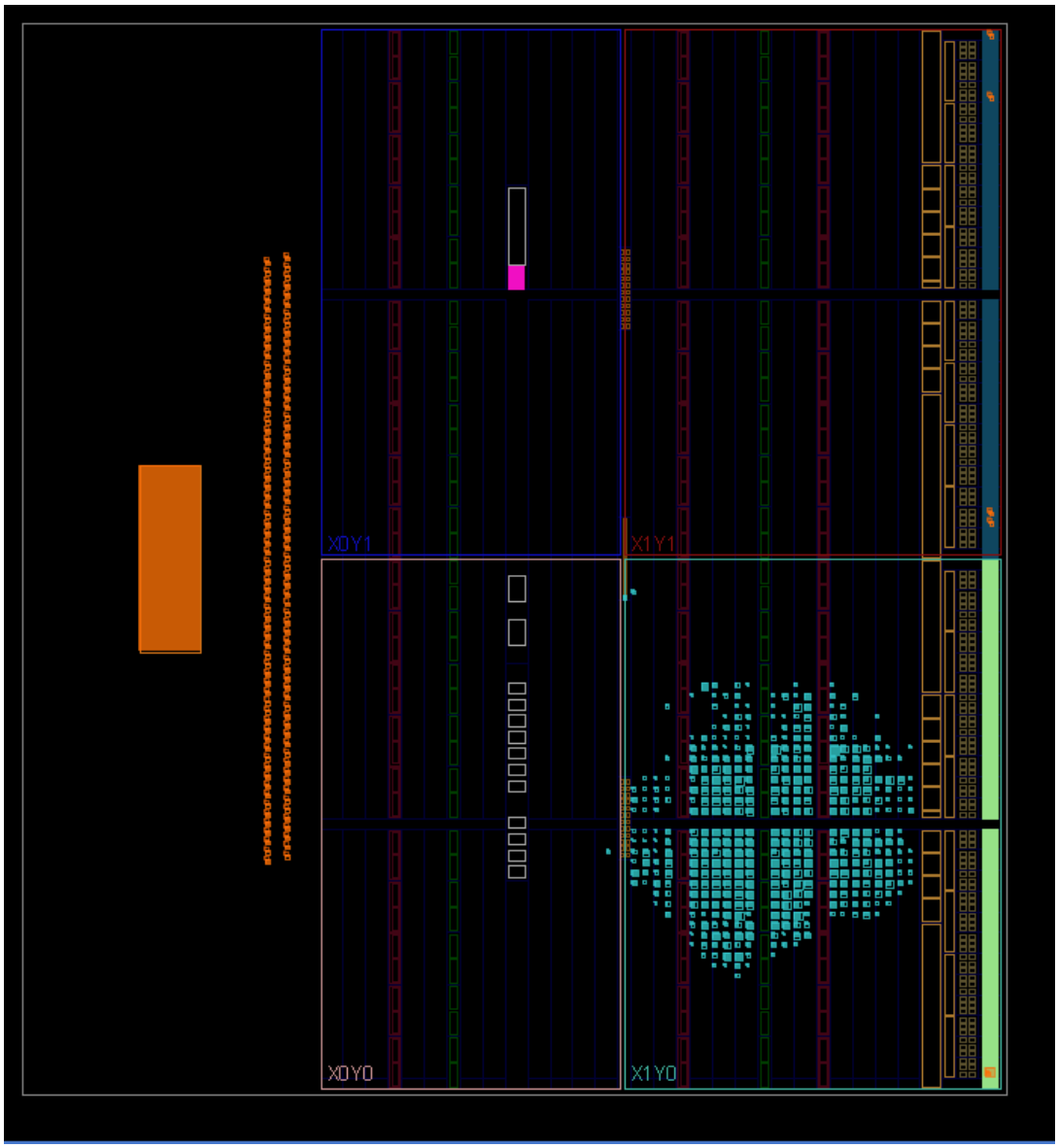


Figure 44: Post-Implementation Layout of the SmartNIC on the ZYBO Z7 FPGA

The post-implementation schematic verifies successful placement and routing of the SmartNIC design on the FPGA. The layout confirms proper integration of the packet classification pipeline, clocking infrastructure, and LED output logic within the programmable logic region.

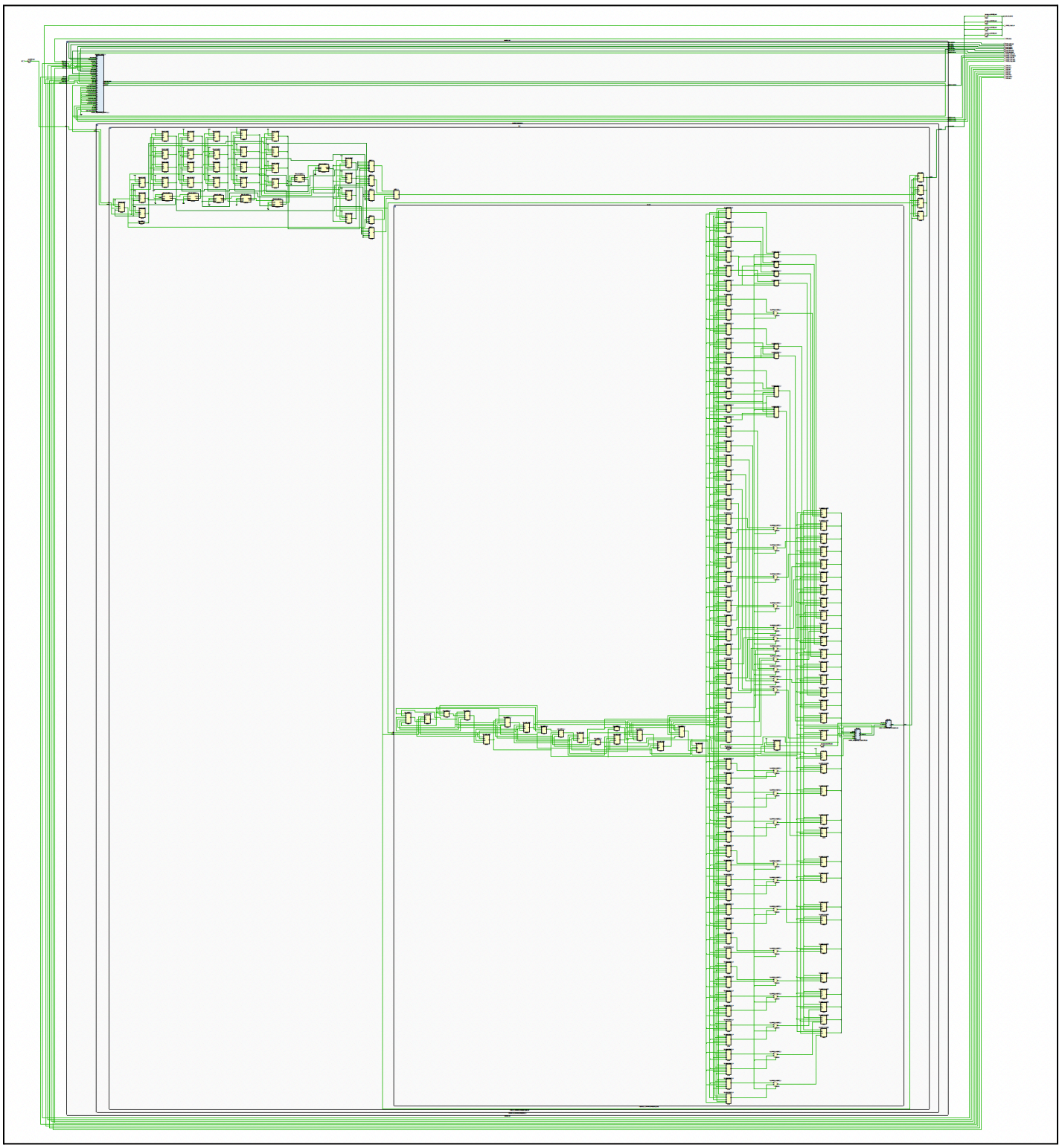
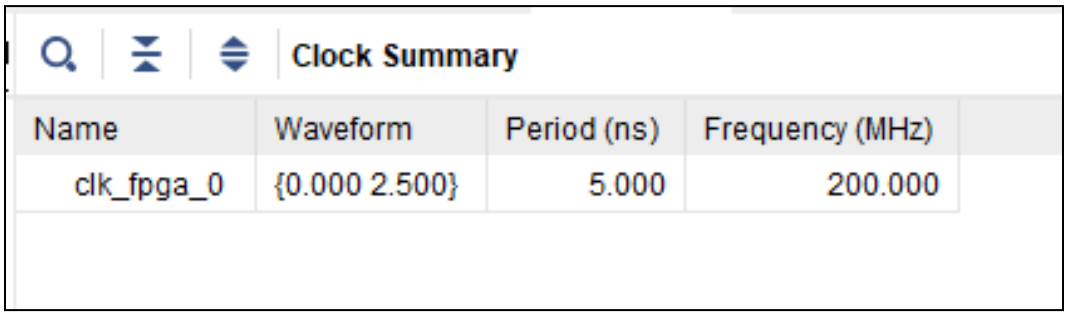


Figure 45: Post-Implementation Schematic Diagram LED FPGA Design

8.1.3.2 Timing Analysis



Clock Summary

Name	Waveform	Period (ns)	Frequency (MHz)
clk_fpga_0	{0.000 2.500}	5.000	200.000

Figure 46: ZYNQ PS Clock Constraint

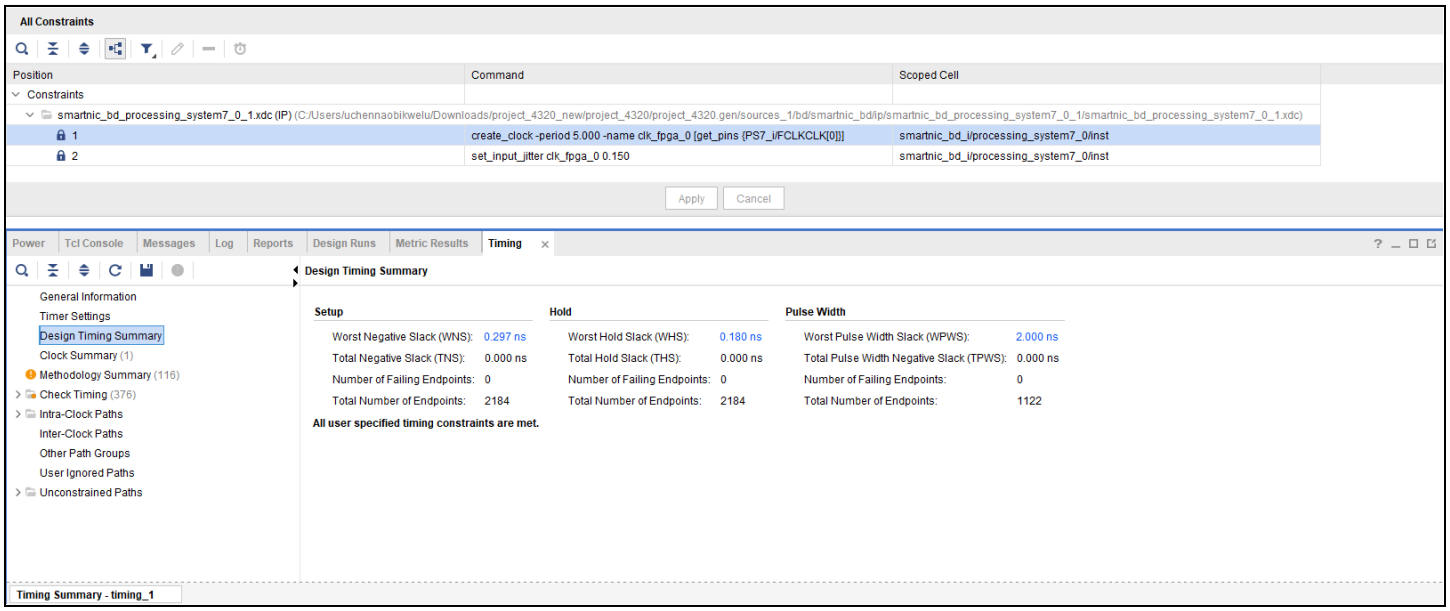


Figure 47: FPGA Timing Report

The FPGA timing report confirms that the hardware implementation remains consistent with the timing results obtained during the earlier synthesis and implementation stages of the SmartNIC design. The same 5 ns clock period (200 MHz) was used for the FPGA clock generated from the Zynq PS, and the design continued to meet timing with positive slack. In hardware, the implemented system achieved a Worst Negative Slack (WNS) of 0.297 ns and a Worst Hold Slack (WHS) of 0.180 ns, which are slightly lower than the earlier implementation-level values due to additional integration overhead from the Zynq processing system, LED interface, and packet ROM. Nevertheless, the positive setup and hold slack

confirm that the timing optimization carried out during the simulation and implementation stages was valid and sufficient for successful FPGA deployment.

8.1.3.3 Resource Utilization Analysis

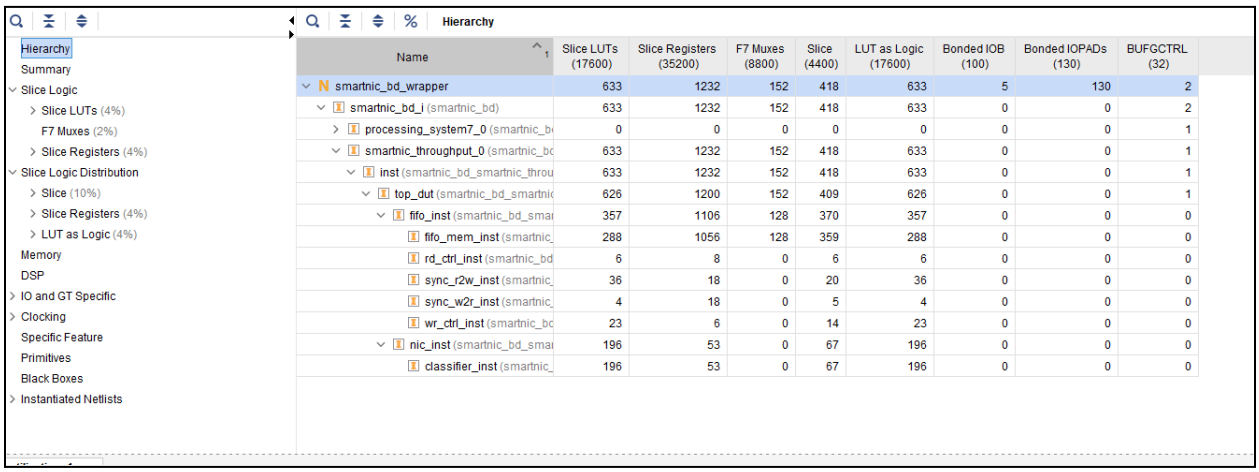


Figure 48: FPGA Resource Utilization Analysis Table

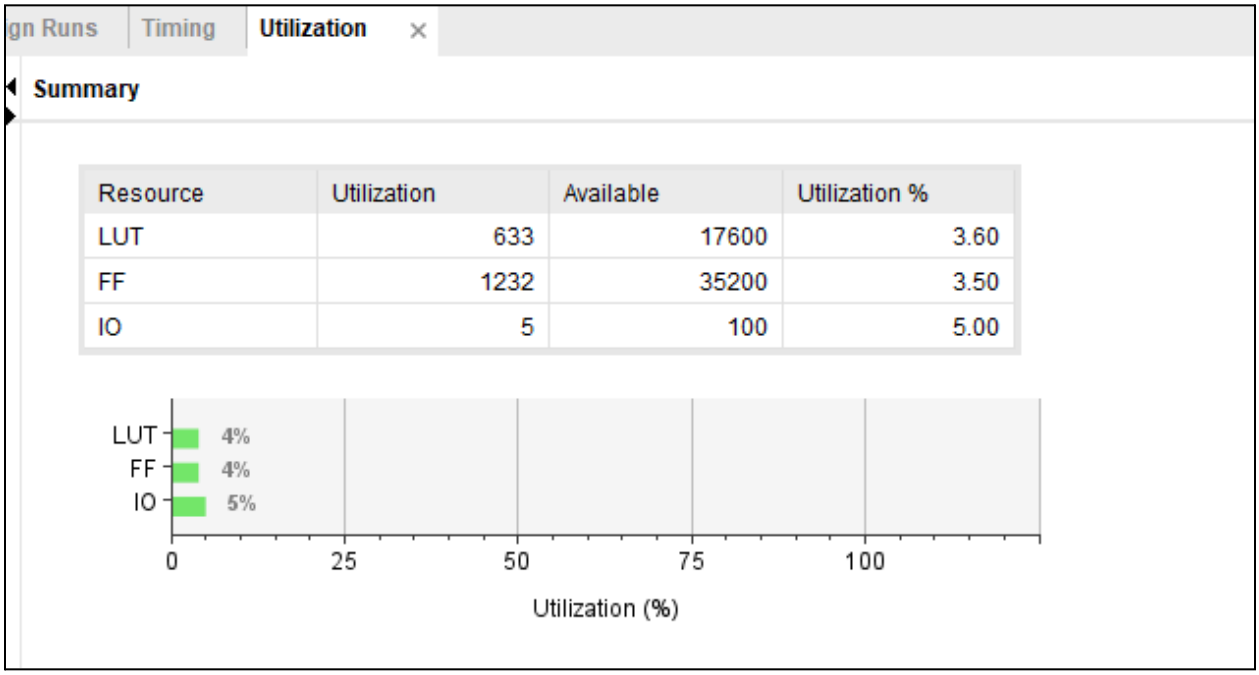


Figure 49: FPGA Resource Utilization Summary

The FPGA resource utilization summary shows that the final hardware implementation remains close to the utilization trends observed during the SmartNIC-only implementation stage, while reflecting the additional overhead of the full board-level integration. The FPGA

implementation used **633 LUTs**, **1232 flip-flops**, and **5 I/O pins**, corresponding to **3.60%**, **3.50%**, and **5.00%** utilization, respectively. Compared to the earlier SmartNIC implementation, the LUT count increased because the FPGA demonstration includes additional hardware components such as the packet ROM, Zynq integration logic, reset/control connections, and LED display interface. Most importantly, the final FPGA deployment confirms that the actual I/O usage is only **5%**, validating the earlier observation that the **49% I/O value** reported during pre-FPGA synthesis and implementation was only a conservative Vivado estimate before physical board pin mapping.

8.1.3.4 Power Analysis

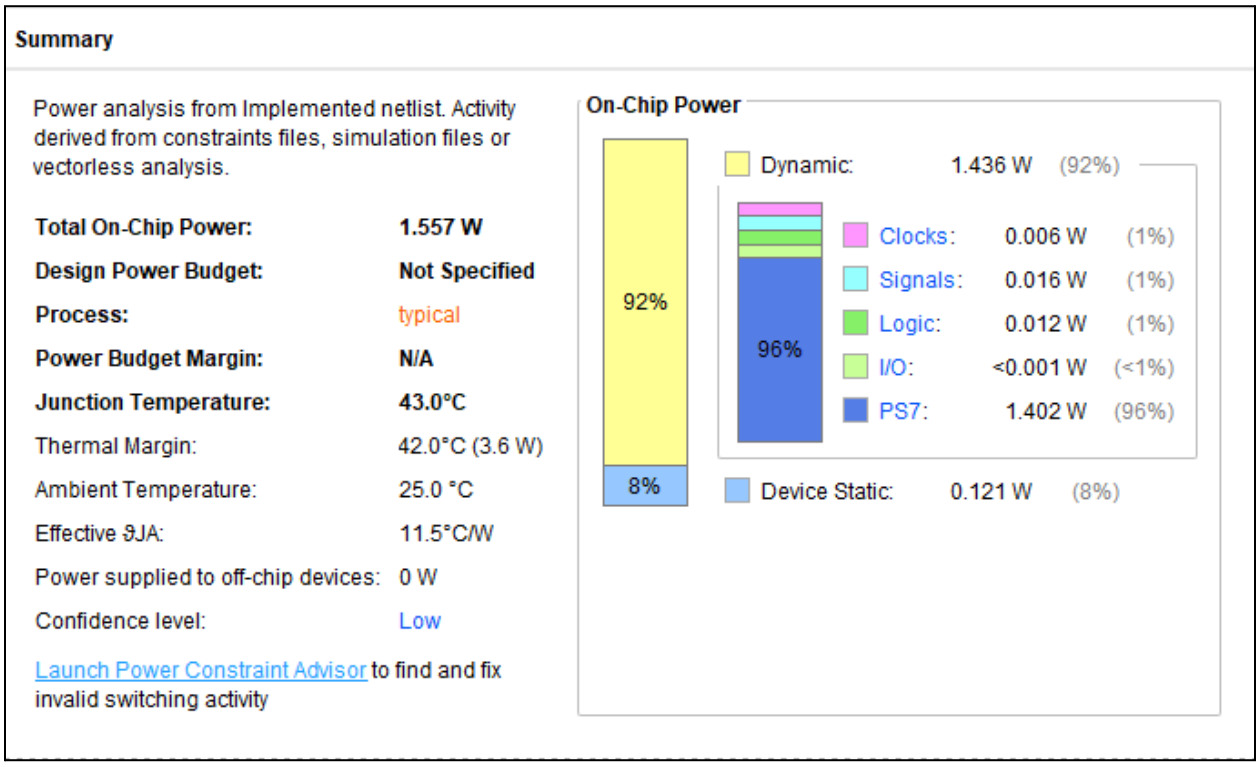


Figure 50. FPGA Power Analysis Report

The FPGA power analysis confirms the expected increase in total power consumption when the SmartNIC is deployed as a complete board-level system. The final implementation consumed **1.557 W** total on-chip power, with **1.436 W (92%)** attributed to dynamic power and **0.121 W (8%)** to static power. This is significantly higher than the earlier SmartNIC-only implementation power because the FPGA demonstration includes the active **Zynq PS7 subsystem**, which alone contributes the majority of the dynamic power. In contrast, the earlier implementation-stage SmartNIC power results were much lower because they

reflected only the programmable logic portion of the design. These results show that, while the classifier itself remains lightweight, full FPGA deployment introduces additional system-level power overhead from the processing system, clocks, and board-level infrastructure.

8.1.3.5 FPGA Execution LED Design Demonstration

A video demonstration: SMART NIC FPGA Video Demo.mp4 has been attached to the project submission demonstrating LED-Based FPGA implementation.

8.2 UART GPIO FPGA Implementation

8.2.1 Architecture

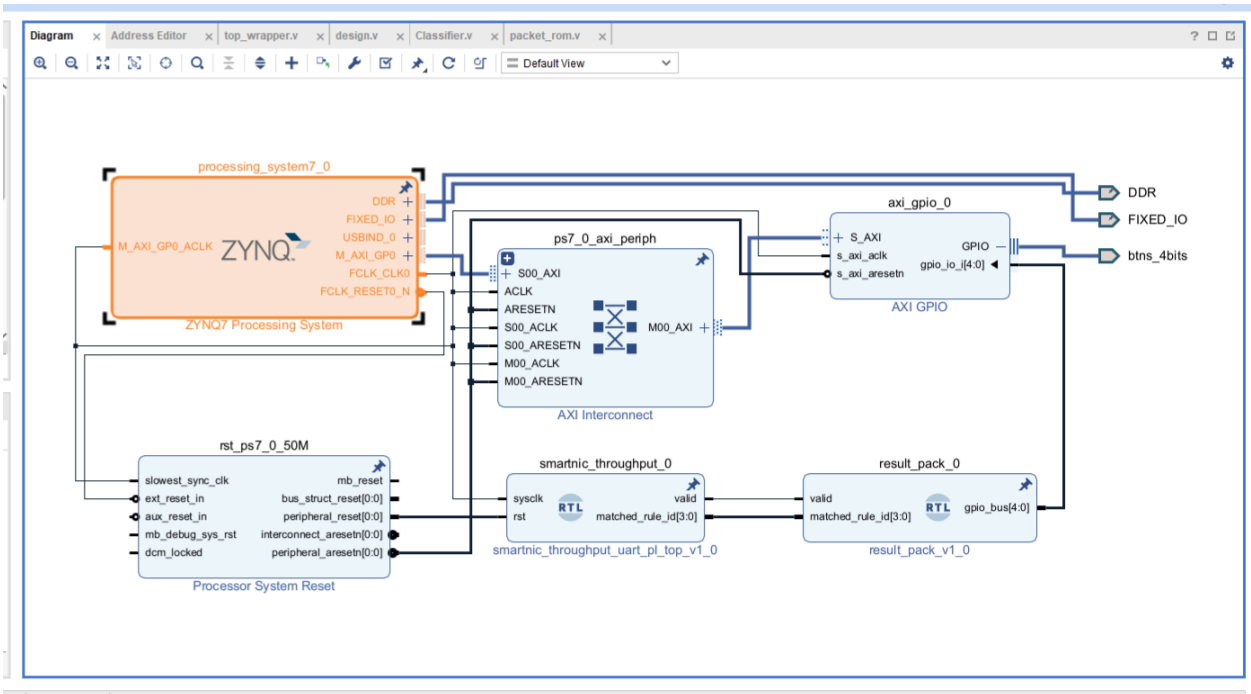


Figure 51. Block Diagram of the UART-Based FPGA Implementation

The UART-based FPGA implementation provides real-time monitoring of the SmartNIC packet classification results through serial communication. Unlike the LED demonstration, which offers limited visual feedback, this approach enables detailed observation and validation of classification outputs using a terminal interface. The architecture integrates the

Zynq Processing System (PS) with the Programmable Logic (PL), where the SmartNIC accelerator operates.

As illustrated in the block diagram, the Zynq PS generates the system clock and reset signals and communicates with the PL through the AXI interconnect. The SmartNIC processing pipeline executes in the PL, and its outputs, specifically the valid signal and the matched rule identifier are transmitted to an AXI GPIO peripheral. These results are then read by software executing on the PS and displayed via UART using the Vitis XSCT terminal. This hardware–software co-design enables efficient verification of the SmartNIC's functionality on the FPGA.

8.2.2 Implementation details

Note: All source files related to the UART FPGA implementation are located in the directory: **UART_FPGA_Implementation (High Throughput Design)**.

8.2.2.1 **RTL Elaboration Diagram**

The RTL elaboration confirms the correct structural integration of all modules involved in the UART-based FPGA implementation. The design consists of the packet generator, asynchronous FIFO for clock domain crossing (CDC), high-throughput classifier, and AXI GPIO interface. The elaborated schematic verifies signal connectivity, clock propagation, and module hierarchy, ensuring correctness prior to synthesis and implementation.

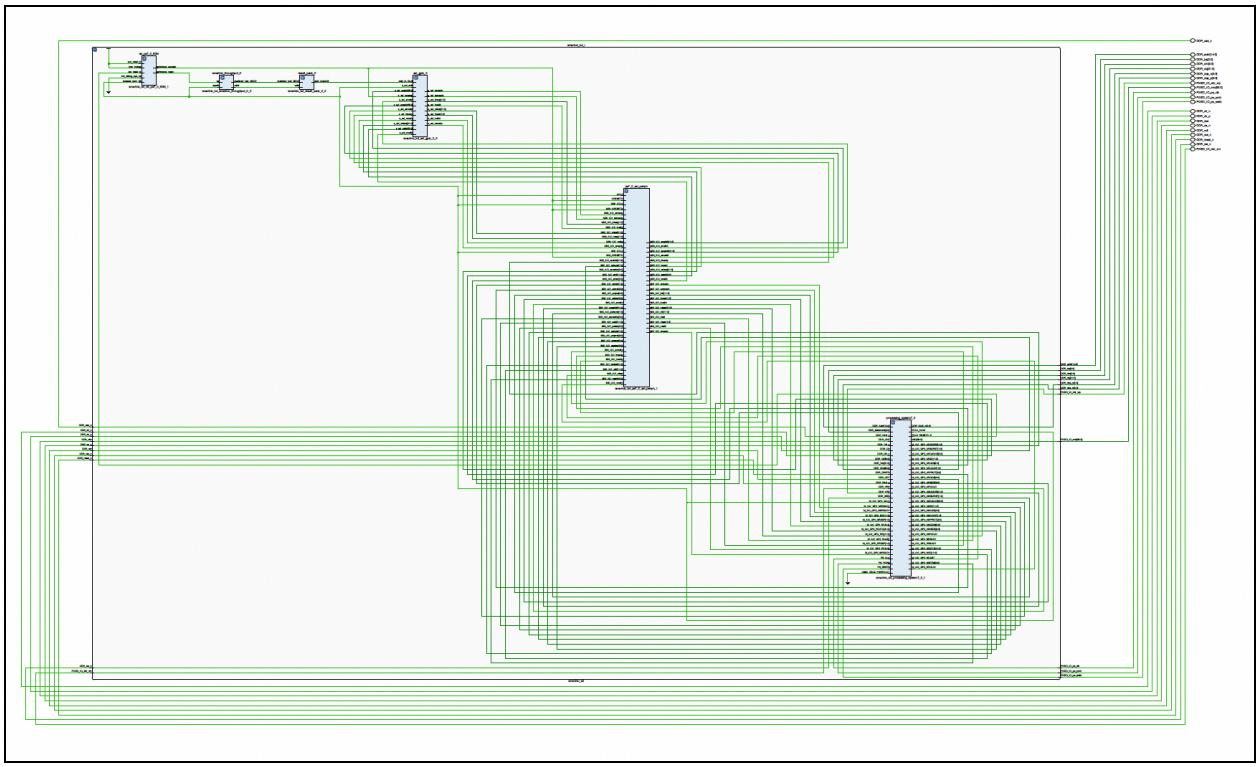


Figure 52. RTL Elaboration of the UART-Based SmartNIC Design

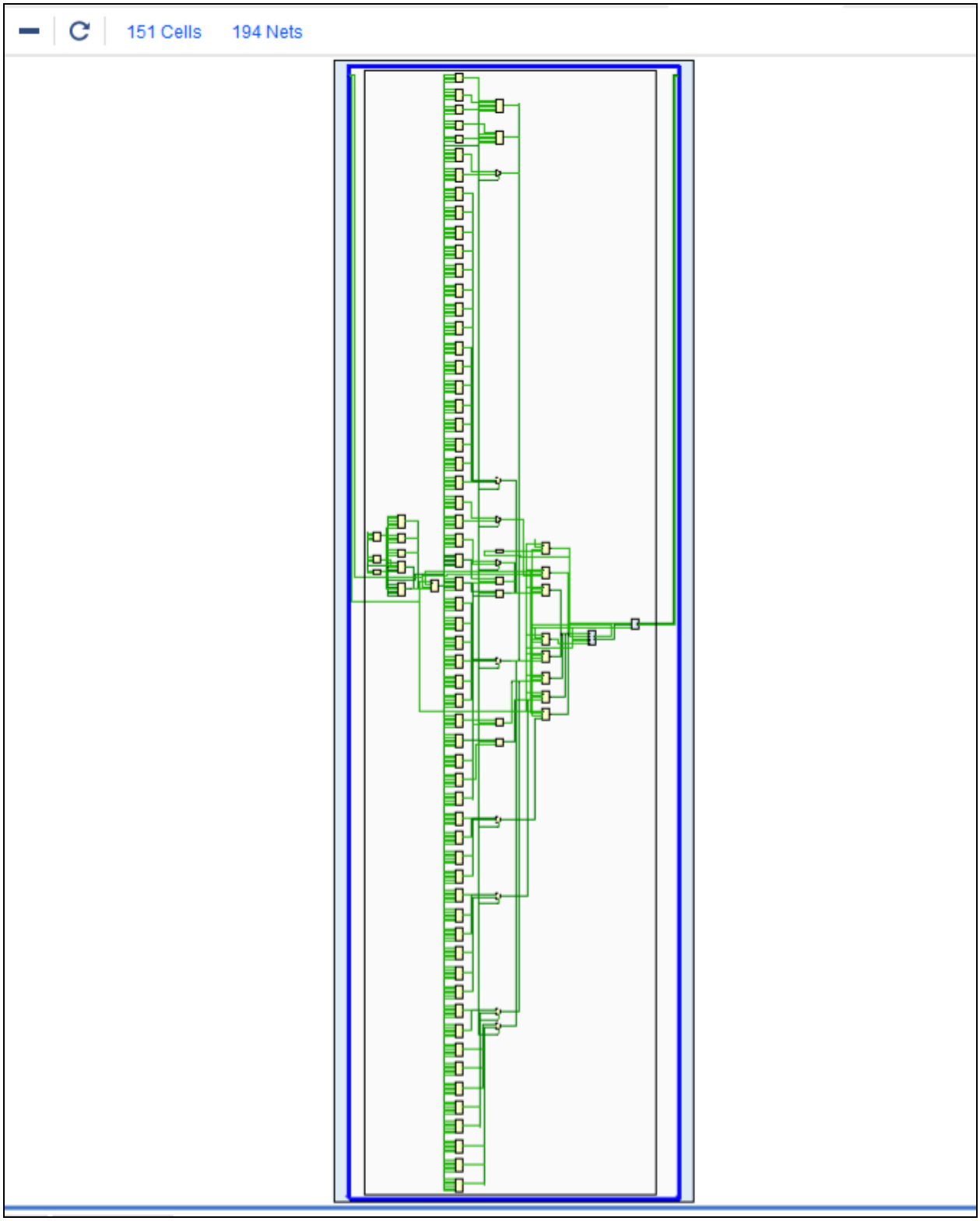


Figure 53. RTL Elaboration Schematic of the UART-Based SmartNIC Design Expanded

8.2.3 Results

8.2.3.1 Post-Implementation Schematic

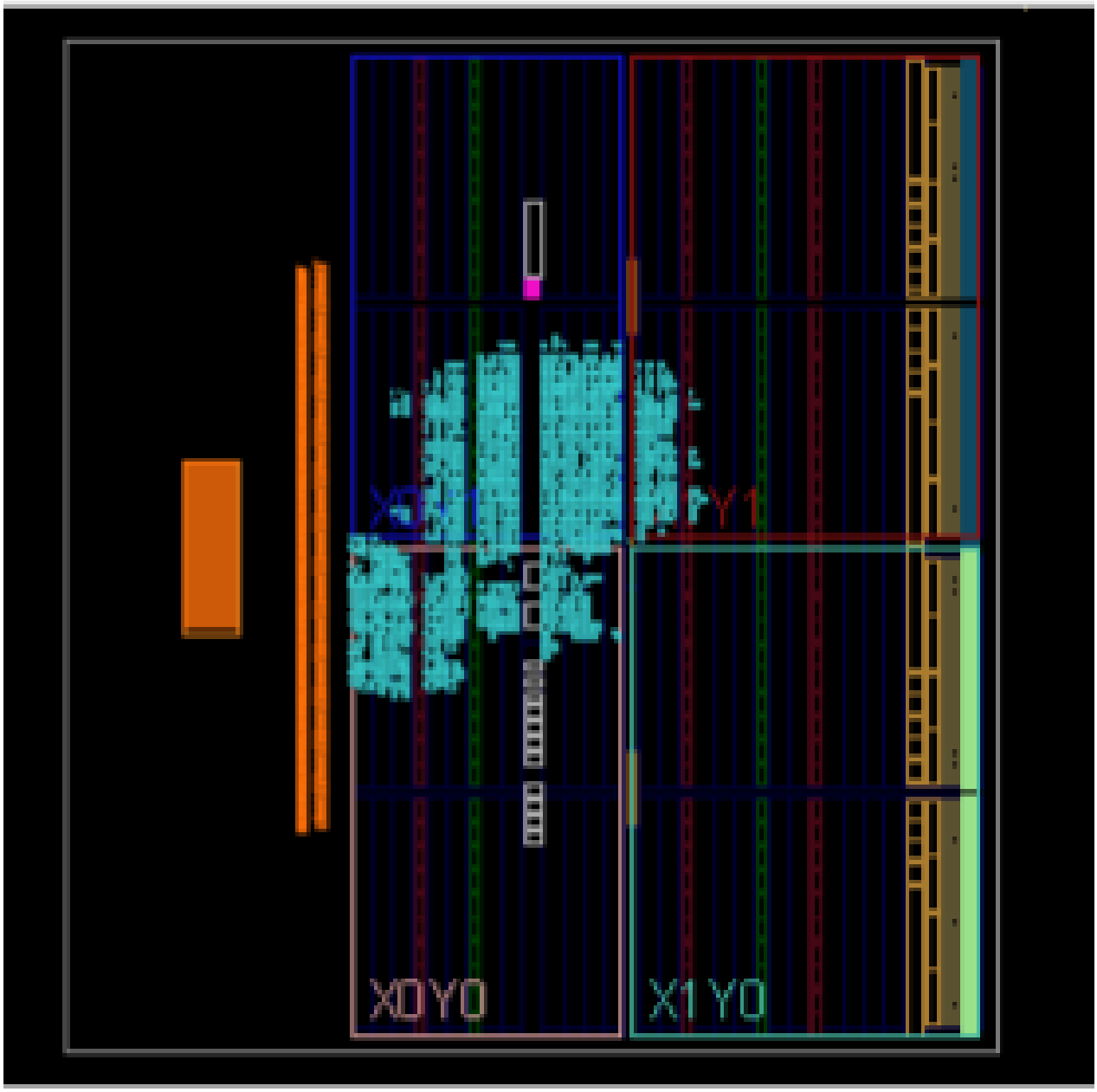


Figure 54. Post-Implementation Layout of the SmartNIC on the ZYBO Z7 FPGA for UART Design

The post-implementation schematic provides a physical representation of the synthesized and routed design on the FPGA. It illustrates the placement of logic resources, routing paths, and interconnections between the Zynq PS, AXI GPIO, and SmartNIC modules. The

clustering of programmable logic resources confirms efficient mapping of the accelerator onto the FPGA fabric.

8.2.3.2 Timing Analysis

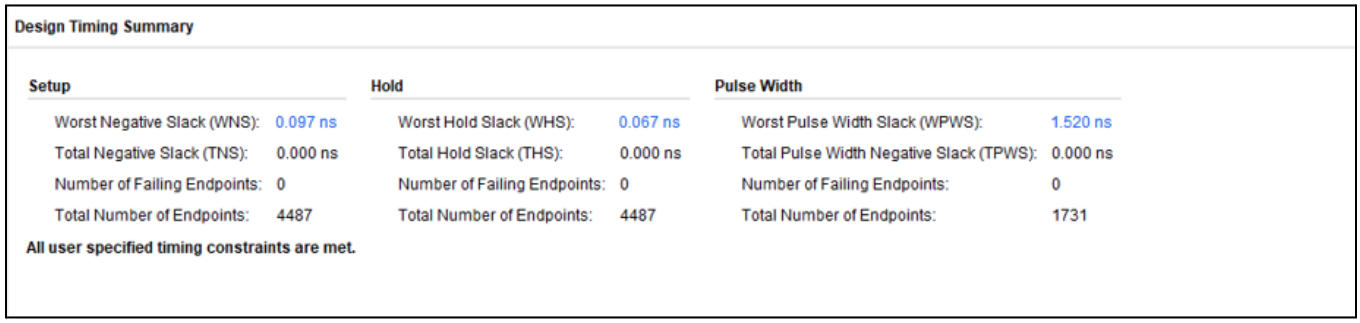
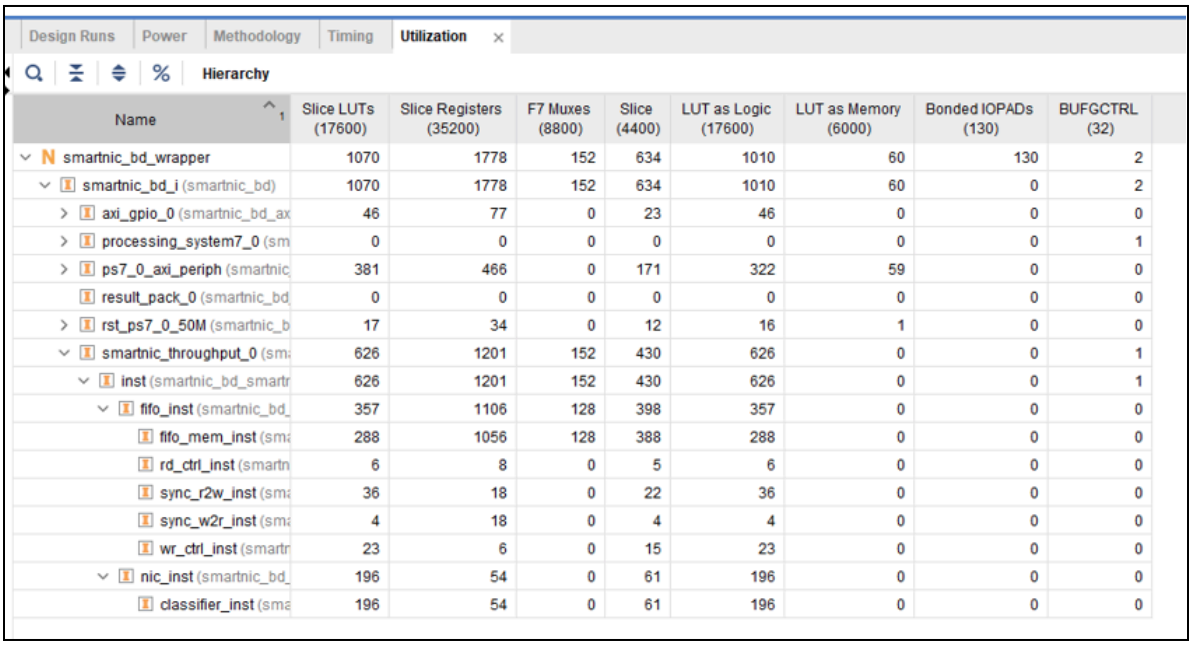


Figure 55. UART FPGA Timing Report

The timing report confirms that the UART-based SmartNIC implementation meets all design constraints. The programmable logic operates using a 200 MHz clock generated by the Zynq Processing System. The post-implementation timing analysis indicates positive setup and hold slack, validating reliable operation at the target frequency. These results align with those obtained during earlier synthesis and simulation stages, demonstrating consistency between theoretical analysis and hardware deployment.

8.2.3.3 Resource Utilization Analysis

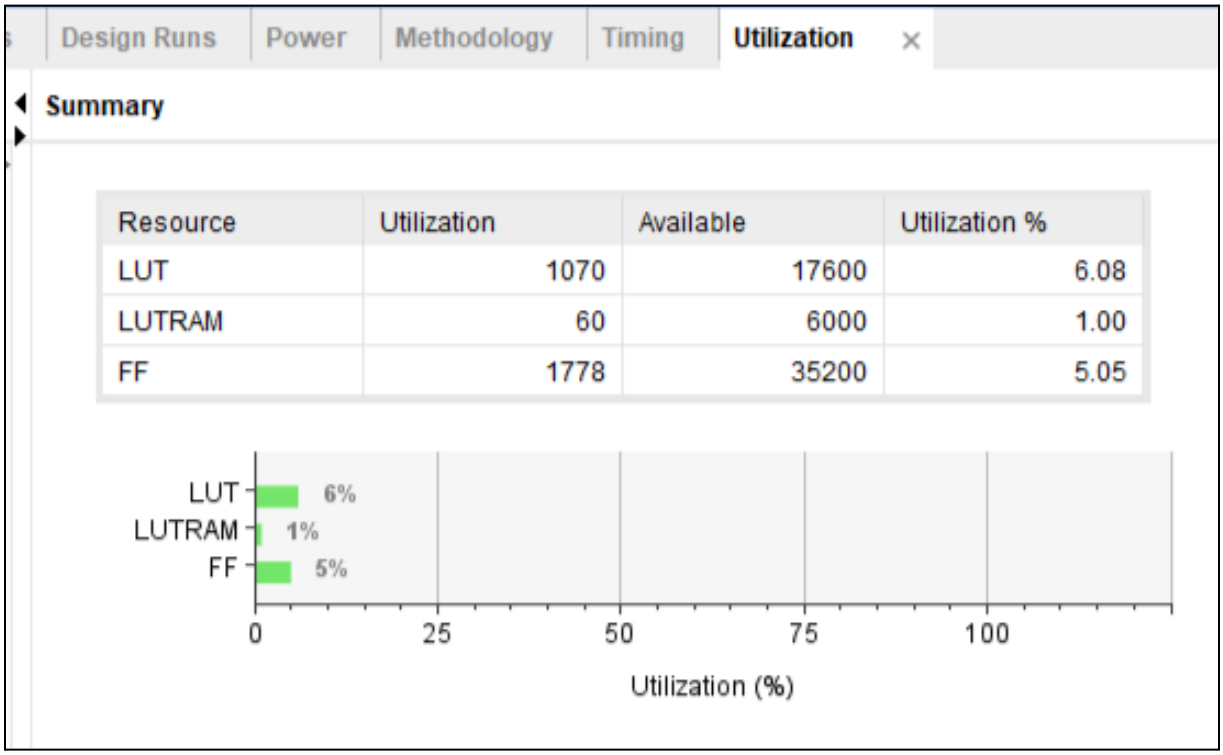


Design Runs | Power | Methodology | Timing | Utilization

Hierarchy

Name	Slice LUTs (17600)	Slice Registers (35200)	F7 Muxes (8800)	Slice (4400)	LUT as Logic (17600)	LUT as Memory (6000)	Bonded IOPADs (130)	BUFGCTRL (32)
smartnic_bd_wrapper	1070	1778	152	634	1010	60	130	2
smartnic_bd_i (smartnic_bd)	1070	1778	152	634	1010	60	0	2
axi_gpio_0 (smartnic_bd_ax	46	77	0	23	46	0	0	0
processing_system7_0 (sm	0	0	0	0	0	0	0	1
ps7_0_axi_periph (smartnic	381	466	0	171	322	59	0	0
result_pack_0 (smartnic_bd	0	0	0	0	0	0	0	0
rst_ps7_0_50M (smartnic_b	17	34	0	12	16	1	0	0
smartnic_throughput_0 (sm	626	1201	152	430	626	0	0	1
inst (smartnic_bd_smartr	626	1201	152	430	626	0	0	1
fifo_inst (smartnic_bd_	357	1106	128	398	357	0	0	0
fifo_mem_inst (sm	288	1056	128	388	288	0	0	0
rd_ctrl_inst (smartn	6	8	0	5	6	0	0	0
sync_r2w_inst (sm	36	18	0	22	36	0	0	0
sync_w2r_inst (sm	4	18	0	4	4	0	0	0
wr_ctrl_inst (smartr	23	6	0	15	23	0	0	0
nic_inst (smartnic_bd_	196	54	0	61	196	0	0	0
classifier_inst (sm	196	54	0	61	196	0	0	0

Figure 56. UART FPGA Timing Report



Design Runs | Power | Methodology | Timing | Utilization

Summary

Resource	Utilization	Available	Utilization %
LUT	1070	17600	6.08
LUTRAM	60	6000	1.00
FF	1778	35200	5.05

Figure 57. UART FPGA Resource Utilization Summary

The UART-based FPGA implementation used 1070 LUTs, 60 LUTRAMs, and 1778 flip-flops, corresponding to 6.08%, 1.00%, and 5.05% utilization, respectively. Compared to the LED-based FPGA implementation, the UART version consumes more logic resources because it includes additional system-level components such as the AXI GPIO interface, Zynq processing system integration, UART monitoring path, and associated control logic. The presence of LUTRAM indicates that part of the design uses LUT-based memory resources, which is consistent with lightweight storage and buffering structures generated during synthesis. Overall, the results show that the SmartNIC still occupies a modest portion of the FPGA, but UART-based monitoring introduces additional logic and memory overhead beyond the classifier itself.

8.2.3.4 Power Analysis

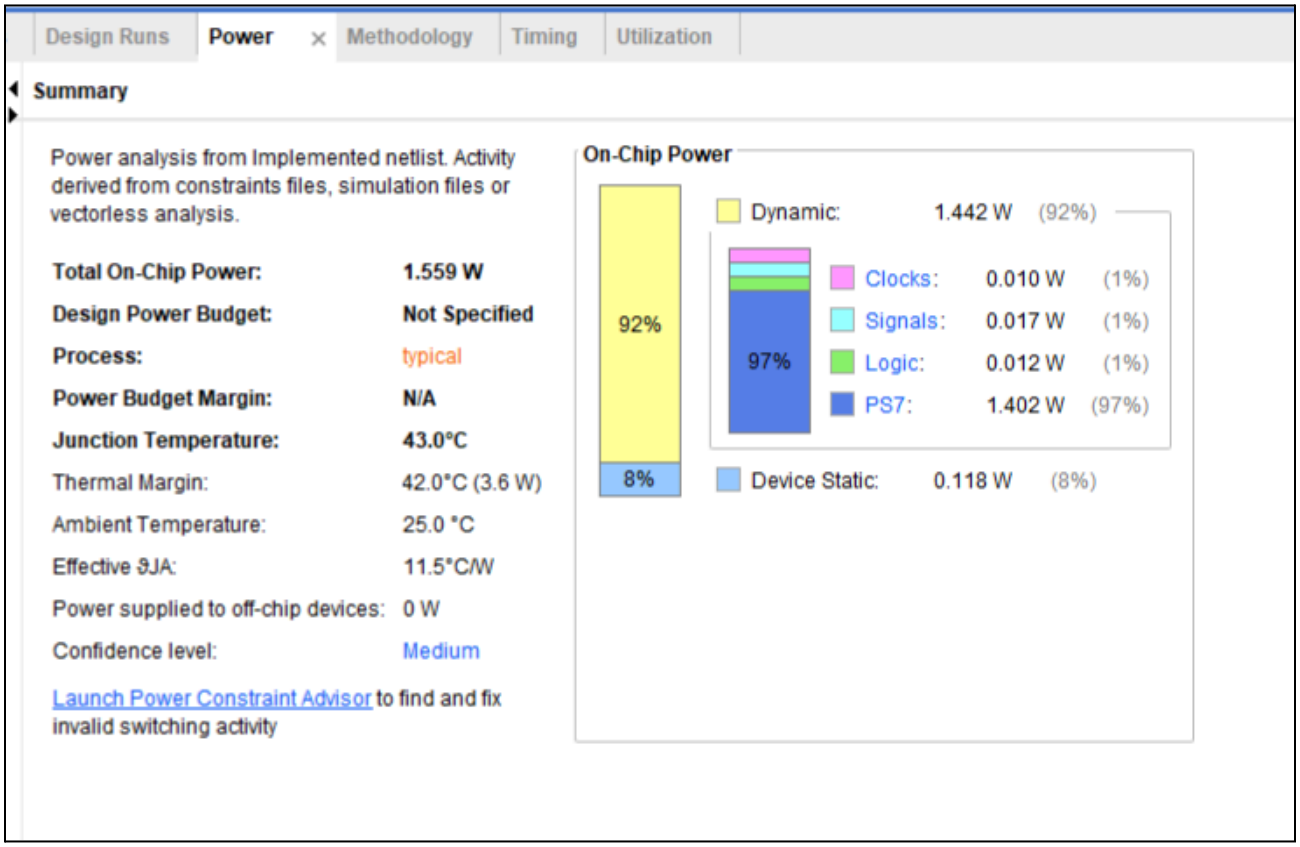


Figure 58. UART FPGA Power Analysis Report

The UART FPGA implementation consumed a total on-chip power of 1.559 W, with 1.442 W (92%) attributed to dynamic power and 0.118 W (8%) to static power. As in the LED-based FPGA demonstration, most of the power is not consumed by the classifier itself, but by the full system-level deployment, particularly the PS7 subsystem, which contributes 1.402 W (97%) of the dynamic component. Smaller contributions come from clocks (0.010 W), signals

(0.017 W), and logic (0.012 W). These results confirm that the UART-enabled FPGA implementation remains functionally practical, but system integration with the Zynq processing system and monitoring infrastructure significantly increases total power compared to the SmartNIC-only programmable logic implementation.

8.2.3.5 FPGA Execution UART-GPIO Design Demonstration

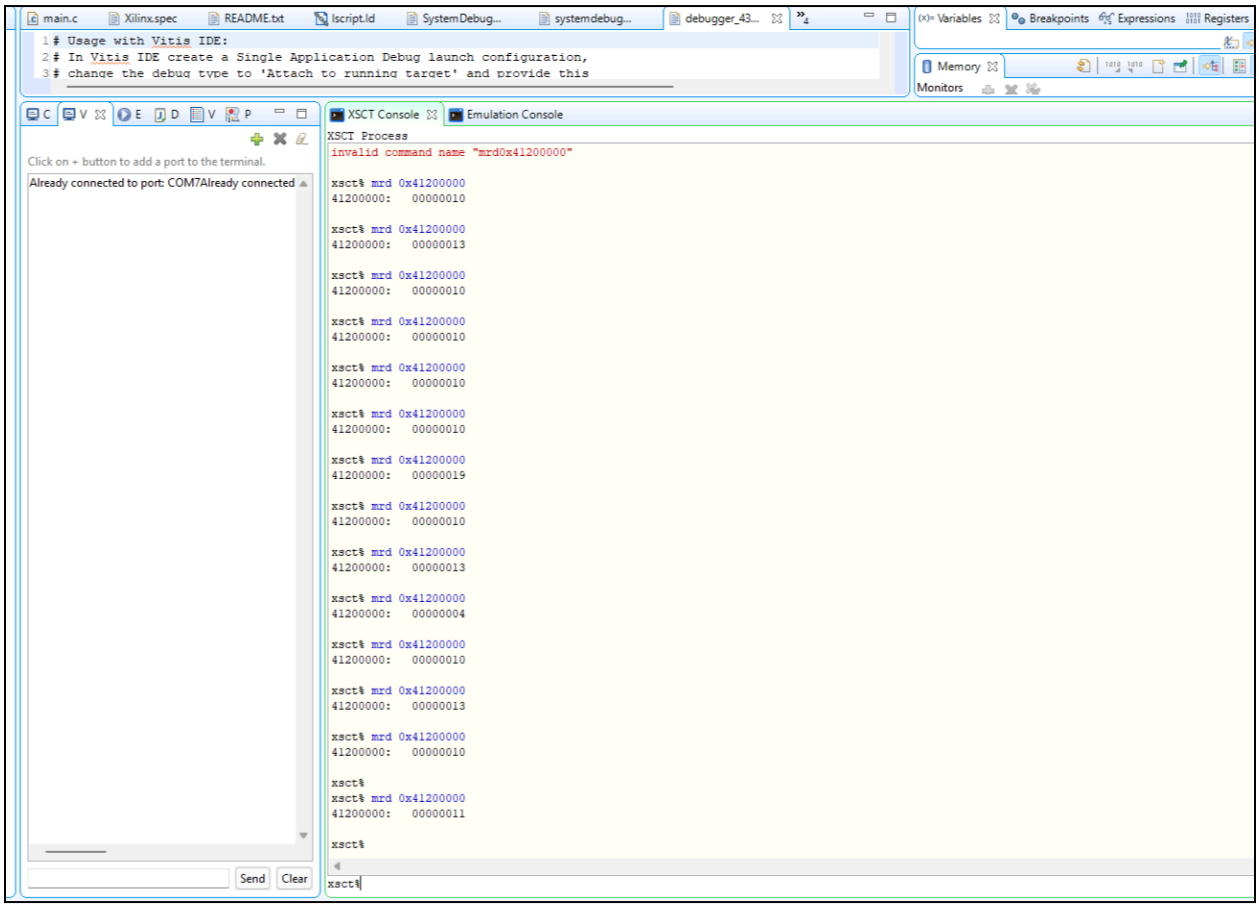


Figure 59. Vitis XSCT Terminal Output

Figure 59 shows the hardware output captured using the Vitis XSCT terminal during FPGA execution. Rather than reading results from a conventional serial printout window, the classification outputs were inspected directly through memory-mapped register reads in the XSCT console. This approach allowed the SmartNIC output values to be observed reliably in hardware even when direct UART terminal capture was less convenient. The repeated values shown in the console correspond to the packed classification output read from the AXI-connected interface, confirming that the hardware was producing valid SmartNIC results during runtime. This validated that the FPGA implementation was functioning correctly and that the observed outputs were consistent with the expected classifier behavior established

during simulation.

9. Conclusion

Lessons Learned

The biggest lesson learned from this project was understanding how to design a complete digital system from start to finish. This experience took us through the entire roadmap of digital design, from conceptualizing the architecture and drawing block diagrams to implementing RTL modules, developing testbenches, and validating functionality through simulation and FPGA deployment. It reinforced the importance of approaching system design methodically and considering both theoretical and practical constraints.

One of the most important lessons was the value of writing individual testbenches and verifying modules independently. Testing each component in isolation ensured correctness before integration, significantly reducing debugging time and improving overall reliability. Throughout the process, we learned that digital design is iterative challenges arise, solutions evolve, and improvements are made as understanding deepens.

For example, during the design phase, we initially assumed that a standard FIFO would work for both the high-throughput and area-optimized architectures. However, we discovered that the area-optimized version required additional control logic. This led to the introduction of a consumer-ready mechanism that prevented the FIFO from advancing when the downstream module was not prepared to receive data. This experience highlighted the importance of tailoring architectural components to specific design goals.

Another key takeaway was the importance of modular design and proper file organization. Structuring files clearly and using meaningful naming conventions simplified debugging, improved collaboration, and enhanced the scalability of the system. A well-organized design significantly streamlined both implementation and analysis.

We also learned that FPGA implementation is intensive and time-consuming, requiring careful analysis and optimization. Reports generated by tools such as Vivado including timing, resource utilization, and power analyses provided invaluable insights into system performance. These reports confirmed that the timing constraints established during synthesis and simulation were accurate and directly transferable to the real FPGA implementation. This demonstrated the reliability and importance of pre-synthesis analysis.

Static timing analysis proved essential in ensuring that the design met performance requirements without violating constraints. Understanding slack, clock domains, and critical paths was crucial in achieving timing closure and guaranteeing reliable operation. Observing

the FPGA fabric and resource utilization further emphasized the importance of optimization and efficient hardware design.

This project provided hands-on experience with industry-standard tools and methodologies, reinforcing the principles of digital system design. It was especially rewarding to see the SmartNIC functioning correctly on real hardware, validating both our theoretical understanding and practical implementation.

Areas for Improvement

While the project achieved its objectives, several opportunities for improvement were identified. One of the primary enhancements involves packet handling. In the current implementation, packets are stored in a ROM on the FPGA for testing and demonstration purposes. In real-world networking systems, packets would instead be streamed dynamically from external interfaces such as Ethernet or PCIe. Future iterations could replace the ROM-based packet generator with a live data stream or implement buffering mechanisms that store only a limited number of packets at a time. This would improve realism, scalability, and overall efficiency.

Another potential improvement is the integration of high-speed communication interfaces such as AXI-Stream, DMA engines, or PCIe. Incorporating these interfaces would enable the SmartNIC to operate in real-time environments and better reflect modern networking architectures.

Further optimization could also be applied to reduce power consumption and resource utilization. Techniques such as clock gating, BRAM-based buffering, and enhanced pipelining could improve efficiency while maintaining performance. Additionally, extending the classifier to support more complex rule sets, deeper packet inspection, and larger datasets would increase its applicability to real-world scenarios.

Final Remarks

This project was both challenging and rewarding. It provided invaluable insight into the design, implementation, and optimization of FPGA-based systems. Seeing the SmartNIC successfully operate on hardware demonstrated the power of parallel processing and highlighted the advantages of FPGA acceleration over traditional CPU-based solutions.

It also reinforced the importance of digital design principles such as modularity, static timing analysis, and hardware-software co-design. The ability to tailor systems for either high throughput or area efficiency illustrated the flexibility and versatility of FPGA technology.

In all, this experience deepened our understanding of computer systems engineering and strengthened our technical skills. It was incredibly fulfilling to carry the project from concept to a fully functional hardware implementation. The lessons learned throughout this process will serve as a strong foundation for future work in digital design, FPGA development, and high-performance computing.

References

[1] R. Gomar, SYSC 4320: Case Studies in Computer Systems – Lecture 10: A Clock Domain Crossing (CDC) Case Study: Asynchronous FIFO with Gearbox Module, Department of Systems and Computer Engineering, Carleton University, 2026.

[2] GeeksforGeeks, “TCP/IP Packet Format.” [Online]. Available: https://www.geeksforgeeks.org/computer-networks/tcp-ip-packet-format/ . Accessed: Apr. 16, 2026.

[3] IEEE Xplore, “FPGA-Based SmartNIC for Distributed Machine Learning,” 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10593663 . Accessed: Apr. 16, 2026.

[4] U. Kumar, “SmartNIC on FPGA,” 2023. [Online]. Available: https://utkar5hm.github.io/posts/smart-nic-on-fpga. Accessed: Apr. 16, 2026.

[5] NVIDIA Corporation, “What Is a SmartNIC?” [Online]. Available: https://blogs.nvidia.com/blog/what-is-a-smartnic. Accessed: Apr. 16, 2026.

[6] A. Marchesi, “NIC Benchmark Slides,” KTH Royal Institute of Technology. [Online]. Available: https://marchiesa.bitbucket.io/docs/chiesa/nic-bench-kth-slides.pdf. Accessed: Apr. 16, 2026.

[7] Reddit, “1G Ethernet Project,” r/FPGA. [Online]. Available: https://www.reddit.com/r/FPGA/comments/1rruvje/1g_ethernet_project/. Accessed: Apr. 16, 2026.

[8] GitHub, “Simple Ethernet Implementation.” [Online]. Available: https://github.com/0BAB1/simple-ethernet. Accessed: Apr. 16, 2026.

[9] Digilent Inc., Zybo Z7 Reference Manual. [Online]. Available: https://digilent.com/reference/programmable-logic/zybo/reference-manual. Accessed: Apr. 16, 2026.

Appendix A: RTL Code

High-Throughput Design:

```
/*
 * File Name  : packet_parser.v
 * Module Name: packet_parser
 * Author     : Uchenna Obikwelu
 * Course  : SYSC 4320 – Case Studies in Computer Systems
 * Project : SmartNIC Packet Classification Accelerator
 * Created : April 2026
 *
 * Description:
 * Parses a simplified 76-bit IPv4 packet header into its constituent fields.
 * The extracted fields include the IP version, protocol, source address,
 * and destination address. This module serves as the first stage in the
 * SmartNIC packet classification pipeline.
 *
 * Fields Extracted:
 *   - Version             : Bits [75:72]
 *   - Protocol            : Bits [71:64]
 *   - Source Address      : Bits [63:32]
 *   - Destination Address  : Bits [31:0]
 *
 * Notes:
 *   - This module implements purely combinational logic with zero latency.
 *   - Used in simulation, synthesis, and FPGA implementation.
 *   - Designed for both area-optimized and high-throughput SmartNIC architectures.
 */


module packet_parser(
  input [75:0] ip_packet,          // Simplified IPv4 header
  output reg [3:0] version,        // IP version (e.g., IPv4 = 4)
  output reg [7:0] protocol,       // Protocol number (e.g., TCP=6, UDP=17)
  output reg [31:0] src_address,   // Source IP address
  output reg [31:0] dest_addr      // Destination IP address
);

  // ----------------------------------------------------
  // Combinational logic to extract fields from the packet
  // ----------------------------------------------------
```

```
  always @(*) begin
    version = ip_packet[75:72];
    protocol = ip_packet[71:64];
    src_address = ip_packet[63:32];
    dest_addr = ip_packet[31:0];
  end
endmodule
```

```
/*
 * File Name  : rule_classifier_high_throughput.v
 * Module Name: rule_classifier_high_throughput
 * Author     : Uchenna Obikwelu
 * Course     : SYSC 4320 – Case Studies in Computer Systems
 * Project    : SmartNIC Packet Classification Accelerator
 * Created    : April 2026
 *
 * Description:
 * Implements a high-throughput packet classification engine using a
 * pipelined, parallel comparator architecture. The module evaluates
 * incoming packet fields against a predefined rule table and outputs
 * the corresponding action and matched rule identifier.
 *
 * Architecture:
 *   - Stage 1: Input latching
 *   - Stage 2: Parallel rule comparison (CAM-style)
 *   - Stage 3: Priority encoding and action selection
 *
 * Notes:
 *   - Optimized for maximum throughput using pipelining and parallelism.
 *   - Designed as the high-performance counterpart to the area-optimized version.
 *   - Used in simulation, synthesis, and FPGA implementation.
 */

module rule_classifier_high_throughput (
    input wire clk,
    input wire rst,
    input wire [31:0] dest_addr_in,
    input wire [7:0] protocol_in,
    output reg valid,
    output reg [3:0] matched_rule_id
```

```
);

parameter NUM_RULES = 16;

// Rule format: {Destination IP [40:9], Protocol [8:1], Action [0]}
// Action (1 = allow, 0 = drop)
reg [40:0] mem [0:NUM_RULES-1];

// Load rule table on reset
always @(posedge clk) begin
   if (rst) begin
      mem[0] <= {32'hE00000FB, 8'd17, 1'b1}; // 224.0.0.251 UDP allow
      mem[1] <= {32'hE0000016, 8'd2, 1'b1}; // 224.0.0.22 IGMP allow
      mem[2] <= {32'h0A0000DE, 8'd17, 1'b1}; // 10.0.0.222 UDP allow
      mem[3] <= {32'h0A0000DE, 8'd6, 1'b1}; // 10.0.0.222 TCP allow
      mem[4] <= {32'hEFFFFFFA, 8'd17, 1'b0}; // 239.255.255.250 UDP drop
      mem[5] <= {32'hFFFFFFFF, 8'd17, 1'b0}; // 255.255.255.255 UDP drop
      mem[6] <= {32'h0A0000FF, 8'd17, 1'b0}; // 10.0.0.255 UDP drop
      mem[7] <= {32'hA29F86EA, 8'd6, 1'b1}; // 162.159.134.234 TCP allow
      mem[8] <= {32'h8E7EF477, 8'd6, 1'b1}; // 142.126.244.119 TCP allow
      mem[9] <= {32'hD8EF2215, 8'd6, 1'b1}; // 216.239.34.21 TCP allow
      mem[10] <= {32'h226BF35D, 8'd6, 1'b1}; // 34.107.243.93 TCP allow
      mem[11] <= {32'h4047FFCC, 8'd17, 1'b0}; // 64.71.255.204 UDP drop
      mem[12] <= {32'h0A0000F0, 8'd6, 1'b0}; // 10.0.0.240 TCP drop
      mem[13] <= {32'hE00000FB, 8'd6, 1'b0}; // 224.0.0.251 TCP drop
      mem[14] <= {32'hE0000016, 8'd17, 1'b0}; // 224.0.0.22 UDP drop
      mem[15] <= {32'hFFFFFFFF, 8'd6, 1'b0}; // 255.255.255.255 TCP drop
   end
end

// Stage 1 pipeline registers used to latch inputs
reg [31:0] dest_addr_s0;
reg [7:0] prtcl_s0;

// Stage 2 combinational registers
reg [NUM_RULES-1:0] rule_match_comb;
reg [NUM_RULES-1:0] action_comb;

// Stage 2 combinational logic
// Parallel comparator outputs
// rule_match_comb and action_comb store the results of every rule in parallel.
// This enables high-throughput packet classification by evaluating all rules within a single
clock cycle.
integer i;
```

```
always @(*) begin
    rule_match_comb = {NUM_RULES{1'b0}};
    action_comb = {NUM_RULES{1'b0}};

    for (i = 0; i < NUM_RULES; i = i + 1) begin
        rule_match_comb[i] = (dest_addr_s0 == mem[i][40:9]) &&
                             (prtcl_s0 == mem[i][8:1]);
        action_comb[i] = mem[i][0];
    end
end

// Stage 2 pipeline registers
reg [NUM_RULES-1:0] rule_match_s1;
reg [NUM_RULES-1:0] action_s1;

// Stage 3 combinational registers
reg match_found;
reg valid_comb;
reg [3:0] matched_rule_id_comb;

// Stage 3 combinational logic
// Priority encoder: the first matching rule determines the output
integer k;
always @(*) begin
    match_found = 1'b0;
    valid_comb = 1'b0;
    matched_rule_id_comb = 4'b0;

    for (k = 0; k < NUM_RULES; k = k + 1) begin
        if (!match_found && rule_match_s1[k]) begin
        valid_comb = action_s1[k];
        matched_rule_id_comb = k[3:0];
        match_found = 1'b1;
        end
    end
end

// Sequential pipeline logic
always @(posedge clk) begin
    if (rst) begin
        dest_addr_s0 <= 32'd0;
        prtcl_s0 <= 8'd0;
        rule_match_s1 <= {NUM_RULES{1'b0}};
        action_s1 <= {NUM_RULES{1'b0}};
```

```
        valid <= 1'b0;
        matched_rule_id <= 4'd0;
    end else begin
        // Stage 1
        dest_addr_s0 <= dest_addr_in;
        prtcl_s0 <= protocol_in;

        // Stage 2
        rule_match_s1 <= rule_match_comb;
        action_s1 <= action_comb;

        // Stage 3
        valid <= valid_comb;
        matched_rule_id <= matched_rule_id_comb;
    end
end

endmodule



/*
 * File Name  : cdc_fifo_throughput.v
 * Modules    : rd_ctrl, wr_ctrl, sync_3ff, fifo_mem, async_fifo_top
 * Author     : Uchenna Obikwelu
 * Course     : SYSC 4320 – Case Studies in Computer Systems
 * Project    : SmartNIC Packet Classification Accelerator
 * Created    : April 2026
 *
 * Description:
 * Implements the asynchronous FIFO used for clock domain crossing in the
 * SmartNIC design. This file includes the read controller, write controller,
 * pointer synchronizers, FIFO memory, and the top-level FIFO integration module.
 *
 * Notes:
 *   - Uses Gray-coded pointers and 3-stage synchronizers for safe CDC.
 *   - Prevents invalid reads and writes using empty/full detection logic.
 *   - Used in simulation, synthesis, and FPGA implementation.
 */

// ----------------------------------------------------------------------------
// Read-side controller for FIFO empty detection and read pointer update
// ----------------------------------------------------------------------------
module rd_ctrl #(parameter PTR_SIZE = 5)
```

```
  (
    output wire os_fifo_empty,
    output wire os_rd_en,
    output reg [PTR_SIZE:0] ov_rd_pointer,
    output wire [PTR_SIZE:0] ov_rd_pointer_gry,
    input wire [PTR_SIZE:0] iv_wr_pointer_sync,
    input wire clk_read,
    input wire rstn
  );

  reg [PTR_SIZE:0] wr_pointer_sync_bin;


  // Convert synchronized write pointer from Gray code to binary for empty detection
  integer i;
  always @(*) begin
      wr_pointer_sync_bin[PTR_SIZE] = iv_wr_pointer_sync[PTR_SIZE];
      for (i = PTR_SIZE-1; i >= 0; i = i-1) begin
        wr_pointer_sync_bin[i] = wr_pointer_sync_bin[i+1] ^ iv_wr_pointer_sync[i];
      end
  end

  //fifo empty logic to avoid reading when empty
  assign os_fifo_empty = wr_pointer_sync_bin == ov_rd_pointer;

  //Read pointer increment when data is available
  always @(posedge clk_read or negedge rstn) begin
    if(!rstn) ov_rd_pointer <= 0;
    else if (os_rd_en) ov_rd_pointer <= ov_rd_pointer + 1'b1;
  end

  //Read enable logic
  assign os_rd_en = !os_fifo_empty;

// Convert read pointer from binary to Gray code before crossing into write domain
  assign ov_rd_pointer_gry = (ov_rd_pointer >> 1) ^ ov_rd_pointer;

endmodule

// ------------------------------------------------------------------------------
// Write-side controller for FIFO full detection and write pointer update
// ------------------------------------------------------------------------------
module wr_ctrl #(parameter PTR_SIZE = 5)
  (
```

```
    output wire os_fifo_full,
    input  wire is_wr_en,
    output reg  [PTR_SIZE:0] ov_wr_pointer,
    output wire [PTR_SIZE:0] ov_wr_pointer_gry,
    input  wire [PTR_SIZE:0] iv_rd_pointer_sync,
    input  wire rstn,
    input  wire clk_write
  );

  reg [PTR_SIZE:0] rd_pointer_sync_bin;

   //Gray to Binary for synchronized read pointer for fifo full logic
  integer i;
  always @(*) begin
    rd_pointer_sync_bin[PTR_SIZE] = iv_rd_pointer_sync[PTR_SIZE];
      for (i = PTR_SIZE-1; i >= 0; i = i-1) begin
        rd_pointer_sync_bin[i] = rd_pointer_sync_bin[i+1] ^ iv_rd_pointer_sync[i];
      end
  end


  //fifo full logic to avoid writing when full
  assign os_fifo_full = (ov_wr_pointer[PTR_SIZE-1:0] ==
rd_pointer_sync_bin[PTR_SIZE-1:0]) && (ov_wr_pointer[PTR_SIZE] !=

rd_pointer_sync_bin[PTR_SIZE]);

  // Convert write pointer from binary to Gray code before crossing into read domain
  assign ov_wr_pointer_gry = (ov_wr_pointer >> 1) ^ ov_wr_pointer;


  // Write pointer increment when write_en & !os_fifo_full
  always @ (posedge clk_write or negedge rstn) begin
    if (!rstn) begin
      ov_wr_pointer <= 0;
    end
    else begin
      if (is_wr_en && !os_fifo_full) begin
        ov_wr_pointer <= ov_wr_pointer + 1;
      end
    end
  end
endmodule
```

```
// ------------------------------------------------------------------------------
// Three-stage synchronizer for safely crossing Gray-coded pointers between clock domains
// ------------------------------------------------------------------------------
// 3-FF synchronizers
module sync_3ff #(parameter PTR_SIZE = 5)

  // inputs and outputs
  (
    output reg [PTR_SIZE:0] ov_dout_sync,
    input wire [PTR_SIZE:0] iv_din_gry,
    input wire sync_clk,
    input wire rstn
  );

  reg [PTR_SIZE:0] sig_meta1, sig_meta2;

  always @(posedge sync_clk or negedge rstn) begin
    if (!rstn)begin
      sig_meta1 <= 0;
      sig_meta2 <= 0;
      ov_dout_sync <= 0;
    end else begin
      sig_meta1 <= iv_din_gry;
      sig_meta2 <= sig_meta1;
      ov_dout_sync <= sig_meta2;
      end
  end
endmodule

// ------------------------------------------------------------------------------
// Dual-clock FIFO memory block for packet storage
// ------------------------------------------------------------------------------
module fifo_mem # (
  parameter PTR_SIZE = 5,
  parameter WIDTH = 160,
  parameter DEPTH = 32
)(
  output reg [WIDTH-1:0] ov_dout,
  input clk_read,
  input clk_write,
  input rstn,
  input wr_en,
  input rd_en,
```

```
  input [WIDTH - 1:0] ip_packet,
  input [PTR_SIZE -1 :0] iv_rd_pointer,
  input [PTR_SIZE-1:0] iv_wr_pointer
);

  reg [WIDTH-1:0] mem [DEPTH - 1:0];
  integer i;

  //write logic
  always @(posedge clk_write or negedge rstn)begin
    if (!rstn) begin
      for (i=0; i<DEPTH; i=i+1) begin
        mem[i] <= {WIDTH{1'b0}};
      end
    end

    else begin
      if (wr_en) begin
        mem[iv_wr_pointer] <= ip_packet;
      end
    end
  end


  //read logic
  always @(posedge clk_read or negedge rstn) begin
    if(!rstn) begin
      ov_dout <= {WIDTH{1'b0}};
    end
    else begin
      if (rd_en) begin
        ov_dout <= mem[ iv_rd_pointer];
      end
    end
  end
endmodule


// ----------------------------------------------------------------------------
// Top-level asynchronous FIFO module integrating controllers, synchronizers, and memory
// ----------------------------------------------------------------------------
module async_fifo_top #(
  parameter PTR_SIZE = 5,
  parameter WIDTH    = 160,
```

```
  parameter DEPTH    = 32
)(
  input wire clk_write,
  input wire clk_read,
  input wire rstn,

  // write side
  input wire wr_en,
  input wire [WIDTH-1:0] ip_packet_in,

  // read side
  output wire [WIDTH-1:0] ov_dout,

  // status
  output wire os_fifo_full,
  output wire os_fifo_empty,

  // Exposed for debug and waveform observation
  output wire [PTR_SIZE:0]  ov_wr_pointer,
  output wire [PTR_SIZE:0]  ov_rd_pointer,
  output wire [PTR_SIZE:0]  ov_wr_pointer_gry,
  output wire [PTR_SIZE:0]  ov_rd_pointer_gry
);

  // Internal synchronized Gray pointers
  wire [PTR_SIZE:0] w_rd_pointer_gry_sync_to_wr;
  wire [PTR_SIZE:0] w_wr_pointer_gry_sync_to_rd;

  // Internal enables
  wire rd_en;
  wire w_wr_en_mem;

  // Write controller
  wr_ctrl #(.PTR_SIZE(PTR_SIZE)) wr_ctrl_inst
  (
    .os_fifo_full(os_fifo_full),
    .is_wr_en(wr_en),
    .ov_wr_pointer(ov_wr_pointer),
    .ov_wr_pointer_gry(ov_wr_pointer_gry),
    .iv_rd_pointer_sync(w_rd_pointer_gry_sync_to_wr),
    .rstn(rstn),
    .clk_write (clk_write)
  );
```

```
// Read controller
rd_ctrl #(.PTR_SIZE(PTR_SIZE)) rd_ctrl_inst
(
  .os_fifo_empty(os_fifo_empty),
  .os_rd_en(rd_en),
  .ov_rd_pointer(ov_rd_pointer),
  .ov_rd_pointer_gry (ov_rd_pointer_gry),
  .iv_wr_pointer_sync(w_wr_pointer_gry_sync_to_rd),
  .clk_read(clk_read),
  .rstn(rstn)
);


// Synchronize read Gray pointer from read side into write domain
sync_3ff #(.PTR_SIZE(PTR_SIZE)) sync_r2w_inst
(
  .ov_dout_sync(w_rd_pointer_gry_sync_to_wr),
  .iv_din_gry(ov_rd_pointer_gry), //ov_rd_pointer_gry -> output from read module
  .sync_clk(clk_write),
  .rstn(rstn)
);

// Synchronize write Gray pointer from write side into read domain
sync_3ff #(.PTR_SIZE(PTR_SIZE)) sync_w2r_inst
(
  .ov_dout_sync(w_wr_pointer_gry_sync_to_rd),
  .iv_din_gry(ov_wr_pointer_gry),  //ov_wr_pointer_gry -> output from write module
  .sync_clk(clk_read),
  .rstn(rstn)
);


// Gate memory write enable to prevent writes when the FIFO is full
assign w_wr_en_mem = wr_en && !os_fifo_full;


// FIFO memory
fifo_mem #(
  .PTR_SIZE(PTR_SIZE),
  .WIDTH(WIDTH),
  .DEPTH(DEPTH)
) fifo_mem_inst
(
```

```
    .ov_dout(ov_dout),
    .clk_read(clk_read),
    .clk_write(clk_write),
    .rstn(rstn),
    .wr_en(w_wr_en_mem),
    .rd_en(rd_en),
    .ip_packet(ip_packet_in),
    .iv_rd_pointer(ov_rd_pointer[PTR_SIZE-1:0]),
    .iv_wr_pointer(ov_wr_pointer[PTR_SIZE-1:0])
  );

endmodule



/*
 * File Name  : packet_parser_rule_classifier_top_sim.v
 * Module Name: smart_nic_high_throughput_top
 * Author     : Uchenna Obikwelu
 * Course     : SYSC 4320 – Case Studies in Computer Systems
 * Project    : SmartNIC Packet Classification Accelerator
 * Created    : April 2026
 *
 * Description:
 * Integrates the packet parser and the high-throughput rule classifier
 * into a complete packet classification pipeline. The parser extracts
 * relevant header fields from the input packet, and the classifier
 * matches the parsed fields against the rule table.
 *
 * Architecture:
 *   - Stage 1: Packet parsing
 *   - Stage 2: High-throughput rule classification
 *
 * Notes:
 *   - This file is an integration module for the parser and classifier.
 *   - It is used as part of the high-throughput SmartNIC design hierarchy.
 *   - It is instantiated alongside the async_fifo_top module in the testbench.
 *   - Used in simulation, synthesis, and FPGA implementation.
 */

module smart_nic_high_throughput_top (
    input  wire        clk,
    input  wire        rst,
    input  wire [75:0] ip_packet,
```

```
    output wire        valid,
    output wire [3:0]  matched_rule_id
);

    // packet_parser outputs
    wire [3:0]  version;
    wire [7:0]  protocol;
    wire [31:0] src_address;
    wire [31:0] dest_addr;

    // Packet parsing stage
    packet_parser parser_inst (
        .ip_packet(ip_packet),
        .version(version),
        .protocol(protocol),
        .src_address(src_address),
        .dest_addr(dest_addr)
    );

    // High-throughput rule classification stage
    rule_classifier_high_throughput classifier_inst (
        .clk(clk),
        .rst(rst),
        .dest_addr_in(dest_addr),
        .protocol_in(protocol),
        .valid(valid),
        .matched_rule_id(matched_rule_id)
    );

endmodule
```

Area-Optimized Design

```
`timescale 1ns/1ps
/*
 * File Name  : cdc_fifo_area.v
 * Modules    : rd_ctrl, wr_ctrl, sync_3ff, fifo_mem, async_fifo_top_area
 * Author     : Ben Narmer
 * Course     : SYSC 4320 – Case Studies in Computer Systems
 * Project    : SmartNIC Packet Classification Accelerator
 * Created    : April 2026
 *
```

```
 * Description:
 * Implements the asynchronous FIFO used for clock domain crossing in the
 * area-optimized SmartNIC design. This file includes the read controller,
 * write controller, pointer synchronizers, FIFO memory, and the top-level
 * FIFO integration module.
 *
 * Notes:
 *   - Uses Gray-coded pointers and 3-stage synchronizers for safe CDC.
 *   - Prevents invalid reads and writes using empty/full detection logic.
 *   - Includes consumer-ready gating on the read side for controlled packet release.
 *   - Used in simulation, synthesis, and implementation analysis.
 */

// Read controller module
module rd_ctrl #(parameter PTR_SIZE = 5)
  (
    output wire os_fifo_empty,
    output wire os_rd_en,
    output reg [PTR_SIZE:0] ov_rd_pointer,
    output wire [PTR_SIZE:0] ov_rd_pointer_gry,
    input wire [PTR_SIZE:0] iv_wr_pointer_sync,
    input wire is_consumer_ready,
    input wire clk_read,
    input wire rstn
  );

  reg [PTR_SIZE:0] wr_pointer_sync_bin;

  //Gray to Binary for synchronized write pointer for fifo empty logic
  integer i;
  always @(*) begin
      wr_pointer_sync_bin[PTR_SIZE] = iv_wr_pointer_sync[PTR_SIZE];
      for (i = PTR_SIZE-1; i >= 0; i = i-1) begin
        wr_pointer_sync_bin[i] = wr_pointer_sync_bin[i+1] ^ iv_wr_pointer_sync[i];
      end
  end

  //fifo empty logic to avoid reading when empty
  assign os_fifo_empty = wr_pointer_sync_bin == ov_rd_pointer;

  //Read pointer increment when data is available
  always @(posedge clk_read or negedge rstn) begin
    if(!rstn) ov_rd_pointer <= 0;
    else if (os_rd_en) ov_rd_pointer <= ov_rd_pointer + 1'b1;
```

```
  end

  //Read enable logic
  assign os_rd_en = !os_fifo_empty && is_consumer_ready;

  //Binary to Gray for read pointer crossing to write side
  assign ov_rd_pointer_gry = (ov_rd_pointer >> 1) ^ ov_rd_pointer;

endmodule


// Write controller module
module wr_ctrl #(parameter PTR_SIZE = 5)
  (
    output wire os_fifo_full,
    input  wire is_wr_en,
    output reg  [PTR_SIZE:0] ov_wr_pointer,
    output wire [PTR_SIZE:0] ov_wr_pointer_gry,
    input  wire [PTR_SIZE:0] iv_rd_pointer_sync,
    input  wire rstn,
    input  wire clk_write
  );

  reg [PTR_SIZE:0] rd_pointer_sync_bin;

   //Gray to Binary for synchronized read pointer for fifo full logic
  integer i;
  always @(*) begin
    rd_pointer_sync_bin[PTR_SIZE] = iv_rd_pointer_sync[PTR_SIZE];
      for (i = PTR_SIZE-1; i >= 0; i = i-1) begin
        rd_pointer_sync_bin[i] = rd_pointer_sync_bin[i+1] ^ iv_rd_pointer_sync[i];
      end
  end

  //fifo full logic to avoid writing when full
  assign os_fifo_full = (ov_wr_pointer[PTR_SIZE-1:0] ==
rd_pointer_sync_bin[PTR_SIZE-1:0]) && (ov_wr_pointer[PTR_SIZE] !=

rd_pointer_sync_bin[PTR_SIZE]);

  //Binary to Grey for write pointer crossing to read domain
  assign ov_wr_pointer_gry = (ov_wr_pointer >> 1) ^ ov_wr_pointer;
```

```
  // Write pointer increment when write_en & !os_fifo_full
  always @ (posedge clk_write or negedge rstn) begin
    if (!rstn) begin
      ov_wr_pointer <= 0;
    end
    else begin
      if (is_wr_en && !os_fifo_full) begin
        ov_wr_pointer <= ov_wr_pointer + 1;
      end
    end
  end
endmodule


// 3-FF synchronizers
module sync_3ff #(parameter PTR_SIZE = 5)

  // inputs and outputs
  (
    output reg [PTR_SIZE:0] ov_dout_sync,
    input wire [PTR_SIZE:0] iv_din_gry,
    input wire sync_clk,
    input wire rstn
  );

  reg [PTR_SIZE:0] sig_meta1, sig_meta2;

  always @(posedge sync_clk or negedge rstn) begin
    if (!rstn)begin
      sig_meta1 <= 0;
      sig_meta2 <= 0;
      ov_dout_sync <= 0;
    end else begin
      sig_meta1 <= iv_din_gry;
      sig_meta2 <= sig_meta1;
      ov_dout_sync <= sig_meta2;
      end
  end
endmodule


module fifo_mem # (
  parameter PTR_SIZE = 5,
  parameter WIDTH = 160,
```

```
  parameter DEPTH = 32
)(
  output reg [WIDTH-1:0] ov_dout,
  input clk_read,
  input clk_write,
  input rstn,
  input wr_en,
  input rd_en,
  input [WIDTH - 1:0] ip_packet,
  input [PTR_SIZE -1 :0] iv_rd_pointer,
  input [PTR_SIZE-1:0] iv_wr_pointer
);

  reg [WIDTH-1:0] mem [DEPTH - 1:0];
  integer i;

  //write logic
  always @(posedge clk_write or negedge rstn)begin
    if (!rstn) begin
      for (i=0; i<DEPTH; i=i+1) begin
        mem[i] <= {WIDTH{1'b0}};
      end
    end

    else begin
      if (wr_en) begin
        mem[iv_wr_pointer] <= ip_packet;
      end
    end
  end


  //read logic
  always @(posedge clk_read or negedge rstn) begin
    if(!rstn) begin
      ov_dout <= {WIDTH{1'b0}};
    end
    else begin
      if (rd_en) begin
        ov_dout <= mem[ iv_rd_pointer];
      end
    end
  end
endmodule
```

```
module async_fifo_top_area #(
  parameter PTR_SIZE = 5,
  parameter WIDTH    = 160,
  parameter DEPTH    = 32
)(
  input wire clk_write,
  input wire clk_read,
  input wire rstn,
  input wire i_consumer_ready,

  // write side
  input wire wr_en,
  input wire [WIDTH-1:0] ip_packet_in,

  // read side
  output wire [WIDTH-1:0] ov_dout,

  // status
  output wire os_fifo_full,
  output wire os_fifo_empty,

  // setting these as output for debug
  output wire [PTR_SIZE:0]  ov_wr_pointer,
  output wire [PTR_SIZE:0]  ov_rd_pointer,
  output wire [PTR_SIZE:0]  ov_wr_pointer_gry,
  output wire [PTR_SIZE:0]  ov_rd_pointer_gry
);

  // Internal synchronized Gray pointers
  wire [PTR_SIZE:0] w_rd_pointer_gry_sync_to_wr;
  wire [PTR_SIZE:0] w_wr_pointer_gry_sync_to_rd;

  // Internal enables
  wire rd_en;
  wire w_wr_en_mem;

  // Write controller
  wr_ctrl #(.PTR_SIZE(PTR_SIZE)) wr_ctrl_inst
  (
    .os_fifo_full(os_fifo_full),
    .is_wr_en(wr_en),
    .ov_wr_pointer(ov_wr_pointer),
```

```
  .ov_wr_pointer_gry(ov_wr_pointer_gry),
  .iv_rd_pointer_sync(w_rd_pointer_gry_sync_to_wr),
  .rstn(rstn),
  .clk_write (clk_write)
);


// Read controller
rd_ctrl #(.PTR_SIZE(PTR_SIZE)) rd_ctrl_inst
(
  .os_fifo_empty(os_fifo_empty),
  .os_rd_en(rd_en),
  .ov_rd_pointer(ov_rd_pointer),
  .ov_rd_pointer_gry (ov_rd_pointer_gry),
  .iv_wr_pointer_sync(w_wr_pointer_gry_sync_to_rd),
  .is_consumer_ready(i_consumer_ready),
  .clk_read(clk_read),
  .rstn(rstn)
);

// Synchronize read Gray pointer from read side into write domain
sync_3ff #(.PTR_SIZE(PTR_SIZE)) sync_r2w_inst
(
  .ov_dout_sync(w_rd_pointer_gry_sync_to_wr),
  .iv_din_gry(ov_rd_pointer_gry), //ov_rd_pointer_gry -> output from read module
  .sync_clk(clk_write),
  .rstn(rstn)
);

// Synchronize write Gray pointer from write side into read domain
sync_3ff #(.PTR_SIZE(PTR_SIZE)) sync_w2r_inst
(
  .ov_dout_sync(w_wr_pointer_gry_sync_to_rd),
  .iv_din_gry(ov_wr_pointer_gry),  //ov_wr_pointer_gry -> output from write module
  .sync_clk(clk_read),
  .rstn(rstn)
);

// Actual memory write enable to ensure not writing when fifo full
assign w_wr_en_mem = wr_en && !os_fifo_full;


// FIFO memory
fifo_mem #(
```

```
    .PTR_SIZE(PTR_SIZE),
    .WIDTH(WIDTH),
    .DEPTH(DEPTH)
  ) fifo_mem_inst
  (
    .ov_dout(ov_dout),
    .clk_read(clk_read),
    .clk_write(clk_write),
    .rstn(rstn),
    .wr_en(w_wr_en_mem),
    .rd_en(rd_en),
    .ip_packet(ip_packet_in),
    .iv_rd_pointer(ov_rd_pointer[PTR_SIZE-1:0]),
    .iv_wr_pointer(ov_wr_pointer[PTR_SIZE-1:0])
  );

endmodule



/*
 * File Name  : rule_classifier_area_optimized.v
 * Module Name: rule_classifier_area_optimized
 * Author     : Ben Narmer
 * Course     : SYSC 4320 – Case Studies in Computer Systems
 * Project    : SmartNIC Packet Classification Accelerator
 * Created    : April 2026
 *
 * Description:
 * Implements the area-optimized rule classifier for the SmartNIC design.
 * Instead of comparing all rules in parallel, this module checks one rule
 * per cycle using shared comparison logic and outputs the corresponding
 * decision when a match is found.
 *
 * Architecture:
 *   - Sequential rule scanning using a shared comparator
 *   - One rule evaluated per clock cycle
 *
 * Notes:
 *   - Optimized for reduced hardware cost through resource sharing.
 *   - Uses a packet-valid handshake and packet ID tracking for simulation.
 *   - Used in simulation, synthesis, and implementation analysis.
 */
```

```
module rule_classifier_area_optimized (
  input  wire        clk,
  input  wire        rst,
  input  wire [31:0] dest_addr_in,
  input  wire [7:0]  protocol_in,
  input  wire        i_packet_valid,   // tells classifier a fresh packet arrived
  input  wire [7:0]  i_packet_id,      // packet tag from TB top
  output reg         valid,
  output reg [3:0]   matched_rule_id,
  output wire        o_ready_for_packet,
  output reg         o_result_done,    // 1-cycle pulse when decision is ready
  output reg [7:0]   o_packet_id       // packet id associated with decision
);

  // Internal state for sequential rule checking
  reg [39:0] key_reg;
  reg [3:0]  rule_idx;
  reg        busy;

  // Current rule selected by rule_idx
  reg [39:0] rule_key;
  reg        rule_action;
  reg [7:0] packet_id_reg;

  // Rule table implemented as combinational selection logic
  always @(*) begin
    case (rule_idx)
      4'd0:  begin rule_key = {32'hE00000FB, 8'd17}; rule_action = 1'b1; end // 224.0.0.251
UDP allow
      4'd1:  begin rule_key = {32'hE0000016, 8'd2 }; rule_action = 1'b1; end // 224.0.0.22
IGMP allow
      4'd2:  begin rule_key = {32'h0A0000DE, 8'd17}; rule_action = 1'b1; end // 10.0.0.222
UDP allow
      4'd3:  begin rule_key = {32'h0A0000DE, 8'd6 }; rule_action = 1'b1; end // 10.0.0.222
TCP allow
      4'd4:  begin rule_key = {32'hEFFFFFFA, 8'd17}; rule_action = 1'b0; end //
239.255.255.250 UDP drop
      4'd5:  begin rule_key = {32'hFFFFFFFF, 8'd17}; rule_action = 1'b0; end //
255.255.255.255 UDP drop
      4'd6:  begin rule_key = {32'h0A0000FF, 8'd17}; rule_action = 1'b0; end // 10.0.0.255
UDP drop
      4'd7:  begin rule_key = {32'hA29F86EA, 8'd6 }; rule_action = 1'b1; end //
162.159.134.234 TCP allow
```

```
    4'd8:  begin rule_key = {32'h8E7EF477, 8'd6 }; rule_action = 1'b1; end // 142.126.244.119
TCP allow
    4'd9:  begin rule_key = {32'hD8EF2215, 8'd6 }; rule_action = 1'b1; end // 216.239.34.21
TCP allow
    4'd10: begin rule_key = {32'h226BF35D, 8'd6 }; rule_action = 1'b1; end // 34.107.243.93
TCP allow
    4'd11: begin rule_key = {32'h4047FFCC, 8'd17}; rule_action = 1'b0; end // 64.71.255.204
UDP drop
    4'd12: begin rule_key = {32'h0A0000F0, 8'd6 }; rule_action = 1'b0; end // 10.0.0.240
TCP drop
    4'd13: begin rule_key = {32'hE00000FB, 8'd6 }; rule_action = 1'b0; end // 224.0.0.251
TCP drop
    4'd14: begin rule_key = {32'hE0000016, 8'd17}; rule_action = 1'b0; end // 224.0.0.22
UDP drop
    default: begin rule_key = {32'hFFFFFFFF, 8'd6 }; rule_action = 1'b0; end //
255.255.255.255 TCP drop
  endcase
 end

 // Compare the current packet key against the selected rule
 wire match;
 assign match = (key_reg == rule_key);

 // Indicates whether the classifier can accept a new packet
 assign o_ready_for_packet = !busy;


 // Sequential control logic for packet acceptance, rule scanning, and result generation
 always @(posedge clk) begin
  if (rst) begin
   key_reg         <= 40'd0;
   packet_id_reg   <= 8'd0;
   rule_idx        <= 4'd0;
   busy            <= 1'b0;
   valid           <= 1'b0;
   matched_rule_id <= 4'd0;
   o_result_done   <= 1'b0;
   o_packet_id     <= 8'd0;
  end else begin

   o_result_done <= 1'b0;    // Keep done low unless classification completes this cycle

   if (!busy) begin
    valid          <= 1'b0;
```

```
        matched_rule_id <= 4'd0;

        // only start when a fresh packet really arrived
        if (i_packet_valid) begin
          key_reg       <= {dest_addr_in, protocol_in};
          packet_id_reg <= i_packet_id;
          rule_idx      <= 4'd0;
          busy          <= 1'b1;
        end
      end else if (match) begin
        valid           <= rule_action;
        matched_rule_id <= rule_idx;
        o_packet_id     <= packet_id_reg;
        o_result_done   <= 1'b1;
        busy            <= 1'b0;
      end else if (rule_idx == 4'd15) begin
        valid           <= 1'b0;
        matched_rule_id <= 4'd0;
        o_packet_id     <= packet_id_reg;
        o_result_done   <= 1'b1;
        busy            <= 1'b0;
      end else begin
        rule_idx        <= rule_idx + 4'd1;
      end
    end
  end
endmodule



/*
 * File Name  : packet_parser.v
 * Module Name: packet_parser
 * Author     : Uchenna Obikwelu
 * Course  : SYSC 4320 – Case Studies in Computer Systems
 * Project : SmartNIC Packet Classification Accelerator
 * Created : April 2026
 *
 * Description:
 * Parses a simplified 76-bit IPv4 packet header into its constituent fields.
 * The extracted fields include the IP version, protocol, source address,
 * and destination address. This module serves as the first stage in the
 * SmartNIC packet classification pipeline.
 *
```

```
 * Fields Extracted:
 *   - Version             : Bits [75:72]
 *   - Protocol            : Bits [71:64]
 *   - Source Address      : Bits [63:32]
 *   - Destination Address  : Bits [31:0]
 *
 * Notes:
 *   - This module implements purely combinational logic with zero latency.
 *   - Used in simulation, synthesis, and FPGA implementation.
 *   - Designed for both area-optimized and high-throughput SmartNIC architectures.
 */


module packet_parser(
  input [75:0] ip_packet,          // Simplified IPv4 header
  output reg [3:0] version,        // IP version (e.g., IPv4 = 4)
  output reg [7:0] protocol,       // Protocol number (e.g., TCP=6, UDP=17)
  output reg [31:0] src_address,   // Source IP address
  output reg [31:0] dest_addr      // Destination IP address
);

  // ----------------------------------------------------
  // Combinational logic to extract fields from the packet
  // ----------------------------------------------------
  always @(*) begin
    version = ip_packet[75:72];
    protocol = ip_packet[71:64];
    src_address = ip_packet[63:32];
    dest_addr = ip_packet[31:0];
  end
endmodule



/*
 * File Name  : packet_parser_rule_classifier_area_top.v
 * Module Name: smart_nic_area_optimized_top
 * Author     : Ben Narmer
 * Course     : SYSC 4320 – Case Studies in Computer Systems
 * Project    : SmartNIC Packet Classification Accelerator
 * Created    : April 2026
 *
 * Description:
 * Integrates the packet parser and the area-optimized rule classifier
```

```
 * into a complete packet classification pipeline. The parser extracts
 * relevant header fields from the input packet, and the classifier
 * sequentially evaluates the parsed fields against a rule table to
 * determine the corresponding action.
 *
 * Architecture:
 *   - Stage 1: Packet parsing
 *   - Stage 2: Area-optimized rule classification using resource sharing
 *
 * Notes:
 *   - This module implements the low-area SmartNIC architecture.
 *   - The classifier evaluates one rule per clock cycle to minimize hardware usage.
 *   - Includes packet-valid handshaking and packet ID tracking for simulation.
 *   - Used in simulation, synthesis, and implementation analysis.
 *   - Instantiated alongside the async_fifo_top_area module in the CDC testbench.
 */

module smart_nic_area_optimized_top (
    input  wire        clk,
    input  wire        rst,
    input  wire [75:0] ip_packet,
    input  wire        i_packet_valid,
    input  wire [7:0]  i_packet_id,
    output wire        valid,
    output wire [3:0]  matched_rule_id,
    output wire        o_ready_for_packet,
    output wire        o_result_done,
    output wire [7:0]  o_packet_id
);

    wire [3:0]  version;
    wire [7:0]  protocol;
    wire [31:0] src_address;
    wire [31:0] dest_addr;

    packet_parser parser_inst (
        .ip_packet(ip_packet),
        .version(version),
        .protocol(protocol),
        .src_address(src_address),
        .dest_addr(dest_addr)
    );

    rule_classifier_area_optimized classifier_inst (
```

```
        .clk(clk),
        .rst(rst),
        .dest_addr_in(dest_addr),
        .protocol_in(protocol),
        .i_packet_valid(i_packet_valid),
        .i_packet_id(i_packet_id),
        .valid(valid),
        .matched_rule_id(matched_rule_id),
        .o_ready_for_packet(o_ready_for_packet),
        .o_result_done(o_result_done),
        .o_packet_id(o_packet_id)
    );

endmodule



`timescale 1ns / 1ps

/*
 * File Name  : smartnic_cdc_area_optimized_top_for_synthesis.v
 * Module Name: smartnic_cdc_area_optimized_top
 * Author     : Ben Narmer
 * Course     : SYSC 4320 – Case Studies in Computer Systems
 * Project    : SmartNIC Packet Classification Accelerator
 * Created    : April 2026
 *
 * Description:
 * Synthesis top wrapper for the area-optimized SmartNIC design with
 * asynchronous FIFO-based clock domain crossing. This module connects
 * the CDC FIFO and the area-optimized SmartNIC pipeline into a single
 * top-level design suitable for RTL elaboration, synthesis, and
 * implementation analysis in Vivado.
 *
 * Architecture:
 *   - Stage 1: Asynchronous FIFO for clock domain crossing
 *   - Stage 2: Packet-valid and packet-ID generation in the read domain
 *   - Stage 3: Area-optimized packet parsing and classification
 *
 * Notes:
 *   - This file was created as the synthesis-level top wrapper for the
 *     complete area-optimized CDC design.
 *   - It enables proper RTL elaboration, synthesis, and implementation
 *     analysis of the full design in Vivado.
```

```
 *   - Instantiates async_fifo_top_area and smart_nic_area_optimized_top.
 *   - Used in RTL elaboration, synthesis, and implementation analysis.
 *   - Not deployed on the physical FPGA board.
 */

module smartnic_cdc_area_optimized_top (
    input  wire        clk_write,
    input  wire        clk_read,
    input  wire        rstn,
    input  wire        wr_en,
    input  wire [75:0] ip_packet_in,

    output wire        valid,
    output wire [3:0]  matched_rule_id
);

    localparam PTR_SIZE = 5;
    localparam WIDTH    = 76;
    localparam DEPTH    = 32;

    reg        fifo_data_valid_d;
    reg [7:0]  packet_id_counter;
    reg [7:0]  packet_id_to_classifier;

    wire       consumer_ready;
    wire       result_done;
    wire [7:0] result_packet_id;

    wire [WIDTH-1:0] fifo_dout;
    wire             os_fifo_full;
    wire             os_fifo_empty;
    wire [PTR_SIZE:0] ov_wr_pointer;
    wire [PTR_SIZE:0] ov_rd_pointer;
    wire [PTR_SIZE:0] ov_wr_pointer_gry;
    wire [PTR_SIZE:0] ov_rd_pointer_gry;

    // FIFO instance
    async_fifo_top_area #(
        .PTR_SIZE(PTR_SIZE),
        .WIDTH(WIDTH),
        .DEPTH(DEPTH)
    ) fifo_dut (
        .clk_write(clk_write),
        .clk_read(clk_read),
```

```
        .rstn(rstn),
        .wr_en(wr_en),
        .ip_packet_in(ip_packet_in),
        .ov_dout(fifo_dout),
        .os_fifo_full(os_fifo_full),
        .os_fifo_empty(os_fifo_empty),
        .ov_wr_pointer(ov_wr_pointer),
        .ov_rd_pointer(ov_rd_pointer),
        .ov_wr_pointer_gry(ov_wr_pointer_gry),
        .ov_rd_pointer_gry(ov_rd_pointer_gry),
        .i_consumer_ready(consumer_ready)
    );

    // Generate packet-valid and packet-id signals in read domain
    always @(posedge clk_read or negedge rstn) begin
        if (!rstn) begin
            fifo_data_valid_d       <= 1'b0;
            packet_id_counter       <= 8'd0;
            packet_id_to_classifier <= 8'd0;
        end else begin
            fifo_data_valid_d <= (!os_fifo_empty && consumer_ready);

            if (!os_fifo_empty && consumer_ready) begin
                packet_id_to_classifier <= packet_id_counter;
                packet_id_counter <= packet_id_counter + 1'b1;
            end
        end
    end

    // SmartNIC area-optimized instance
    smart_nic_area_optimized_top smartnic_dut (
        .clk(clk_read),
        .rst(~rstn),
        .ip_packet(fifo_dout),
        .i_packet_valid(fifo_data_valid_d),
        .i_packet_id(packet_id_to_classifier),
        .valid(valid),
        .matched_rule_id(matched_rule_id),
        .o_ready_for_packet(consumer_ready),
        .o_result_done(result_done),
        .o_packet_id(result_packet_id)
    );

endmodule
```

Appendix B: Testbenches

High Throughput Design

```
`timescale 1ns/1ps
/*
 * File Name  : snic_throughput_cdc_testbench.v
 * Module Name: cdc_high_tp_tb
 * Author     : Uchenna Obikwelu
 * Course     : SYSC 4320 – Case Studies in Computer Systems
 * Project    : SmartNIC Packet Classification Accelerator
 * Created    : April 2026
 *
 * Description:
 * Testbench for the high-throughput SmartNIC design with asynchronous FIFO-
 * based clock domain crossing. This testbench feeds packets from memory into
 * the FIFO write domain and verifies packet classification in the read domain.
 *
 * Architecture:
 *   - Write domain: packet injection into async_fifo_top
 *   - Read domain : packet parsing and high-throughput classification
 *
 * Notes:
 *   - Uses separate write and read clocks to verify CDC behavior.
 *   - Instantiates async_fifo_top and smart_nic_high_throughput_top.
 *   - Used for functional simulation and verification.
 */

module cdc_high_tp_tb;

  parameter PTR_SIZE = 5;
  parameter WIDTH    = 76;
  parameter DEPTH    = 32;
  parameter NUM_PKTS = 124;

  // Clock and reset signals
  reg clk_write;
  reg clk_read;
```

```
reg rstn;

// Write-side stimulus signals
reg wr_en;
reg [WIDTH-1:0] ip_packet_in;

// FIFO outputs
wire [WIDTH-1:0] fifo_dout;
wire os_fifo_full;
wire os_fifo_empty;
wire [PTR_SIZE:0] ov_wr_pointer;
wire [PTR_SIZE:0] ov_rd_pointer;
wire [PTR_SIZE:0] ov_wr_pointer_gry;
wire [PTR_SIZE:0] ov_rd_pointer_gry;

// SmartNIC outputs
wire valid;
wire [3:0] matched_rule_id;

// Packet memory loaded from pck_stream.mem
reg [WIDTH-1:0] packet_mem [0:NUM_PKTS-1];
integer i;

// FIFO DUT
async_fifo_top #(
  .PTR_SIZE(PTR_SIZE),
  .WIDTH   (WIDTH),
  .DEPTH   (DEPTH)
) fifo_dut (
  .clk_write(clk_write),
  .clk_read (clk_read),
  .rstn     (rstn),
  .wr_en    (wr_en),
  .ip_packet_in(ip_packet_in),
  .ov_dout  (fifo_dout),
  .os_fifo_full(os_fifo_full),
  .os_fifo_empty(os_fifo_empty),
  .ov_wr_pointer(ov_wr_pointer),
  .ov_rd_pointer(ov_rd_pointer),
  .ov_wr_pointer_gry(ov_wr_pointer_gry),
  .ov_rd_pointer_gry(ov_rd_pointer_gry)
);
```

```
// SmartNIC DUT operating in the read clock domain
smart_nic_high_throughput_top smartnic_dut (
  .clk(clk_read),          // classifier runs in read domain
  .rst(~rstn),             // smartnic uses active-high reset
  .ip_packet(fifo_dout),
  .valid(valid),
  .matched_rule_id(matched_rule_id)
);

// Clocks
// write clock = 250 MHz (4 ns period)
// read clock  = 200 MHz (5 ns period)
initial begin
  clk_write = 0;
  forever #2 clk_write = ~clk_write;
end

initial begin
  clk_read = 0;
  forever #2.5 clk_read = ~clk_read;
end


// Load packet memory
initial begin
  $readmemh("pck_stream.mem", packet_mem);
end

// Reset sequence and packet injection stimulus
initial begin
  rstn = 0;
  wr_en = 0;
  ip_packet_in = 0;
  i = 0;

  #20;
  rstn = 1;

  // feed packets into FIFO
  while (i < NUM_PKTS) begin
    @(negedge clk_write);
    if (!os_fifo_full) begin
      wr_en = 1'b1;
      ip_packet_in = packet_mem[i];
```

```
      i = i + 1;
    end
    else begin
      wr_en = 1'b0;
    end
  end

  @(negedge clk_write);
  wr_en = 1'b0;

  // Allow FIFO contents to drain and classifier pipeline to settle
  #500;
  $finish;
 end

  integer cyc;
 initial cyc = 0;

 always @(posedge clk_read) begin
  if (rstn) begin
    cyc = cyc + 1;
    $display("time=%0t | cycle=%0d | fifo_empty=%0d | fifo_full=%0d | fifo_dout=%h |
valid=%0d | rule=%0d | wr_ptr=%0d | rd_ptr=%0d",
            $time, cyc, os_fifo_empty, os_fifo_full, fifo_dout, valid, matched_rule_id,
            ov_wr_pointer, ov_rd_pointer);
  end
 end

endmodule
```

Area-Optimized Design

```
/*
 * File Name  : snic_area_cdc_testbench.v
 * Module Name: cdc_area_tb
 * Author     : Ben Narmer
 * Course     : SYSC 4320 – Case Studies in Computer Systems
 * Project    : SmartNIC Packet Classification Accelerator
 * Created    : April 2026
 *
 * Description:
 * Testbench for the area-optimized SmartNIC design with asynchronous FIFO-
 * based clock domain crossing. This testbench feeds packets from memory into
```

```
 * the FIFO write domain and verifies packet classification in the read domain
 * using the sequential area-optimized classifier.
 *
 * Architecture:
 *   - Write domain: packet injection into async_fifo_top_area
 *   - Read domain : packet parsing and area-optimized classification
 *
 * Notes:
 *   - Uses separate write and read clocks to verify CDC behavior.
 *   - Instantiates async_fifo_top_area and smart_nic_area_optimized_top.
 *   - Includes packet-valid handshaking and packet ID tracking for result checking.
 *   - Used for functional simulation and verification.
 */

module cdc_area_tb;

  parameter PTR_SIZE = 5;
  parameter WIDTH    = 76;
  parameter DEPTH    = 32;
  parameter NUM_PKTS = 124;

  // clocks and reset signals
  reg clk_write;
  reg clk_read;
  reg rstn;

  // write-side stimulus
  reg wr_en;
  reg [WIDTH-1:0] ip_packet_in;

  // FIFO outputs
  wire [WIDTH-1:0] fifo_dout;
  wire os_fifo_full;
  wire os_fifo_empty;
  wire [PTR_SIZE:0] ov_wr_pointer;
  wire [PTR_SIZE:0] ov_rd_pointer;
  wire [PTR_SIZE:0] ov_wr_pointer_gry;
  wire [PTR_SIZE:0] ov_rd_pointer_gry;

  // SmartNIC handshake and classification outputs
  wire valid;
  wire consumer_ready;
  wire [3:0] matched_rule_id;
```

```
wire result_done;
wire [7:0] result_packet_id;

reg fifo_data_valid_d;
reg [7:0] packet_id_counter;
reg [7:0] packet_id_to_classifier;

// packet memory
reg [WIDTH-1:0] packet_mem [0:NUM_PKTS-1];
integer i;

// FIFO DUT
// Area-optimized asynchronous FIFO DUT with consumer-ready gating on the read side
async_fifo_top_area #(
  .PTR_SIZE(PTR_SIZE),
  .WIDTH   (WIDTH),
  .DEPTH   (DEPTH)
) fifo_dut (
  .clk_write(clk_write),
  .clk_read (clk_read),
  .rstn     (rstn),
  .wr_en    (wr_en),
  .ip_packet_in(ip_packet_in),
  .ov_dout  (fifo_dout),
  .os_fifo_full(os_fifo_full),
  .os_fifo_empty(os_fifo_empty),
  .ov_wr_pointer(ov_wr_pointer),
  .ov_rd_pointer(ov_rd_pointer),
  .ov_wr_pointer_gry(ov_wr_pointer_gry),
  .ov_rd_pointer_gry(ov_rd_pointer_gry),
  .i_consumer_ready(consumer_ready)
);

// SmartNIC DUT
// Area-optimized SmartNIC DUT operating in the read clock domain
smart_nic_area_optimized_top smartnic_dut (
  .clk(clk_read),
  .rst(~rstn),
  .ip_packet(fifo_dout),
  .i_packet_valid(fifo_data_valid_d),
  .i_packet_id(packet_id_to_classifier),
  .valid(valid),
  .matched_rule_id(matched_rule_id),
  .o_ready_for_packet(consumer_ready),
```

```
  .o_result_done(result_done),
  .o_packet_id(result_packet_id)
);


// Clocks
// write clock = 250 MHz (4 ns period)
// read clock  = 200 MHz (5 ns period)
initial begin
  clk_write = 0;
  forever #2 clk_write = ~clk_write;
end

initial begin
  clk_read = 0;
  forever #2.5 clk_read = ~clk_read;
end

// Generate packet-valid pulses and assign packet IDs when FIFO data is consumed
always @(posedge clk_read or negedge rstn) begin
  if (!rstn) begin
    fifo_data_valid_d   <= 1'b0;
    packet_id_counter   <= 8'd0;
    packet_id_to_classifier <= 8'd0;
  end else begin
    // delayed version of actual FIFO read
    fifo_data_valid_d <= (!os_fifo_empty && consumer_ready);

    // when FIFO performs a read, assign an id to that packet
    if (!os_fifo_empty && consumer_ready) begin
      packet_id_to_classifier <= packet_id_counter;
      packet_id_counter <= packet_id_counter + 1'b1;
    end
  end
end

// Load packet memory
initial begin
  $readmemh("pck_stream.mem", packet_mem);
end

// Stimulus
initial begin
  rstn = 0;
```

```
    wr_en = 0;
    ip_packet_in = 0;
    i = 0;

    #20;
    rstn = 1;

    // feed packets into FIFO
    while (i < NUM_PKTS) begin
      @(negedge clk_write);
      if (!os_fifo_full) begin
        wr_en = 1'b1;
        ip_packet_in = packet_mem[i];
        i = i + 1;
      end
      else begin
        wr_en = 1'b0;
      end
    end

    @(negedge clk_write);
    wr_en = 1'b0;

    // let FIFO drain + area-optimized classifier finish scanning
    #5000;
    $finish;
  end

  integer cyc;
  initial cyc = 0;

  // Per-cycle debug trace for FIFO and classifier activity
  always @(posedge clk_read) begin
    if (rstn) begin
      cyc = cyc + 1;
      $display("time=%0t | cycle=%0d | fifo_empty=%0d | fifo_full=%0d | fifo_dout=%h |
valid=%0d | rule=%0d | wr_ptr=%0d | rd_ptr=%0d",
               $time, cyc, os_fifo_empty, os_fifo_full, fifo_dout, valid, matched_rule_id,
               ov_wr_pointer, ov_rd_pointer);
    end
  end

  // Print a result message whenever the classifier finishes processing a packet
  always @(posedge clk_read) begin
```

```
    if (rstn && result_done) begin
      $display("RESULT -> time=%0t | packet_id=%0d | fifo_dout=%h | valid=%0d | rule=%0d",
               $time, result_packet_id, fifo_dout, valid, matched_rule_id);
    end
  end

endmodule
```

Appendix C: Constraints Files

High-Throughput Design:

```
##############################################################################
#####
# File Name  : smartnic_cdc_constraints.xdc
# Author     : Uchenna Obikwelu and Ben Narmer
# Course     : SYSC 4320 – Case Studies in Computer Systems
# Project    : SmartNIC Packet Classification Accelerator
# Created    : April 2026
#
# Description:
# Timing constraints for the SmartNIC Packet Classification Accelerator.
# This file defines clock constraints and clock-domain crossing (CDC)
# relationships for both the area-optimized and high-throughput designs.
#
# Notes:
# - The write and read clocks operate in independent domains.
# - Asynchronous clock groups prevent false timing violations.
# - The reset path is excluded from timing analysis.
# - Used in synthesis and implementation analysis.
##############################################################################
#####

# Write Clock (200 MHz → 5 ns period)
create_clock -name clk_write -period 5.000 [get_ports clk_write]

# Read Clock (≈166.67 MHz → 6 ns period)
create_clock -name clk_read -period 6.000 [get_ports clk_read]

# Declare asynchronous clock domains for CDC analysis
set_clock_groups -asynchronous \
```

```
-group [get_clocks clk_write] \
-group [get_clocks clk_read]

# Exclude asynchronous reset from timing analysis
set_false_path -from [get_ports rstn]
```

Area-Optimized Design

```
##############################################################################
#####
# File Name  : smartnic_cdc_constraints.xdc
# Author     : Uchenna Obikwelu and Ben Narmer
# Course     : SYSC 4320 – Case Studies in Computer Systems
# Project    : SmartNIC Packet Classification Accelerator
# Created    : April 2026
#
# Description:
# Timing constraints for the SmartNIC Packet Classification Accelerator.
# This file defines clock constraints and clock-domain crossing (CDC)
# relationships for both the area-optimized and high-throughput designs.
#
# Notes:
# - The write and read clocks operate in independent domains.
# - Asynchronous clock groups prevent false timing violations.
# - The reset path is excluded from timing analysis.
# - Used in synthesis and implementation analysis.
##############################################################################
#####

# Write Clock (200 MHz → 5 ns period)
create_clock -name clk_write -period 5.000 [get_ports clk_write]

# Read Clock (≈166.67 MHz → 6 ns period)
create_clock -name clk_read -period 6.000 [get_ports clk_read]

# Declare asynchronous clock domains for CDC analysis
set_clock_groups -asynchronous \
-group [get_clocks clk_write] \
-group [get_clocks clk_read]

# Exclude asynchronous reset from timing analysis
set_false_path -from [get_ports rstn]
```

Appendix D: AI Disclosure

AI Tool Used: ChatGPT (OpenAI)

In accordance with the course policy for SYSC 4320: Case Studies in Computer Systems, ChatGPT was used as a supplementary tool to support documentation, formatting, and minor coding assistance. All design decisions, implementation steps, simulations, FPGA deployment, and analytical evaluations were independently conceived, executed, and verified by the authors.

Scope of AI Assistance

ChatGPT was used in the following limited capacities:

- **Code Documentation and Formatting:** Generated and refined top-level comments for Verilog, C, and constraint files to improve readability and maintain professional documentation.

- **Testbench Output Formatting:** Assisted in formatting console outputs and improving syntax for simulation testbenches to ensure clarity and structured presentation of results.

- **Rule Table Preparation:** Provided support in structuring and formatting rule tables used in the SmartNIC packet classification design.

- **Report Writing Assistance**
  - Helped refine explanations for figures and technical sections.
  - Assisted in expanding text-heavy or repetitive explanations for clarity and coherence.
  - Supported proofreading, grammar correction, and formatting of the final report.

Verification and Responsibility

All AI-assisted outputs were thoroughly reviewed, validated, and modified by the authors to ensure correctness and alignment with project requirements. Every line of code and all analytical results were manually verified through simulation, synthesis, and FPGA implementation using Vivado and Vitis tools.

The authors assume full responsibility for the functional correctness, synthesis results, and integrity of the project, regardless of whether AI assistance was used.

Example Prompts Used

- “Generate professional header comments for a Verilog FPGA top-level module.”
- “Format UART console output for a SmartNIC classification testbench.”
- “Help structure a rule table for packet classification.”
- “Refine and expand explanations for FPGA timing, power, and resource utilization reports.”
- “Convert technical descriptions into clear, concise, detailed form.”

Appendix E: Project Contribution Breakdown

Section / Task	Uchenna Obikwelu	Ben Narmer
1. Introduction		✔️
2. Background	✔️	✔️
3. Architecture Design	✔️	
4. Version 1 – High-Throughput Design	✔️	
5. Version 2 – Area-Optimized Design		✔️
6. Functional Simulations	✔️	
7. Comparative Analysis		✔️
8. FPGA Implementation	✔️	
RTL Design – High-Throughput	✔️	
RTL Design – Area-Optimized		✔️
FPGA Implementation Code	✔️	
Report Writing (Overall)	✔️	✔️
Presentation	✔️	✔️